

Microcontrollers, Microprocessors & Computer Architecture

The Complete Guide - From Beginner to Expert

Transistors · Logic · Arithmetic · Instruction Sets · Pipelines · Branch
Prediction · Out-of-Order Execution · Caches · Virtual Memory · DRAM ·
Coherence · SoCs · Interconnect · Accelerators · Power · Security · RTL
Design

Contents	38 chapters in 7 parts, plus 3 appendices
Level	From 'what is a transistor' to designing and evaluating a pipelined processor
Style	Build the mechanism, then compute its cost, then measure it on a real machine
Worked examples	IEEE-754 encoded by hand, RISC-V instructions assembled bit by bit, a pipeline trace with hazards, cache address splits and AMAT, a page-table walk, DRAM latency and bandwidth, MESI transitions, Amdahl and roofline - all computed and checkable
Hardware used	RISC-V RV32I as the teaching ISA, ARM Cortex-M and Cortex-A for the real-world view, with x86-64 contrasts
Assumed	Programming in some language. No electronics background - Chapter 2 starts at the transistor.

How To Use This Book

Computer architecture is the study of how a machine that only knows voltage levels comes to run your program, and why one such machine is a hundred times faster - or a thousand times more efficient - than another. This book builds that machine from the bottom, then takes it apart from the top.

Every chapter follows the same three beats: **the mechanism**, drawn and explained so you could implement it; **the arithmetic**, because architecture is a quantitative discipline and every design choice has a number attached; and **the consequence for software**, because the reason to learn this is to write and reason about code that runs on real hardware.

The seven parts

Part	Chapters	What you get out of it
I - Digital Foundations	1-5	Transistors, gates, Boolean algebra, flip-flops, timing, memory cells, and arithmetic circuits - everything a processor is built from.
II - Instruction Set Architecture	6-10	What an ISA is and how one is encoded; RV32I in full, Thumb-2 and x86-64 contrasted; assembly, calling conventions and the ABI; exceptions and privilege.
III - Microarchitecture	11-17	Single-cycle and pipelined datapaths, hazards, branch prediction, superscalar and out-of-order execution, SIMD, multithreading, and how to measure performance honestly.
IV - The Memory System	18-23	Caches from first principles, AMAT and the three Cs, virtual memory and TLBs, DRAM organisation and timing, storage, coherence and memory consistency models.
V - Real Devices	24-29	What actually distinguishes a microcontroller from a microprocessor; core families; SoC integration; AMBA and NoC interconnect; interrupt controllers and DMA; how to choose a device.
VI - Physical and Modern	30-35	Power, clocking and thermal limits; reliability, ECC and test; security architecture from privilege rings to Spectre; accelerators, GPUs and NPUs; RTL design; and where the field is going.
VII - Practice	36-38	Using performance counters and simulators, writing cache- and SIMD-aware code, and one complete walkthrough of a single line of C from source to transistors.

Reading paths

- **Complete beginner:** 1 -> 2 -> 3 -> 4 -> 5, then 6 -> 7 -> 11 -> 12. Draw every circuit yourself; the diagrams are meant to be redrawn, not admired.
- **Programmer who wants to know what the machine does:** 6, 7, 12, 13, 18, 19, 20, 36 - the chapters that change how you write code.
- **Embedded engineer:** 5, 6, 8, 10, 24, 25, 27, 28, 30 - the microcontroller path, with just enough microarchitecture to reason about timing.
- **Preparing for an architecture course or interview:** Parts II and III in full, then 18-21 and 17 (performance). Do every worked example with a pencil.
- **Hardware designer:** all of Part I, then 11-15, and 34 (RTL, and the project of building a working CPU), plus 30 (power) and 31 (reliability and test).
- **Performance engineer:** 17, 18-23, 32, 36, 37 - measurement, memory, and vectorisation.

Conventions

Style	Meaning
0x2A, 0b1010, 42	Hexadecimal, binary and decimal
KB / MB / GB	Powers of two for memory (1 KB = 1024 B); powers of ten for transfer rates, as the industry does

Style	Meaning
RV32I	The 32-bit RISC-V base integer ISA, used for every teaching example because its encoding is small enough to do by hand
Cycle, ns, CPI	Timing is always given in both cycles and seconds, because only one of them survives a frequency change
Coloured boxes	KEY IDEA = the one thing to remember. MATH = an arithmetic derivation you can check. INTUITION = the mental picture. PRACTICAL TIP = what to do with this. PITFALL = a misconception that causes real errors. EXPERT CORNER = depth you can skip on a first read.

THE QUANTITATIVE PRINCIPLE

Architecture has no absolutes, only trade-offs measured against a workload. Every claim in this book that something is 'better' is accompanied by the number that makes it so - cycles, joules, millimetres of silicon, or dollars. When you meet a new design, the first question is always: **compared with what, on what workload, measured how?**

Table of Contents

PART I - Digital Foundations	10
1. What a Computer Is	11
1.1 The stored-program idea	11
1.2 The layers, and why each exists	11
1.3 Microprocessor, microcontroller, SoC	12
1.4 How to read the rest of this book	12
2. Transistors, Gates and Combinational Logic	14
2.1 A bit is a voltage	14
2.2 The MOSFET as a switch, and why CMOS won	14
2.3 Gates, and the one that is really primitive	14
2.4 Boolean algebra and minimisation	15
2.5 The blocks a datapath is made of	15
3. Sequential Logic, Clocks and Timing	17
3.1 From feedback to a flip-flop	17
3.2 The three timing numbers, and the one equation	17
3.3 Finite state machines	17
3.4 Metastability and crossing clock domains	18
3.5 Registers, counters and the pipelining idea	19
4. Computer Arithmetic	20
4.1 Representing integers	20
4.2 Adders: the critical path of the machine	20
4.3 The ALU and its flags	21
4.4 Multiplication and division	21
4.5 Floating point, and what every programmer should know	21
5. Memory Building Blocks	23
5.1 The storage cells	23
5.2 How an array is organised	23
5.3 Register files and the cost of ports	24
5.4 Content-addressable memory and other structures	24
PART II - Instruction Set Architecture	26
6. What an Instruction Set Architecture Is	27
6.1 What the specification must contain	27
6.2 The design space	27
6.3 RISC and CISC, forty years on	27
6.4 Encoding: the engineering under the elegance	28
6.5 Four ways to compute one expression	28
6.6 How much can one instruction encode?	29

7. RISC-V RV32I, Encoded by Hand	30
7.1 The register model	30
7.2 The six formats	30
7.3 The instruction set itself	31
7.4 The extensions, and why the ISA is modular	31
8. Assembly, the ABI and What the Compiler Emits	33
8.1 The calling convention	33
8.2 A stack frame, drawn and built	33
8.3 How C constructs become instructions	34
8.4 From object files to a running program	34
9. ARM, x86-64 and the Wider World of ISAs	36
9.1 ARM: two architectures in one family	36
9.2 x86-64: complexity that pays for compatibility	36
9.3 The rest of the landscape	36
9.4 What a modern x86 core does with a CISC instruction	37
10. Exceptions, Privilege and the System Interface	39
10.1 A taxonomy that avoids confusion	39
10.2 The mechanism, on RISC-V	39
10.3 System calls, and what they cost	40
10.4 Interrupts as an architectural feature	40
10.5 Virtualisation, briefly	40
PART III - Microarchitecture	42
11. Building a Datapath: The Single-Cycle Processor	43
11.1 The five things every instruction needs	43
11.2 The datapath, instruction by instruction	43
11.3 Control: a truth table with a name	43
11.4 Why nobody builds it this way	44
11.5 The multi-cycle alternative	44
11.6 Control as microcode	44
12. Pipelining and Hazards	46
12.1 The idea, and its arithmetic	46
12.2 The three hazards	46
12.3 Data hazards and forwarding, traced	46
12.4 Control hazards and the cost of depth	47
12.5 What else the pipeline must handle	47
13. Branch Prediction and Speculation	49
13.1 Two questions, two predictors	49
13.2 From one bit to gigabytes of history	49
13.3 Speculation: acting on the guess	50
14. Instruction-Level Parallelism and Superscalar Issue	51

14.1 Measuring the parallelism in a piece of code	51
14.2 In-order superscalar	51
14.3 What the compiler can do about it	51
14.4 VLIW: give the whole job to the compiler	52
14.5 How much ILP is actually there?	52
15. Out-of-Order Execution	53
15.1 Why it exists: hiding memory	53
15.2 Register renaming: removing false dependences	53
15.3 The structures of a modern core	53
15.4 Loads, stores and speculation	54
15.5 What it costs, and the alternative	54
16. Data and Thread Parallelism: SIMD, Vectors, Threads and GPUs	55
16.1 SIMD: one instruction, many elements	55
16.2 Vector architectures: length-agnostic SIMD	55
16.3 Thread-level parallelism	56
16.4 GPUs, as an architecture rather than a product	56
17. Measuring Performance Without Fooling Yourself	58
17.1 The iron law	58
17.2 Amdahl, and what parallelism can and cannot do	58
17.3 Benchmarks, and their failure modes	58
17.4 Top-down analysis with performance counters	59
17.5 The roofline model	59
17.6 Measuring honestly	59
PART IV - The Memory System	61
18. Caches from First Principles	62
18.1 Why caches work at all	62
18.2 Anatomy: how an address becomes a hit	62
18.3 Writes, and the policies that matter	63
18.4 What a miss actually costs	63
18.5 How the parameters trade against each other	63
18.6 Caches you can see: microcontrollers and scratchpads	64
19. Cache Performance and Optimisation	65
19.1 Average memory access time	65
19.2 The three Cs, and what to do about each	65
19.3 The hardware techniques, and what each buys	65
19.4 What software can do	66
20. Virtual Memory and the TLB	67
20.1 What translation buys	67
20.2 Page tables and the walk	67
20.3 The TLB: a cache for translations	67

20.4 Where the TLB sits relative to the cache	68
20.5 Context switches, ASIDs and virtualisation	68
21. DRAM and the Memory Controller	70
21.1 The structure, from chip to system	70
21.2 What the memory controller does	70
21.3 Address mapping and bank parallelism	71
21.4 Commands, in the order they happen	71
22. Storage and the Memory-Storage Boundary	73
22.1 The full hierarchy, with numbers	73
22.2 Inside a flash SSD	73
22.3 Interfaces, and why NVMe changed things	74
23. Cache Coherence and Memory Consistency	75
23.1 Coherence: keeping caches honest	75
23.2 Atomics, and what they cost	75
23.3 Consistency: what orderings software may observe	76
23.4 Scaling: NUMA and the limits of sharing	76
PART V - Real Devices	77
24. Inside a Microcontroller	78
24.1 The block diagram, and what each block costs	78
24.2 Deterministic execution, and what it is worth	78
24.3 The memory map is the architecture	79
25. Core Families: What You Can Actually Buy	80
25.1 The ARM Cortex families	80
25.2 RISC-V profiles	80
25.3 The rest of the field	80
26. Systems on Chip: Application Processors	82
26.1 A representative floorplan	82
26.2 The boot chain	82
27. Interconnect: Buses, Fabrics and Networks on Chip	84
27.1 The AMBA family, in order of complexity	84
27.2 From buses to networks on chip	84
27.3 Budgeting bandwidth across a system	85
28. Interrupts, DMA and Platform Peripherals as Architecture	86
28.1 Interrupt controllers compared	86
28.2 DMA as a second processor	86
28.3 Timers and the sense of time	87
29. Choosing a Device: MCU, MPU, DSP, FPGA or ASIC	88
29.1 The five options	88
29.2 A decision procedure that works	88

29.3 A selection worked through	88
---	----

PART VI - Physical and Modern **90**

30. Power, Clocking and Thermal Limits	91
30.1 The two power terms	91
30.2 The techniques, and what each saves	91
30.3 Clocking a large chip	92
30.4 Heat, and why it is the real limit	92
31. Reliability, Error Correction and Test	93
31.1 How things go wrong	93
31.2 Detection and correction	93
31.3 Testing a chip you cannot probe	94
32. Security Architecture	95
32.1 The mechanisms, from oldest to newest	95
32.2 Side channels: when performance features leak	95
32.3 Building a trustworthy device	96
33. Accelerators: GPUs, NPU, DSPs and Custom Silicon	97
33.1 Where the hundred-fold gap comes from	97
33.2 The families	97
33.3 The systolic array, in one page	97
34. RTL Design and the Path to Silicon	99
34.1 RTL is registers plus logic, and nothing else	99
34.2 The flow, from text to silicon	99
34.3 Higher-level ways in	100
35. Where Computer Architecture Is Going	102
35.1 The forces	102
35.2 What is actually being built	102
35.3 What will still be true in twenty years	102

PART VII - Practice **104**

36. Tools: Counters, Profilers and Simulators	105
36.1 Performance counters	105
36.2 Profiling strategies	105
36.3 A top-down analysis, walked through	106
36.4 Microbenchmarks that tell you about your machine	106
37. Writing Code That Fits the Machine	108
37.1 A worked optimisation, step by step	108
37.2 The checklist, by chapter	108
37.3 The transformations, in code	109
38. One Line of C, All the Way Down	110
38.1 Layer 1: the compiler	110

38.2 Layer 2: the instruction stream	110
38.3 Layer 3: the core	110
38.4 Layer 4: the memory system	110
38.5 Layer 5: the arithmetic, the gates, the transistors	111
38.6 The accounting	111
38.7 Where to go next	111

APPENDICES **113**

Appendix A. Formula and Reference Sheet	115
Performance	115
Memory hierarchy	115
Logic, timing and power	115
Number representation	116
RV32I encoding	116
Latency and energy, orders of magnitude	117
Coherence and consistency	117
Appendix B. Glossary	118
Appendix C. Study Roadmap, Projects and Tools	120
A 16-week study plan	120
Projects, in increasing order of ambition	120
Tools worth knowing	120
How to keep learning	121

PART I

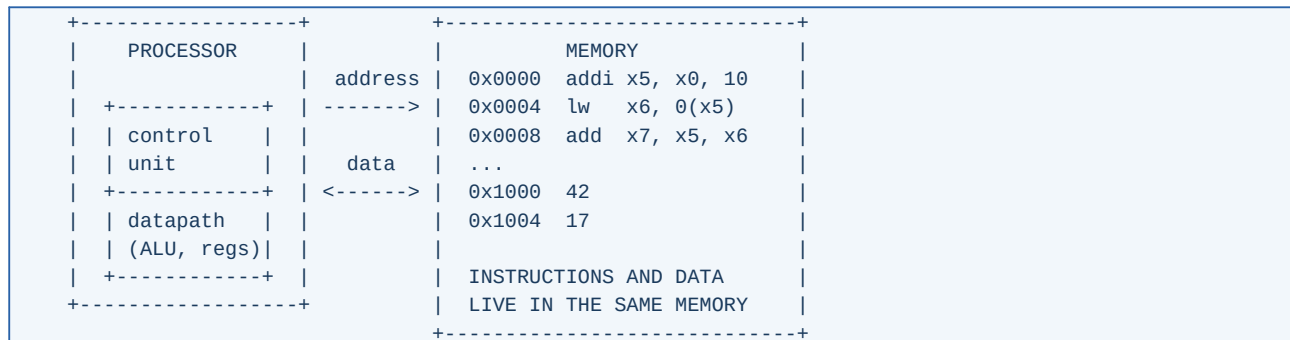
Digital Foundations

How a voltage becomes a bit, a bit becomes a gate, a gate becomes arithmetic and memory, and why a clock is what makes any of it reliable. Everything in the rest of the book is built from the five chapters in this part.

1. What a Computer Is

A computer is a machine that holds a description of a computation in the same memory as the data it operates on, and then carries it out one step at a time, faster than anything else humans have built. Every device in this book - an 8-bit microcontroller costing twenty cents and a server processor costing ten thousand dollars - is that same idea, differing only in how much work it does per second and per joule.

1.1 The stored-program idea



The von Neumann organisation. Its one radical claim - that a program is just data in memory - is why a computer can be reprogrammed, why a compiler can exist, and why a buffer overflow can execute.

Element	Role	Where it appears later
Program counter	Holds the address of the next instruction	Chapters 11-13
Instruction memory	Where code lives	Caches (Ch 18), flash (Ch 24)
Register file	A few dozen very fast named storage locations	Chapters 5, 11
ALU	Computes arithmetic and logic results	Chapter 4
Data memory	The large, slower store	Chapters 18-21
Control	Decides what every part does each cycle	Chapters 11-12

VON NEUMANN AND HARVARD

A **Harvard** machine has separate instruction and data memories, so it can fetch an instruction and a datum in the same cycle. Most microcontrollers are Harvard internally (flash for code, SRAM for data) while presenting one address space to the programmer; most application processors are von Neumann in their address space with a split L1 instruction and data cache, which recovers the same bandwidth advantage. The distinction is about the **memory ports**, not about philosophy.

1.2 The layers, and why each exists

application	your program	
-----	-----	compiler
high-level language	C, Rust, Python	
-----	-----	compiler/assembler
assembly / machine code	add x7, x5, x6	
-----	-----	THE ISA - the
instruction set architecture	hardware/software contract	
-----	-----	implementation
microarchitecture	pipeline, caches, predictors	
-----	-----	RTL synthesis
register-transfer logic	Verilog: registers and combinational logic	
-----	-----	standard cells
gates	AND, OR, NOT, flip-flops	
-----	-----	physics
transistors	switches made of doped silicon	

Each line is an interface that lets the layers above ignore how the layers below work. The ISA is the most important of them: it is a contract, and everything below it may be redesigned freely as long as the contract holds.

ARCHITECTURE VERSUS MICROARCHITECTURE

Architecture (the ISA) is what a programmer can observe: the instructions, the registers, the addressing modes, the memory model. **Microarchitecture** is how a particular chip implements it: how deep the pipeline is, how big the caches are, whether it executes out of order. Two chips with the same architecture run the same binaries; their microarchitectures may differ by a factor of fifty in performance and a factor of a thousand in power. Confusing the two is the most common error in discussions of processors.

1.3 Microprocessor, microcontroller, SoC

	Microprocessor (MPU)	Microcontroller (MCU)
What is on the chip	A CPU (plus caches, and today a GPU and controllers)	CPU plus flash, SRAM, timers, ADC, serial ports, everything
Memory	External DRAM, gigabytes	On-chip, kilobytes to a few megabytes
Boot	A firmware chain from external storage	Runs from internal flash within microseconds
Typical clock	1-5 GHz	8-600 MHz
Determinism	Low: caches, virtual memory, an OS scheduler	High: often cycle-countable
Power	1-200 W	microwatts to a few hundred milliwatts
Unit cost	Tens to thousands of dollars	Cents to a few dollars
Runs	Linux, Windows, Android	Bare metal or an RTOS

The line between them has blurred from both directions: microcontrollers now have caches, DSP and vector units and run at hundreds of megahertz, while application processors are sold as **systems on chip** with dozens of integrated peripherals. What still separates them is the memory system - **on-chip and deterministic against off-chip and cached** - and that single difference drives everything from boot time to the programming model, which is why it is the organising distinction of Part V.

1.4 How to read the rest of this book

Chapters 2 to 5 build the hardware vocabulary; if you already know logic design, skim them for the timing and memory-cell sections and move on. Part II defines the interface between hardware and software, and Part III is the heart of the subject: how a modern processor extracts performance from a sequential program. Part IV is where most real performance problems live, and Part V is where the abstractions meet the devices you can actually buy.

DO THE ARITHMETIC

Every worked example in this book is small enough to check with a pencil, and doing so is the difference between recognising a concept and understanding it. Encoding one RISC-V instruction by hand teaches more about ISA design than reading three chapters about it, and computing one AMAT teaches more about caches than any diagram.

Exercises

- 1 List the layers in the diagram above for a program you have written, naming the actual tool or component at each level.
- 2 Find the datasheet of one microcontroller and one application processor and fill in the comparison table with their real numbers.
- 3 Explain, in two sentences each, the difference between architecture and microarchitecture, and between von Neumann and Harvard.
- 4 Estimate how many instructions your laptop executes per second. Then look up its clock and core count and see how close you were.

2. Transistors, Gates and Combinational Logic

Everything in a processor is built from one component used one way: a transistor acting as a voltage-controlled switch. This chapter goes from that switch to the arithmetic and selection circuits that a datapath is made of, and introduces the two costs that every architectural decision is eventually paid in - **delay** and **area**.

2.1 A bit is a voltage

A digital circuit agrees to treat voltages near ground as 0 and voltages near the supply as 1, with a forbidden band between them. That convention is what buys noise immunity: a signal can be corrupted by hundreds of millivolts and still be restored to a clean level by the next gate, which is why digital systems can be built to arbitrary size and analogue ones cannot.

A 3.3 V CMOS input typically guarantees:

$V_{IL(max)} = 0.8 \text{ V}$ anything below this reads as 0

$V_{IH(min)} = 2.0 \text{ V}$ anything above this reads as 1

0.8 V to 2.0 V undefined - the output may be anything

Noise margins: $NM_{low} = V_{IL(max)} - V_{OL(max)}$

$NM_{high} = V_{OH(min)} - V_{IH(min)}$

A gate outputs a stronger level than it is required to accept; the difference is the margin that lets noise be absorbed rather than accumulated.

2.2 The MOSFET as a switch, and why CMOS won

NMOS: conducts when the gate is HIGH

PMOS: conducts when LOW

```

      drain
      |
gate --+
      |
      source = GND
  
```

```

      source = Vdd
      |
gate --o
      |
      drain
  
```

CMOS inverter:

```

      Vdd
      |
in --o| PMOS
      +---- out
in --| NMOS
      |
      GND
  
```

in=0 -> PMOS on, NMOS off -> out=1
in=1 -> PMOS off, NMOS on -> out=0

Exactly ONE path conducts at a time, so no current flows from Vdd to GND except briefly while switching. That is why CMOS replaced everything else: static power is (almost) zero.

Complementary MOS: an NMOS network pulls the output down, a PMOS network pulls it up, and the two are never on together in the steady state.

WHERE THE ENERGY GOES

Switching a gate charges and discharges its load capacitance, costing $C V^2$ of energy per full cycle, so dynamic power is $P = \alpha C V^2 f$, where α is the fraction of gates that switch. That single equation explains voltage scaling, clock gating, frequency scaling, and why architects care about how many bits toggle - and it returns as the central formula of Chapter 30. Leakage - current through a nominally-off transistor - adds a static term that grew from negligible to roughly a third of total power as transistors shrank, which is why 'turn it off' beats 'slow it down' on modern silicon.

2.3 Gates, and the one that is really primitive

Gate	Transistors (CMOS)	Note
NOT	2	The building block of restoring logic
NAND	4	The cheapest useful gate - PMOS in parallel, NMOS in series
NOR	4	Same count, but slower: PMOS in series is weak
AND	6	NAND followed by NOT - AND is *more* expensive than NAND
OR	6	NOR followed by NOT
XOR	8-12	Expensive, and it is on the critical path of every adder
2:1 multiplexer	~6 (or 4 with pass gates)	The most-used structure in a datapath

NAND and NOR are each **universal**: any Boolean function can be built from either alone. Real designs are not drawn gate by gate - a synthesis tool maps your description onto a **standard cell library** of a few hundred pre-characterised cells - but the counts above still explain why an architect avoids XOR chains and loves multiplexers.

2.4 Boolean algebra and minimisation

Identity	$A + 0 = A$	$A \cdot 1 = A$
Null	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotent	$A + A = A$	$A \cdot A = A$
Complement	$A + A' = 1$	$A \cdot A' = 0$
Distributive	$A(B + C) = AB + AC$	$A + BC = (A+B)(A+C)$
Absorption	$A + AB = A$	$A(A + B) = A$
De Morgan	$(AB)' = A' + B'$	$(A + B)' = A' \cdot B'$

De Morgan's laws are the ones you use daily: they turn an AND of inverted signals into a NOR, which is how a netlist gets cheaper without changing behaviour.

MINIMISING A FUNCTION BY HAND

$F(A,B,C) = A'BC + AB'C + ABC' + ABC$. Group the last two: $ABC' + ABC = AB$. Group $A'BC + ABC$ (reusing ABC , which is legal since $X + X = X$): BC . Group $AB'C + ABC$: AC . So **$F = AB + BC + AC$** - the majority function, true when at least two inputs are true. The naive form needs four 3-input ANDs and a 4-input OR (about 26 transistors in CMOS); the minimised form needs three 2-input ANDs and a 3-input OR (about 18). **A third of the area and one gate less delay, from algebra alone** - and this is exactly what a synthesis tool does, millions of times, in a modern design.

2.5 The blocks a datapath is made of

Block	Function	Where it is used
Multiplexer (n:1)	Selects one of n inputs	Everywhere: forwarding paths, ALU input selection, register write sources
Decoder (n to 2^n)	One-hot output from a binary code	Register-file write enables, memory row selection, instruction decode
Encoder / priority encoder	Binary code from a one-hot input	Interrupt controllers (Chapter 28), free-list allocation
Comparator	Equality or magnitude	Branch conditions, tag compare in a cache (Chapter 18)
Barrel shifter	Shifts by any amount in one pass	Shift instructions, floating-point normalisation
Adder	The heart of the ALU	Chapter 4

DELAY THROUGH A CHAIN, AND WHY THE CRITICAL PATH RULES

Suppose each 2-input gate has 50 ps of delay. A 4:1 multiplexer built as a tree of two 2:1 stages costs 100 ps; built as one flat AND-OR structure with 4-input gates it may cost 80 ps but with larger, slower gates it can be worse. Now chain 32 such stages - a naive 32-bit ripple structure - and the delay is $32 \times 50 \text{ ps} = \mathbf{1.6 \text{ ns}}$, which caps the clock at 625 MHz **no matter how fast everything else is**. The longest path between two registers is the **critical path**, and the entire craft of high-frequency design is finding it and shortening it, either with better logic (Chapter 4's carry-lookahead) or by cutting it with a register (Chapter 12's pipelining).

Exercises

- 1 Draw the CMOS transistor networks for a 2-input NAND and a 2-input NOR and explain why NOR is the slower of the two.
- 2 Prove both De Morgan laws with truth tables, then use one to convert an AND-OR circuit into a NAND-only circuit.
- 3 Minimise $F(A,B,C,D) = \text{sum of minterms } 0,2,5,7,8,10,13,15$ with a Karnaugh map and count the gates you saved.
- 4 Build a 4:1 multiplexer from 2:1 multiplexers, and compute its delay in gate delays.
- 5 Compute the dynamic power of a block with 100,000 gates, 10% activity factor, 5 fF load per gate, at 1.0 V and 500 MHz. Then repeat at 0.8 V and explain the difference.

3. Sequential Logic, Clocks and Timing

Combinational logic computes; sequential logic **remembers**, and memory is what makes a machine able to take one step after another. The price of memory is timing: signals must be stable at the right moment, and almost every hard bug in digital hardware is a violation of that requirement.

3.1 From feedback to a flip-flop

Cross-coupled inverters hold a bit:

```
+--->0--+
|         |
+---0<---+
```

Two stable states: 0 and 1.
Adding write control gives a latch;
two latches in series give a flop.

Edge-triggered D flip-flop:

```
D --| master |--| slave |-- Q
    (latch)   (latch)
clk --+-----0-----+
```

The master captures while clk
is low, the slave passes it on
the rising edge - so Q changes
exactly once per clock edge.

A latch is level-sensitive (transparent while enabled); a flip-flop is edge-triggered. Modern synchronous design uses flip-flops almost exclusively, because reasoning about a single instant per cycle is vastly simpler than reasoning about a window.

3.2 The three timing numbers, and the one equation

t_{cq} clock-to-Q: how long after the edge Q is valid

t_{setup} data must be stable this long BEFORE the edge

t_{hold} data must stay stable this long AFTER the edge

Setup (maximum-delay) constraint:

$$T_{\text{clk}} \geq t_{\text{cq}} + t_{\text{logic(max)}} + t_{\text{setup}} + t_{\text{skew}}$$

Hold (minimum-delay) constraint:

$$t_{\text{cq}} + t_{\text{logic(min)}} \geq t_{\text{hold}} + t_{\text{skew}}$$

The first sets your maximum frequency. The second does not depend on frequency at all - a hold violation cannot be fixed by slowing the clock down, which is what makes it the more dangerous of the two.

COMPUTING THE MAXIMUM CLOCK FREQUENCY

A pipeline stage with t_{cq} = 100 ps, worst-case combinational delay 1,200 ps, t_{setup} = 80 ps and clock skew 50 ps requires T_{clk} ≥ 100 + 1200 + 80 + 50 = **1,430 ps**, giving f_{max} = 1/1.43 ns = **699 MHz**. To reach 1 GHz (1,000 ps) the logic must be cut to 1000 - 100 - 80 - 50 = **770 ps** - a 36% reduction, which usually means splitting the stage in two and adding a pipeline register (Chapter 12). Notice that the flip-flop overhead - t_{cq} plus t_{setup} plus skew, here 230 ps - is paid **every stage**: pipelining deeper gives diminishing returns and eventually none, which is why processor pipelines stopped getting longer after about 20 stages.

3.3 Finite state machines

```

      1      0      1      1
[S0] ---> [S1] ---> [S2] ---> [S3: detected 1011]
  ^ \      ^ |0      | 1      | (back to S1 on 1, S0 on 0)
  | \0      | v      v
+---+---+---+---+

```

Moore machine: output depends on the STATE only -> registered,
glitch-free, one cycle of latency

Mealy machine: output depends on state AND input -> faster response,
but the output is combinational and can glitch

A sequence detector for the pattern 1011. Every control unit in this book - the instruction decoder in Chapter 11, the cache controller in Chapter 18, the coherence protocol in Chapter 23 - is a finite state machine drawn exactly like this.

```

// The standard three-block FSM in Verilog: state register,
// next-state logic, output logic. Keeping them separate is what
// makes an FSM readable and synthesisable without surprises.
always @(posedge clk or negedge rst_n)
    if (!rst_n) state <= S0; else state <= next;

always @(*) begin
    // pure combinational
    case (state)
        S0: next = in ? S1 : S0;
        S1: next = in ? S1 : S2;
        S2: next = in ? S3 : S0;
        S3: next = in ? S1 : S2;
        default: next = S0; // never omit this
    endcase
end

assign detected = (state == S3); // Moore output

```

3.4 Metastability and crossing clock domains

If a signal changes within the setup/hold window, the flip-flop can enter a **metastable** state - an output hovering between levels for an unbounded time. This is not a rare theoretical concern: it happens whenever an asynchronous input (a button, a signal from another clock domain) is sampled, which is always.

$$MTBF = \exp(t_r / \tau) / (T_0 \times f_{clk} \times f_{data})$$

t_r time allowed for the flop to resolve (about one period)
 τ the flop's resolution time constant (tens of ps)
 T_0 a process constant (tens to hundreds of ps)
 f_{clk} sampling frequency, f_{data} rate of asynchronous edges

The exponential is the whole story: every extra nanosecond of settling time multiplies the mean time between failures enormously.

WHY ONE FLIP-FLOP IS NOT ENOUGH, IN NUMBERS

Take $\tau = 50$ ps, $T_0 = 100$ ps, a 100 MHz clock and 1 million asynchronous edges per second. With **1 ns** of settling time: $MTBF = \exp(20) / (1e-10 \times 1e8 \times 1e6) = 4.85e8 / 1e4 = 48,500$ seconds = **13 hours**. A product that glitches twice a day is unshippable. Add a **second** flip-flop, giving the first a full extra cycle - about 5 ns of settling: $\exp(100) = 2.7e43$, so $MTBF = 2.7e39$ seconds, which is longer than the age of the universe by 22 orders of magnitude. **That is why the two-flop synchroniser is a universal rule** and why it is never optional, no matter how slow the signal seems.

Clock-domain crossing rules

- ☐ Every asynchronous single-bit signal passes through two flip-flops in the destination domain before use.
- ☐ Multi-bit values never cross bit by bit: use a handshake, a Gray-coded pointer, or an asynchronous FIFO.

- ☐ Gray coding is used for counters that cross, so only one bit changes per step.
- ☐ A synchronised signal is used in exactly one place; re-synchronising the same source twice can produce two different answers.
- ☐ Reset is asserted asynchronously if needed but always **released** synchronously to each domain.
- ☐ The static timing analysis has the crossings declared as false paths deliberately, not by accident.

3.5 Registers, counters and the pipelining idea

Put a register between two blocks of logic and you have halved the critical path - each half now has a full clock period - at the cost of one extra cycle of latency. Throughput doubles; latency rises slightly. That trade, made at the level of two flip-flops here, is exactly the trade a processor pipeline makes in Chapter 12, and the same one a DMA engine or a network link makes elsewhere. It is worth recognising as one idea rather than three.

Unpipelined: $\text{throughput} = 1 / (t_A + t_B)$ $\text{latency} = t_A + t_B$

Pipelined: $\text{throughput} = 1 / (\max(t_A, t_B) + t_{\text{reg}})$

$\text{latency} = 2 \times (\max(t_A, t_B) + t_{\text{reg}})$

Balanced stages matter: the slowest stage sets the clock for all.

Exercises

- 1 Draw a D flip-flop from two D latches and explain what happens if the clock has a slow edge.
- 2 Compute f_{max} for a stage with $t_{\text{cq}} = 60$ ps, logic = 900 ps, $t_{\text{setup}} = 70$ ps and skew = 40 ps, then find the logic delay needed for 1.5 GHz.
- 3 Explain why a hold violation cannot be fixed by lowering the clock frequency, and name two things that can fix it.
- 4 Compute the metastability MTBF for your own numbers at 1 ns and at 2 ns of settling; note how many orders of magnitude one nanosecond buys.
- 5 Design a Moore FSM that detects the sequence 110 and write it in the three-block style above.

4. Computer Arithmetic

The ALU is where a processor earns its name, and its design sets both the critical path and a large part of the power budget. This chapter covers how numbers are represented, how the circuits that operate on them are built, and why some operations cost one cycle and others forty.

4.1 Representing integers

Unsigned n bits: $0 \dots 2^n - 1$
 Two's complement: $-2^{(n-1)} \dots 2^{(n-1)} - 1$
 negate: invert all bits and add 1
 value: $-b_{(n-1)} 2^{(n-1)} + \sum_{i<n-1} b_i 2^i$
 8-bit examples: $0000_0101 = +5$ $1111_1011 = -5$
 $1000_0000 = -128$ $0111_1111 = +127$
 Sign extension: copy the top bit leftwards (needed on every load
 of a narrow signed value, Chapter 6)

Two's complement wins because addition, subtraction and comparison use **the same circuit** as unsigned - only the overflow test differs. That is the whole reason it displaced sign-magnitude.

OVERFLOW, SIGNED AND UNSIGNED, ARE DIFFERENT TESTS

Add $0x70 + 0x20$ in 8 bits: the result is $0x90$. **Unsigned**: $112 + 32 = 144$, which fits in 8 bits, and there is no carry out - no unsigned overflow. **Signed**: $+112 + +32 = +144$, but $0x90$ as two's complement is **-112** - a signed overflow, detected because two positive operands produced a negative result (equivalently, the carry into the sign bit differs from the carry out of it). The hardware computes both conditions and stores them as separate flags (C and V); it is the **instruction** - or the compiler - that decides which one matters. This is also why signed overflow is undefined behaviour in C while unsigned wraps: the language exposes the difference.

4.2 Adders: the critical path of the machine

Ripple-carry (n stages, delay $\sim n$):

```

a0 b0      a1 b1      a2 b2      a3 b3
| |        | |        | |        | |
[FA] --c1->[FA] --c2->[FA] --c3->[FA] --> c4
s0         s1         s2         s3

```

Carry-lookahead: compute all carries in parallel from

```

generate  g_i = a_i . b_i      (this bit makes a carry)
propagate p_i = a_i XOR b_i    (this bit passes one along)
c_(i+1) = g_i + p_i c_i

```

Expanding gives every carry as a 2-level function of the inputs:

delay $\sim \log(n)$ instead of n , at the cost of area.

Every high-performance adder - carry-select, carry-skip, Kogge-Stone and the other parallel-prefix forms - is a different point on the same area/delay curve.

Adder	Delay (32-bit, gate delays)	Relative area
Ripple-carry	~ 64	1.0 - the smallest
Carry-select	~ 16	~ 1.8
Carry-lookahead (4-bit groups)	~ 10	~ 1.6
Kogge-Stone (parallel prefix)	~ 7	~ 2.5 , plus heavy wiring

The numbers are approximate and technology-dependent, but the shape is universal: **an order of magnitude of delay is available for roughly 2x the area**, and a processor pays it happily because the adder is on the critical path of address generation, branch resolution and every arithmetic instruction.

4.3 The ALU and its flags

Flag	Set when	Used by
Z (zero)	Result is all zeros	<code>beq</code> , <code>bne</code> , equality tests
N (negative)	Top bit of the result is 1	Signed comparisons
C (carry)	Carry out of the top bit (or no borrow)	Unsigned comparisons, multi-word arithmetic
V (overflow)	Carry into the sign bit differs from carry out	Signed overflow detection, saturating arithmetic

RISC-V HAS NO FLAGS, AND THAT IS A DESIGN DECISION

ARM, x86 and most CISC machines keep condition flags in a status register; RISC-V deliberately does not, using compare-and-branch instructions instead (`blt x5, x6, label`). Flags create an implicit dependency between almost every pair of instructions, which is awkward for out-of-order execution (Chapter 15) - the renaming logic must track the flags register as a hot resource. The cost of removing them is slightly larger code and a busier branch instruction; the benefit shows up in the complexity of the machine that runs it.

4.4 Multiplication and division

Shift-and-add: n cycles for n bits (the classic small implementation)

Array multiplier: n^2 adders, one result per cycle, large area

Wallace/Dadda tree: reduces partial products in $\log(n)$ depth

Booth encoding: halves the number of partial products by treating runs of 1s as (a shifted subtract, a shifted add)

Typical costs:

32x32 multiply	1 cycle throughput, 3-5 latency
32/32 divide	10-40 cycles, often not pipelined
Cortex-M0	multiply 1 or 32 cycles (option), divide: none - a library call

This is why compilers replace division by a constant with a multiply and a shift, and why a divide in an inner loop is worth eliminating even at the cost of some algebra.

BOOTH'S TRICK, SEEN ONCE

Multiplying by 0b0111 (7) naively adds three shifted copies. Booth sees the run of ones and rewrites 7 as $8 - 1$, so the product is $(x \ll 3) - x$: **one shift and one subtract instead of three adds**. Modern radix-4 Booth encoding examines overlapping 3-bit windows and halves the partial-product count for any input, which halves the adder tree - the same idea, applied systematically.

4.5 Floating point, and what every programmer should know

IEEE-754 binary32 (float): 1 sign | 8 exponent | 23 fraction

value = $(-1)^s \times 1.\text{fraction} \times 2^{(\text{exponent} - 127)}$

binary64 (double): 1 | 11 | 52, bias 1023

Special encodings:

exponent all 0s, fraction 0 -> +/- zero

exponent all 0s, fraction != 0 -> subnormal (gradual underflow)

exponent all 1s, fraction 0 -> +/- infinity

exponent all 1s, fraction != 0 -> NaN

ENCODING -6.25 BY HAND

Sign: negative, so $s = 1$. Magnitude: $6.25 = 110.01$ in binary $= 1.1001 \times 2^2$, so the exponent field is $2 + 127 = 129 = \mathbf{0b10000001}$. The fraction is the bits after the leading 1: 1001 followed by zeros $= \mathbf{0b1001000000000000000000}$, which is $0.5625 \times 2^{23} = 4,718,592 = 0x480000$. Assembling: $1\ 10000001\ 1001000000000000000000 = \mathbf{0xC0C80000}$ - which is exactly what a debugger shows for `float f = -6.25f;`. Now try 0.1: it is $0.0001100110011\dots$ repeating forever in binary, so it is **not representable**, and the nearest float is $0x3DCCCCD = 0.100000001490116\dots$. That is the entire explanation of why $0.1 + 0.2 \neq 0.3$.

Property	Consequence for code
Addition is not associative	$(a+b)+c$ may differ from $a+(b+c)$; vectorising or reordering a reduction changes the result, which is why <code>-ffast-math</code> is not free
Catastrophic cancellation	Subtracting nearly-equal numbers destroys precision; rearrange the algebra instead
Comparison to zero is exact, general equality is not	Compare with a tolerance chosen from the magnitudes involved
NaN is not equal to itself	<code>x != x</code> is the standard NaN test, and a NaN silently poisons every later result
Subnormals may be slow	Some hardware traps to microcode; a flush-to-zero mode exists for signal processing
FMA computes $a \times b + c$ with one rounding	More accurate and faster - but changes results relative to separate operations

Format	Bits	Use
binary64 (double)	64	Scientific computing, the C default
binary32 (float)	32	Graphics, DSP, embedded
binary16 (half)	16	Machine learning inference, graphics
bfloat16	16	ML training: float32's exponent range with fewer mantissa bits
FP8 (E4M3 / E5M2)	8	Recent ML accelerators
Fixed point / integer	any	Microcontrollers without an FPU; deterministic and exact

Exercises

- 1 Add $0x7F + 0x01$ in 8-bit two's complement and state the C, V, N and Z flags.
- 2 Compute the carry-lookahead expressions for a 4-bit group and count the gate levels.
- 3 Encode $+3.14159$ and -0.5 as binary32 by hand and check them against a program.
- 4 Find two floats a and b for which $(a+b)+c$ and $a+(b+c)$ differ, and explain which is more accurate.
- 5 Replace a division by 10 with a multiply-and-shift for 32-bit inputs, and prove the result is exact over the whole range.

5. Memory Building Blocks

Memory is where most of a chip's area, most of its energy, and most of its performance problems live. Before caches and DRAM controllers make sense (Part IV), the cells themselves must: they explain why registers are fast and few, SRAM is fast and expensive, and DRAM is cheap and slow.

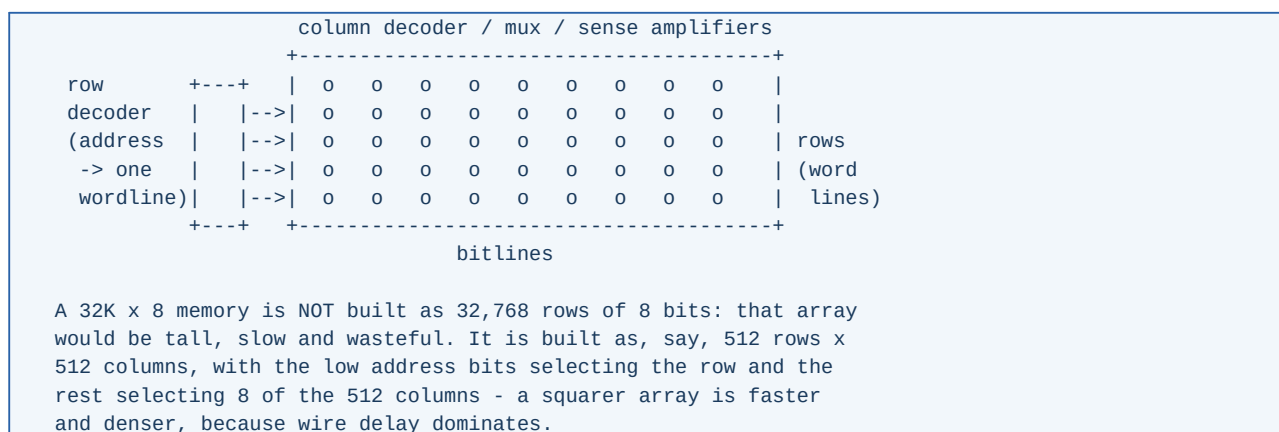
5.1 The storage cells

Cell	Transistors per bit	Access time	Property
Flip-flop (register)	~20-30	Same cycle	Multi-ported, addressable by name in the ISA
SRAM	6	0.5-2 ns	Static: holds while powered; the cache and MCU main memory
DRAM	1 transistor + 1 capacitor	15-50 ns	Destructive read , needs refresh; 20-100x denser than SRAM
NOR flash	1 (floating gate)	50-100 ns read	Random read, execute in place; slow, block-erased writes
NAND flash	<1 effective (3D stacked)	Tens of us	Page-oriented, dense, needs ECC and wear management
MRAM / FeRAM / ReRAM	1-2	10-100 ns	Non-volatile with RAM-like writes; appearing in microcontrollers

6T SRAM cell <pre> WL ---+-----+ +---+---+ +---+---+ BL - ===== - BL' inv inv +-----+ +-----+ two cross-coupled inverters + two access transistors Holds its value indefinitely. Reading is non-destructive. </pre>	1T1C DRAM cell <pre> WL -----+ +---+---+ BL - +---+---+ == C (~25 fF) GND </pre> Reading DRAINS the cap: every read must write back, and every cell needs refresh every ~32-64 ms.
---	--

One picture explains the whole memory hierarchy: six transistors that hold a value against thirty-something femtofarads of charge that leaks away.

5.2 How an array is organised



Address decoding is therefore split into row and column parts - the same split that reappears as the row/column addressing of DRAM in Chapter 21.

5.3 Register files and the cost of ports

A register file is a small, extremely fast, **multi-ported** SRAM: a typical RISC core needs two reads and one write per cycle, and a superscalar machine may need eight reads and four writes. Ports are not free.

Area of a multiported cell ~ (number of ports)^2

because each port adds both a wordline and a bitline pair

Example: 32 registers x 32 bits with 2R1W ~ 1-2 KB-equivalent of area but many times the area per bit of single-ported SRAM, and it is on the critical path of every instruction.

This quadratic cost is why wide superscalar designs bank or duplicate the register file, and why ISAs settle on 16 or 32 architectural registers rather than 256.

ENERGY PER BIT IS THE NUMBER THAT DECIDES ARCHITECTURE

Reading a bit from a register file costs on the order of **0.01-0.1 pJ**; from an on-chip SRAM cache, **1-10 pJ**; from off-chip DRAM, **50-200 pJ** once the interface and the row activation are included - a factor of a thousand between the top and the bottom of the hierarchy. An arithmetic operation costs well under a picojoule. **Moving data therefore costs far more than computing on it**, which is the single most important fact in modern architecture: it justifies caches (Chapter 18), blocking and tiling (Chapter 37), and every accelerator that keeps data local (Chapter 33).

5.4 Content-addressable memory and other structures

Structure	What it does	Where it appears
FIFO / queue	Decouples a producer from a consumer	Every interface between clock domains or rate mismatches
CAM (content-addressable)	Searches all entries for a value in one cycle	Fully-associative TLBs (Chapter 20), issue queues (Chapter 15)
TCAM (ternary)	Matching with don't-care bits	Network routing tables; very power-hungry
Banked memory	Independent arrays served in parallel	Multi-port emulation, cache banks, DRAM banks
Scratchpad / TCM	Software-managed SRAM at a fixed address	Real-time cores, DSPs, GPUs' shared memory

SCRATCHPAD VERSUS CACHE, STATED ONCE

A cache decides automatically what to keep, and gives you speed you cannot predict. A scratchpad is explicit memory at a fixed address: you decide what lives there, so the timing is exactly known. Real-time and DSP systems prefer scratchpads for the code and data on their critical path (Chapter 24), general-purpose systems prefer caches, and GPUs give you both and expect you to use them deliberately.

Exercises

- 1 Draw the 6T SRAM cell and explain what happens on the bitlines during a read and during a write.
- 2 Explain why DRAM needs refresh and estimate how much bandwidth refresh consumes if a row takes 50 ns and every row must be refreshed every 64 ms.
- 3 Organise a 64K x 16 memory into a roughly square array and give the row and column address bit counts.
- 4 Estimate the area cost of doubling the read ports of a register file, and explain the effect on the critical path.
- 5 For a loop you have written, count the arithmetic operations and the bytes moved, then use the energy figures above to say which dominates.

PART II

Instruction Set Architecture

The contract between hardware and software: what an ISA specifies and what it deliberately leaves open, RV32I encoded bit by bit, the assembly and calling conventions a compiler emits, how ARM and x86 differ and why, and the system-level interface of exceptions, privilege and control registers.

6. What an Instruction Set Architecture Is

An ISA is the specification a processor promises to obey and a compiler is allowed to assume. It is the most durable artefact in computing - microarchitectures are redesigned every two years, but a binary compiled in 1995 still runs on an x86 processor made today.

6.1 What the specification must contain

Element	Question it answers	Example
Register model	How much named state is there?	32 x 32-bit integer registers, x0 hardwired to zero
Data types and sizes	What can be operated on?	8/16/32/64-bit integers, IEEE floats
Instruction formats	How is an instruction encoded?	Fixed 32-bit words, six formats
Addressing modes	How is an operand located?	register, immediate, base + displacement
Memory model	What can be reordered, and what is atomic?	Chapter 23 - the hardest part of any modern ISA
Exceptions and privilege	What happens on a fault, and who may do what?	Chapter 10
Calling convention (ABI)	How do separately-compiled pieces interoperate?	Chapter 8 - strictly speaking a convention, not the ISA

WHAT AN ISA DELIBERATELY DOES NOT SPECIFY

Nothing about pipelines, caches, branch prediction, clock speed or the number of execution units - all of that is microarchitecture and may change freely. The exceptions prove the rule: **timing is not architectural** (which is why side-channel attacks in Chapter 32 are so awkward - they exploit what the specification does not mention), and **the memory model is** (because software can observe it).

6.2 The design space

Axis	Choices	Consequence
Operand location	Stack / accumulator / register-register / register-memory	Register-register (load-store) is simplest to pipeline; register-memory shortens code
Instruction length	Fixed vs variable	Fixed decodes trivially and in parallel; variable is denser
Number of registers	8, 16, 32, 128	More registers = fewer spills but larger encodings and a bigger, slower register file (Chapter 5)
Addressing modes	Few vs many	Complex modes hide address arithmetic in one instruction, at the cost of a longer critical path
Condition handling	Flags vs compare-and-branch	See Chapter 4's note - flags create implicit dependencies
Endianness	Little / big / bi-endian	Only matters at the boundary: files, networks, and shared memory

6.3 RISC and CISC, forty years on

	RISC (RV32I, ARM, MIPS)	CISC (x86, 68000, VAX)
Instruction length	Fixed (or two sizes)	1 to 15 bytes

	RISC (RV32I, ARM, MIPS)	CISC (x86, 68000, VAX)
Memory operands	Loads and stores only	Most instructions can address memory
Instruction count for a task	More	Fewer
Decode	Trivial, parallelisable	Complex; x86 cracks instructions into internal micro-operations
Code density	Lower (recovered by compressed encodings)	Higher
Where it wins	Simplicity, power, ease of implementation	Legacy compatibility and code size

THE ARGUMENT WAS SETTLED BY A MERGER, NOT A VICTORY

Modern x86 processors decode CISC instructions into RISC-like micro-operations and execute those out of order, while modern ARM cores have accumulated addressing modes and fused operations that would once have been called CISC. What survived from the RISC argument is the **principle**: make the common case fast, keep the hardware simple enough to pipeline deeply, and let the compiler do the work that can be done at compile time. The remaining practical differences are decode power and code density - both real, neither decisive.

6.4 Encoding: the engineering under the elegance

An instruction encoding is a packing problem with constraints: every field a decoder needs early (the opcode, the register numbers) should sit at a fixed position, immediates should be as large as possible, and the whole thing must fit in a word. The RISC-V solution, examined in the next chapter, keeps the source and destination register fields at **identical bit positions in every format**, so the register file can be read before the instruction is fully decoded - a saving of gate delay in the most timing-critical part of the pipeline.

6.5 Four ways to compute one expression

The operand model is the most visible choice in an ISA. Compare $a = b + c * d$ on four classical machine types, counting instructions and memory references - the two costs that mattered when each was designed.

STACK machine (no registers) push b push c push d mul add pop a -> 6 instructions, 4 memory refs	ACCUMULATOR machine (one register) load c mul d add b store a -> 4 instructions, 4 memory refs
REGISTER-MEMORY (x86-like) mov eax, c imul eax, d add eax, b mov a, eax -> 4 instructions, 4 memory refs (but each is longer to decode)	LOAD-STORE (RISC-V, ARM) lw t0, c lw t1, d mul t0, t0, t1 lw t2, b add t0, t0, t2 sw t0, a -> 6 instructions, 4 memory refs (but each is trivial to pipeline)

The instruction counts differ by 50%; the **memory references are identical**, because the data has to move regardless. That is the insight behind load-store architectures: since the memory traffic is fixed, optimise instead for instructions that are uniform, short and easy to overlap.

WHY THE REGISTER-REGISTER MODEL WON

Registers are 100x faster than memory and can be renamed (Chapter 15). Making memory access explicit and rare means the pipeline can assume that almost every instruction takes one cycle in the execute stage, that only two instruction types touch memory, and that dependences are visible in the register numbers alone. Every modern high-performance implementation - including the ones that **execute** x86 - works this way internally.

6.6 How much can one instruction encode?

A 32-bit fixed encoding must spend its bits somewhere:

opcode + funct	~10 bits	what to do
3 register fields	15 bits	5 bits each for 32 registers

remaining	7 bits	for an immediate - too few!

This is why RISC-V has SIX formats: an instruction that needs a large immediate gives up a register field to get it (I, S, B, U, J types).

It is also why building a 32-bit constant needs TWO instructions (lui + addi) - a real cost, paid so that decode stays trivial.

Every ISA is a negotiation between these fields, and the number of registers is usually the variable that gives way: 32 registers cost 15 of your 32 bits.

Exercises

- 1 For an ISA you use, list its register count, instruction lengths and three addressing modes.
- 2 Explain why a load-store architecture is easier to pipeline than a register-memory one.
- 3 Write the same small function in C and inspect the compiler's assembly on two different ISAs; compare the instruction counts and the code size.
- 4 Design an encoding for a hypothetical 16-bit ISA with 8 registers, and state what you had to give up.

7. RISC-V RV32I, Encoded by Hand

RV32I is the base 32-bit integer instruction set of RISC-V: 47 instructions, six formats, no flags, no condition codes, and an encoding regular enough to work through with a pencil. That is why it is the teaching ISA of this book and of most modern courses.

7.1 The register model

Register	ABI name	Role
x0	zero	Hardwired to 0. Writes are discarded - which makes many pseudo-instructions free
x1	ra	Return address
x2	sp	Stack pointer
x3, x4	gp, tp	Global and thread pointers
x5-x7, x28-x31	t0-t6	Temporaries - caller-saved
x8-x9, x18-x27	s0-s11	Saved registers - callee-saved
x10-x17	a0-a7	Arguments; a0 and a1 also return values
pc	-	Program counter; not a general register (unlike ARM's r15)

WHY A HARDWIRED ZERO REGISTER IS WORTH ONE OF YOUR 32 SLOTS

`mv rd, rs` is `addi rd, rs, 0`. `li rd, 5` is `addi rd, x0, 5`. `j label` is `jal x0, label` - jump and discard the return address. `nop` is `addi x0, x0, 0`. **Four instructions the hardware does not need to implement**, plus a free source of zero for comparisons and address bases. It is the highest-value single decision in the encoding.

7.2 The six formats

```

31      25 24   20 19   15 14   12 11   7 6     0
+-----+-----+-----+-----+-----+-----+
| funct7 | rs2 | rs1 | funct3 | rd | opcode | R  add, sub, and
+-----+-----+-----+-----+-----+-----+
| imm[11:0] | rs1 | funct3 | rd | opcode | I  addi, lw, jalr
+-----+-----+-----+-----+-----+-----+
| imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode | S  sw, sh, sb
+-----+-----+-----+-----+-----+-----+
| imm[12:10:5] | rs2 | rs1 | funct3 | imm[4:1:11] | op | B  beq, blt
+-----+-----+-----+-----+-----+-----+
|      imm[31:12] |      | rd | opcode | U  lui, auipc
+-----+-----+-----+-----+-----+-----+
| imm[20:10:1:11:19:12] | rd | opcode | J  jal
+-----+-----+-----+-----+-----+-----+

```

rs1, rs2 and rd are ALWAYS in the same bit positions.
The branch/jump immediates look scrambled because the bits are placed so that each one lands in the same wire position as in the other formats - it costs the assembler nothing and saves muxes.

Six formats, one register-field layout. Read the diagram twice: the regularity is the design.

ENCODING FOUR INSTRUCTIONS BY HAND

`addi x5, x6, 100` - I-type, opcode 0010011, funct3 000. Fields from the top: imm = 100 = 000001100100, rs1 = 6 = 00110, funct3 = 000, rd = 5 = 00101, opcode = 0010011. Concatenated: 0000 0110 0100 0011 0000 0010 1001 0011 = **0x06430293**. `add x7, x5, x6` - R-type: funct7 = 0000000, rs2 = 00110, rs1 = 00101, funct3 = 000, rd = 00111, opcode = 0110011 = **0x006283B3**. `lw x6, 8(x5)` - I-type with opcode 0000011 and funct3 010: **0x0082A303**. `sw x7, 12(x5)` - S-type, so the immediate 12 = 000000001100 splits into imm[11:5] = 0000000 at the top and imm[4:0] = 01100 in the rd position: **0x0072A623**. Check any of these against an assembler; getting the S-type split right the first time is the moment the format table stops being decoration.

7.3 The instruction set itself

Class	Instructions	Notes
Integer register-register	add sub sll slt sltu xor srl sra or and	R-type; sub and sra are distinguished by a bit in funct7
Integer register-immediate	addi slti sltiu xori ori andi slli srli srai	I-type; the shift amount is 5 bits
Upper immediates	lui (load upper), auipc (add upper to pc)	Build 32-bit constants and pc-relative addresses in two instructions
Loads	lb lh lw lbu lhu	Sign- or zero-extended; base + 12-bit signed offset
Stores	sb sh sw	S-type
Branches	beq bne blt bge bltu bgeu	Compare two registers and branch in one instruction - no flags
Jumps	jal, jalr	Link register written; jalr also provides indirect calls and returns
System	ecall, ebreak, fence, CSR access (Zicsr)	Chapter 10

```
# A complete function in RV32I: sum an array of n words.
# a0 = pointer, a1 = n, returns the sum in a0
sum_array:
    li      t0, 0          # accumulator (addi t0, x0, 0)
    beqz    a1, done        # bne a1, x0, ... (pseudo)
loop:      lw      t1, 0(a0) # load element
            add     t0, t0, t1 # accumulate
            addi    a0, a0, 4  # advance the pointer
            addi    a1, a1, -1 # decrement the count
            bnez    a1, loop   # loop while non-zero
done:      mv      a0, t0     # addi a0, t0, 0
            ret           # jalr x0, 0(ra)
```

Nine instructions, four of which are pseudo-instructions that the assembler expands using x0. Count the memory accesses: exactly one per element, which is what a load-store architecture guarantees and what makes the pipeline of Chapter 12 straightforward.

7.4 The extensions, and why the ISA is modular

Extension	Adds	Typical user
M	Multiply and divide	Almost everything
A	Atomic memory operations, load-reserved/store-conditional	Any multi-core or RTOS system (Chapter 23)
F / D / Q	Single, double, quad floating point	Anything numerical
C	16-bit compressed encodings of common instructions	Embedded - typically 25-30% smaller code

Extension	Adds	Typical user
B	Bit manipulation	Cryptography, compression, firmware
V	Scalable vector operations	DSP, ML, HPC (Chapter 16)
Zicsr, Zifencei	Control/status registers, instruction fence	Any system with privilege or self-modifying code
Custom	Your own instructions	The reason many companies chose RISC-V at all

A part is described by concatenating them: **RV32IMAC** is the 32-bit base with multiply, atomics and compressed instructions - the usual microcontroller profile - while an application processor running Linux needs at least **RV64GC** (G = IMAFD). This modularity is the commercial argument for RISC-V as much as the technical one: you implement what your product needs and pay no area for the rest.

Exercises

- 1 Encode `beq x5, x6, +16` by hand as a B-type instruction and check it against an assembler (the answer is 0x00628863).
- 2 Decode 0x00A50533 by hand: name the format, the fields and the instruction.
- 3 Write `li a0, 0x12345678` using `lui` and `addi`, and explain why the immediate in the `addi` may need to be adjusted by one.
- 4 Rewrite the array-sum loop to process two elements per iteration and count the instructions saved per element.
- 5 List which pseudo-instructions your assembler expands and what each becomes.

8. Assembly, the ABI and What the Compiler Emits

An ISA says what instructions exist; an **ABI** - application binary interface - says how independently compiled pieces of code agree to use them. Between them they explain every line a compiler produces, and reading that output is the fastest way to understand both the machine and your own program.

8.1 The calling convention

Rule	RISC-V	Why it exists
Arguments	a0-a7, then the stack	Registers are free; the stack is a fallback for the ninth argument and for large structs
Return values	a0, a1	Two registers cover most cases; larger results are returned through a hidden pointer
Return address	ra (x1), set by <code>jal</code>	A link register avoids a memory write on every leaf call
Caller-saved	t0-t6, a0-a7, ra	The caller preserves what it still needs
Callee-saved	s0-s11, sp	The callee preserves what it wants to use
Stack alignment	16 bytes at every call	So that vector and double-word accesses are aligned
Stack direction	Grows downward	Universal convention

WHY TWO CLASSES OF REGISTER EXIST AT ALL

If every register were caller-saved, a function calling in a loop would spill everything on each iteration. If every register were callee-saved, a leaf function using one temporary would have to save and restore it. Splitting the set lets the compiler put short-lived values in caller-saved registers and long-lived ones in callee-saved registers, and **the split is a negotiation, not a law of nature** - it is a convention chosen to minimise total spills across real code.

8.2 A stack frame, drawn and built

```

higher addresses
+-----+
| caller's frame      |
+-----+ <- sp on entry
| saved ra            | \
| saved s0 (frame pointer) | | prologue writes these
| saved s1 ...        | |
+-----+ /
| local variables, arrays |
+-----+
| outgoing arguments 9+  |
+-----+ <- sp during the body
lower addresses (stack grows down)

```

```

func:  addi    sp, sp, -32      # 1. allocate the frame (16-aligned)
       sw     ra, 28(sp)      # 2. save the return address
       sw     s0, 24(sp)      #   and any callee-saved register
       addi   s0, sp, 32      # 3. optional frame pointer
       # ---- body: locals live at 0(sp) .. 20(sp) ----
       lw     a0, -20(s0)
       call   other_function  # clobbers t0-t6, a0-a7, ra
       # -----
       lw     s0, 24(sp)      # 4. restore
       lw     ra, 28(sp)
       addi   sp, sp, 32      # 5. deallocate
       ret    # 6. jalr x0, 0(ra)

```

A **leaf** function - one that calls nothing - can skip saving **ra** entirely and often needs no frame at all, which is why small functions are so much cheaper than their call sites suggest and why inlining pays.

WHAT A FUNCTION CALL ACTUALLY COSTS

In the frame above: one **addi** to allocate, two stores to save, two loads to restore, one **addi** to deallocate, the **jal** and the **ret** - **eight instructions of overhead**, plus the stores and loads touch memory (usually L1, a few cycles each). A leaf function needing no saved registers costs just the **jal** and the **ret**: **two**. This is the arithmetic behind inlining, behind avoiding tiny virtual functions in hot loops, and behind the fact that a deeply recursive algorithm can be memory-bound on its own stack traffic.

8.3 How C constructs become instructions

C	Typical assembly
Local scalar	A register, if the compiler can keep it there; otherwise a stack slot
a[i] with 4-byte elements	slli t0, i, 2 then add t0, base, t0 then lw
Struct field	A constant offset in the load or store - free
if (x < y)	One compare-and-branch on RISC-V; a compare then a conditional branch on ARM/x86
switch	A jump table (an indirect jump through a table of addresses) when the cases are dense, a branch chain when sparse
Function pointer / virtual call	An indirect jump - which the branch predictor of Chapter 13 finds much harder than a direct call
Loop	Body plus a compare-and-branch at the bottom; the compiler rotates the loop so there is one branch per iteration
volatile access	A load or store the compiler may neither remove nor reorder

READ THE COMPILER'S OUTPUT REGULARLY

gcc -O2 -S, objdump -d, or an online explorer will show exactly what your source became. Ten minutes of this answers questions that hours of speculation cannot: whether the loop was vectorised, whether the division survived, whether the bounds check was eliminated, whether the function was inlined. It is also the only way to develop an accurate intuition for the cost of language features.

8.4 From object files to a running program

```

main.c ---[compile]--> main.o  --+
util.c ---[compile]--> util.o  --+--[link]--> program (ELF)
                                |
                                v
ELF: header | program headers | .text .rodata .data .bss | symbols
                                |
loader maps segments into memory, resolves dynamic symbols,
then jumps to the entry point. On a microcontroller there is no
loader: the linker's addresses ARE the final addresses (Ch 24).

```

- **Relocation** is the linker patching addresses that were unknown at compile time. Each unresolved reference carries a relocation type saying which bits of which instruction to fix - which is why the immediate-splitting of Chapter 7 matters to toolchain authors.
- **Static linking** copies library code into the binary; **dynamic linking** resolves it at load time through the GOT and PLT, trading a small indirection at every call for shared memory and independent updates.
- **Position-independent code** uses pc-relative addressing (**auipc** on RISC-V) so a library can be mapped anywhere - required for shared libraries and for the address-space randomisation of Chapter 32.
- **Debug information** (DWARF) is a separate, large section mapping addresses back to source lines and variables. Strip it from the shipped binary and keep it forever on the build server - it is what makes a crash address meaningful.

Exercises

- 1 Compile a three-line function at -O0 and -O2 and diff the assembly. Account for every instruction that disappeared.
- 2 Draw the stack frame for a function with four locals and one call, and give every offset.
- 3 Find a function in your code where the compiler chose a jump table for a **switch**, and one where it chose a branch chain.
- 4 Write a leaf function in assembly that obeys the calling convention, and call it from C.
- 5 Use **nm** and **objdump -r** to inspect the relocations in an object file and explain two of them.

9. ARM, x86-64 and the Wider World of ISAs

RV32I is a teaching instrument. The machines you will actually target are usually ARM or x86-64, and knowing where they differ - and why - makes both easier to reason about.

9.1 ARM: two architectures in one family

	Cortex-M (ARMv7-M / ARMv8-M)	Cortex-A (AArch64 / A64)
Instruction set	Thumb-2: mixed 16- and 32-bit for density	A64: fixed 32-bit, 31 general registers
Registers	R0-R12, SP, LR, PC (PC is a general register)	X0-X30 plus a dedicated zero/SP encoding; PC is not a general register
Exceptions	Hardware stacks 8 registers; handlers are plain C functions	Exception levels EL0-EL3, vector tables per level, banked registers
Memory management	MPU: a few protection regions	MMU with full paging and two-stage translation for virtualisation
Condition codes	Flags plus IT blocks for short predication	Flags, with conditional select instructions instead of general predication
Typical use	Microcontrollers, real-time cores	Phones, servers, laptops

The instructive change from A32 to A64 was the **removal** of features: general predication, the program counter as a writable register, and load/store multiple all went away, because each of them complicates out-of-order execution (Chapter 15). A64 is in that sense more RISC than its predecessor - a design cleaned up with thirty years of implementation experience.

9.2 x86-64: complexity that pays for compatibility

- **Variable-length encoding**, 1 to 15 bytes, with prefixes and the REX byte that extended the register set to 16 in 64-bit mode. Decoding is genuinely hard: you cannot know where instruction $n+1$ starts until instruction n is decoded, so wide decode needs predictors, marker caches or a micro-op cache.
- **Register-memory operations**: `add rax, [rbx+rcx*4+8]` performs an address computation, a load and an add. Internally this is cracked into micro-operations, so the machine executing it looks much like a RISC machine underneath.
- **A rich addressing mode** (base + index $\times$ scale + displacement) that maps beautifully onto array indexing, and which ARM and RISC-V approximate with an extra instruction.
- **Vector extensions** in successive layers - SSE, AVX, AVX2, AVX-512 - each adding registers and widths, and each requiring runtime detection because binaries must run on older parts.
- **Strong memory ordering** (TSO): far fewer barriers are needed than on ARM or RISC-V, which is why porting lock-free code *from* x86 *to* ARM exposes latent bugs (Chapter 23).

CODE DENSITY, MEASURED

For the same C program compiled at -Os, typical relative code sizes are: **Thumb-2 = 1.00** (the densest mainstream encoding), x86-64 about **1.1-1.3**, RV32IMC about **1.0-1.1**, RV32IMA without compressed instructions about **1.3-1.4**, and AArch64 about **1.3-1.5** because every instruction is four bytes. On a microcontroller with 64 KB of flash that difference decides whether the product fits; on a server it decides instruction-cache pressure, which is a real performance factor for large codebases.

9.3 The rest of the landscape

Family	Distinguishing idea	Where you meet it
8051, AVR, PIC	8-bit accumulator machines with tiny register files	Legacy and ultra-low-cost embedded

Family	Distinguishing idea	Where you meet it
MSP430	16-bit, orthogonal, very low power	Battery-powered instruments
MIPS / SPARC / POWER	Classic RISC designs	Networking, historical systems, IBM servers
DSP ISAs (TI C6000, SHARC)	VLIW issue, MAC units, circular and bit-reversed addressing	Audio, radar, communications
GPU ISAs (PTX/SASS, RDNA)	SIMT: one instruction stream over many lanes with divergence handling	Chapter 33
Vector ISAs (RVV, SVE)	Length-agnostic vectors: the same binary runs on wider hardware	Chapter 16

THE ISA MATTERS LESS THAN IT USED TO

Binary translation - Rosetta 2, QEMU, and the Windows-on-ARM layer - now runs x86 binaries on ARM at 70-90% of native speed, and most software is distributed as source or as bytecode that is compiled per target. What still depends on the ISA is the **ecosystem**: toolchains, operating-system support, verified libraries, and the licensing model. RISC-V's momentum comes at least as much from being royalty-free and extensible as from any technical property of its encoding.

9.4 What a modern x86 core does with a CISC instruction

```

variable-length bytes
|
[ length decode ]  <- the hard part; often assisted by a
|                  marker cache or a micro-op cache
[ decoders x4-6 ]  <- simple instructions -> 1 uop each
|                  complex ones -> several, or microcode
[ micro-op cache ] <- recently decoded uops, skipping decode
|                  entirely for hot loops (a big power saving)
[ rename / dispatch ] --> the RISC-like out-of-order core of Ch 15

Fusion works both ways:
macro-fusion:  cmp + jcc      -> one uop
micro-fusion:  load + operate -> one uop until execution

```

The practical consequence: an x86 instruction count is not comparable with a RISC instruction count, and the machine's real width is measured in micro-operations per cycle, not instructions.

When porting between ISAs, check	Because
Memory ordering assumptions	x86's TSO hides missing barriers that ARM and RISC-V expose (Chapter 23)
Alignment assumptions	x86 tolerates unaligned access almost everywhere; other ISAs may fault or split the access
char signedness and integer sizes	Implementation-defined, and it differs between ARM and x86 toolchains
Floating-point evaluation	x87 legacy modes, FMA contraction and denormal handling can change results in the last bits
Intrinsics and inline assembly	Entirely ISA-specific; isolate them behind a portable interface
Timing assumptions in code	Cycle counts, cache sizes and prediction behaviour all differ

CODE DENSITY STILL MATTERS, AND HERE IS WHERE

In a microcontroller it decides whether the program fits in flash (Chapter 24). In a server it decides instruction-cache pressure: a large application with a working set of several megabytes of code can spend 10-20% of its cycles front-end bound (Chapter 17), and denser encodings directly reduce that. This is why ARM added Thumb and RISC-V added the C extension, and why both are enabled by default in embedded toolchains.

Exercises

- 1 Compile the same function for RV32IMC, Thumb-2, AArch64 and x86-64 and compare instruction counts and byte counts.
- 2 Find an x86-64 instruction that performs a load, an arithmetic operation and a store, and write the equivalent RISC-V sequence.
- 3 Explain why removing the program counter from the general register file simplifies an out-of-order implementation.
- 4 Look up which vector extension your machine supports at runtime and write the detection code.
- 5 Take a lock-free algorithm written for x86 and list the barriers it would need on ARM. (Chapter 23 gives the rules.)

10. Exceptions, Privilege and the System Interface

Everything so far assumed instructions execute one after another and nothing goes wrong. The system interface is what happens when something does - a page fault, a divide by zero, a timer, a system call - and who is allowed to fix it. It is the part of the ISA that operating systems are built on.

10.1 A taxonomy that avoids confusion

Event	Source	Timing	Example
Fault	The instruction itself, before it completes	Synchronous; the instruction is restarted after handling	Page fault, MPU violation
Trap	The instruction itself, deliberately	Synchronous; execution resumes after it	System call, breakpoint
Abort	A failure that cannot be attributed precisely	Synchronous but imprecise	Bus error on a buffered write
Interrupt	External hardware	Asynchronous; occurs between instructions	Timer, UART, DMA
Non-maskable interrupt	Critical hardware	Asynchronous, cannot be disabled	Watchdog, power failure

PRECISE EXCEPTIONS, AND WHY THEY COST SO MUCH

An exception is **precise** if, when the handler runs, all instructions before the faulting one have completed and none after it have had any effect. Software needs this: a page fault must be restartable, a debugger must show a coherent state. On a simple in-order pipeline it is easy. On an out-of-order machine that has fifty instructions in flight, executed in the wrong order, it requires the entire reorder buffer and in-order retirement machinery of Chapter 15. **Precise exceptions are the main reason out-of-order processors are as complicated as they are.**

10.2 The mechanism, on RISC-V

```

On a trap the hardware does, atomically:
mepc  <- pc of the faulting/trapping instruction
mcause <- why (interrupt bit + exception code)
mtval  <- extra information (faulting address, instruction bits)
mstatus.MPP <- previous privilege mode
mstatus.MPIE <- previous interrupt-enable; MIE <- 0
pc     <- mtvec (direct) or mtvec + 4 x cause (vectored)

The handler ends with MRET, which reverses all of it.

```

ARM does the same work with different names (ELR, ESR, SPSR, VBAR) and the Cortex-M profile additionally stacks eight registers in hardware so that handlers can be ordinary C functions.

Privilege level	RISC-V	ARM	x86	May do
Application	U-mode	EL0	Ring 3	Ordinary instructions only
Operating system	S-mode	EL1	Ring 0	Page tables, device access, privileged instructions
Hypervisor	HS-mode	EL2	VMX root	Second-stage translation, trapping guest operations

Privilege level	RISC-V	ARM	x86	May do
Firmware / secure monitor	M-mode	EL3	SMM	Everything; the first code to run after reset

The rule that makes this work is that a lower level can only enter a higher one **through a defined door**: a trap instruction or an exception, landing at an address the higher level chose. That is the whole basis of isolation between a program and an operating system, and between a virtual machine and a hypervisor.

10.3 System calls, and what they cost

THE PRICE OF CROSSING THE BOUNDARY

A system call must switch privilege, switch stacks, save and restore state, and (on modern hardware with speculation mitigations) flush or tag predictor state. Typical measured costs: **50-150 ns** on a current x86-64 Linux system for a trivial call such as `getpid`, which at 3 GHz is **150-450 cycles**; more when mitigations for Spectre-class attacks are enabled. That is why high-performance code batches I/O (one `writelv` instead of ten `write` calls), why `io_uring` and similar interfaces exist, and why the kernel provides a **vDSO** so that `clock_gettime` can be answered in user space with no trap at all - about 20 ns instead of 100.

10.4 Interrupts as an architectural feature

Controller	Found on	Character
NVIC	Cortex-M	Tightly coupled to the core: hardware register stacking, tail-chaining, and 8-bit priorities - designed for microsecond latency
GIC	Cortex-A	A separate distributor and per-core interfaces, handling hundreds of sources across many cores, with affinity routing
PLIC / CLIC	RISC-V	Platform-level (PLIC: simple, priority arbitration) or core-local with vectoring and preemption (CLIC)
APIC / x2APIC	x86	Local and I/O APICs, message-signalled interrupts from PCIe devices

INTERRUPT LATENCY, IN-ORDER VERSUS OUT-OF-ORDER

A Cortex-M4 takes **12 cycles** from the interrupt request to the first handler instruction, and the hardware has already saved the caller-saved registers - at 100 MHz, 120 ns, and it is deterministic. A large out-of-order core must reach a precise point: it stops fetching, lets the reorder buffer drain or squashes what it can, then traps - typically **hundreds of cycles**, and variable. The **absolute** latency may still be lower because the clock is thirty times faster, but the **variability** is enormous by comparison. This is the architectural reason real-time control lives on microcontrollers even when a fast application processor sits on the same board (Chapter 27).

10.5 Virtualisation, briefly

A hypervisor runs guest operating systems that believe they are privileged. Classically this was done by **trap and emulate**: run the guest deprivileged, trap every privileged instruction, and emulate it. Modern hardware makes it cheap with a dedicated mode (EL2, HS-mode, VMX) and **two-stage address translation**, where the guest's page tables map virtual to guest-physical and the hypervisor's map guest-physical to real physical - so ordinary memory accesses need no traps at all. Chapter 20 returns to what that does to TLB pressure.

Exercises

- 1 Classify each of these as fault, trap, abort or interrupt: divide by zero, `ecall`, a timer expiring, a store to a read-only page, an ECC error detected three cycles late.
- 2 Write a minimal RISC-V trap handler that decodes `mcause` and distinguishes an interrupt from an exception.
- 3 Measure the cost of a `getpid` system call on a machine you have, then measure `clock_gettime` and explain the difference.

- 4 Explain why a page fault must be precise but a performance-counter overflow interrupt need not be.
- 5 Compare the interrupt entry sequence of Cortex-M and Cortex-A and list what the M profile does in hardware that the A profile leaves to software.

PART III

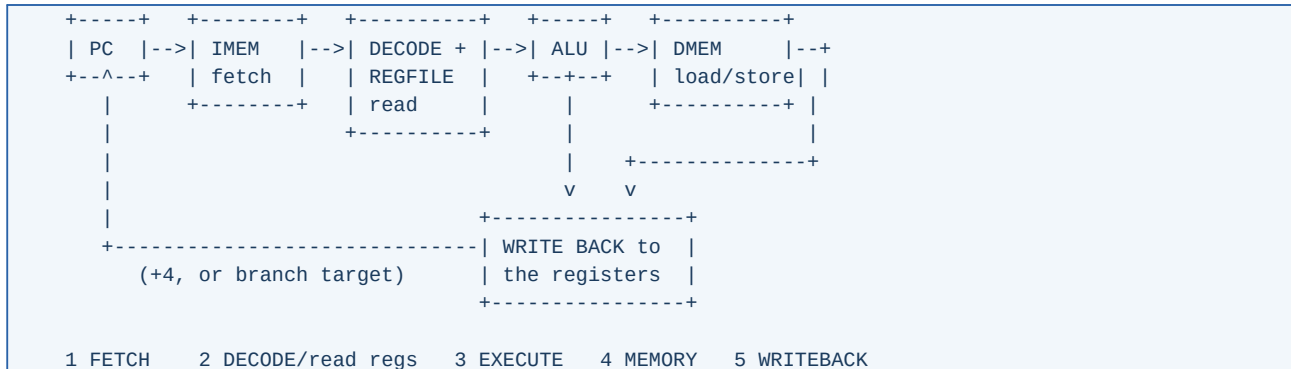
Microarchitecture

How the contract of Part II is actually implemented, and how fifty years of engineering turned one instruction per several cycles into several instructions per cycle: datapaths, pipelines, hazards, branch prediction, out-of-order execution, vectors, threads, and the measurement discipline that keeps all of it honest.

11. Building a Datapath: The Single-Cycle Processor

The shortest path from an ISA to working hardware is a machine that executes exactly one instruction per clock cycle, with a clock slow enough for the longest instruction. It is inefficient and nobody builds one - but every later design is a modification of it, so it is worth building once, on paper.

11.1 The five things every instruction needs



These five steps are the skeleton of every processor in this book. The single-cycle machine performs all five in one clock period; the pipeline of Chapter 12 performs five instructions' worth of steps at once.

11.2 The datapath, instruction by instruction

Instruction	What the datapath must provide
<code>add rd, rs1, rs2</code>	Two register reads, ALU add, write back the ALU result
<code>addi rd, rs1, imm</code>	One register read, an immediate through a multiplexer into the ALU's second input
<code>lw rd, off(rs1)</code>	ALU computes $rs1 + off$ as an address; data memory read; write back the memory result, so another multiplexer is needed at the write-back stage
<code>sw rs2, off(rs1)</code>	Same address computation, but $rs2$ goes to the memory write port and nothing is written back
<code>beq rs1, rs2, off</code>	ALU (or a comparator) tests equality; a separate adder computes $pc + offset$; a multiplexer chooses the next PC
<code>jal rd, off</code>	Write $pc + 4$ into rd while jumping - so the write-back multiplexer needs a third input

Notice the pattern: each new instruction class adds a **multiplexer** and a **control signal**, not a new datapath. That is the essence of datapath design - build the union of what the instructions need, then let control select.

11.3 Control: a truth table with a name

Signal	Purpose	<code>add</code>	<code>lw</code>	<code>sw</code>	<code>beq</code>
RegWrite	Enable the register write port	1	1	0	0
ALUSrc	0 = register, 1 = immediate	0	1	1	0
MemRead	Data memory read	0	1	0	0
MemWrite	Data memory write	0	0	1	0
MemToReg	0 = ALU result, 1 = memory	0	1	x	x
Branch	Take the branch if the comparison holds	0	0	0	1

For a fixed-format ISA like RISC-V this table is a small combinational function of the opcode and funct fields - a few dozen gates. For x86 it is a microcoded ROM, which is the historical origin of the word **microarchitecture**.

11.4 Why nobody builds it this way

THE COST OF THE SLOWEST INSTRUCTION

Suppose the components have these delays: instruction memory 200 ps, register file read 100 ps, ALU 200 ps, data memory 200 ps, register write setup 50 ps. Then **add** needs $200 + 100 + 200 + 50 = 550$ ps, but **lw** needs $200 + 100 + 200 + 200 + 50 = 750$ ps. The clock must accommodate the worst case, so **every** instruction takes 750 ps and the machine runs at 1.33 GHz with a CPI of 1. The **add** instructions waste 200 ps each - and in a real design the gap is far wider, because a multiply or a floating-point operation is several times longer than an add. **Two fixes exist:** multi-cycle execution, where each instruction takes as many short cycles as it needs, and pipelining, which is the next chapter and is what won.

11.5 The multi-cycle alternative

Instead of one long cycle, give each instruction as many short cycles as it needs, reusing one ALU and one memory port across the steps. The control unit becomes a finite state machine (Chapter 3) rather than a truth table.

```

S0 FETCH      IR <- MEM[PC]; PC <- PC + 4
S1 DECODE     A <- reg[rs1]; B <- reg[rs2]; compute branch
              target speculatively (the adder is idle anyway)
              |
              +--> R-type: S6 EXECUTE ALUout <- A op B
              |          S7 WRITEBACK reg[rd] <- ALUout          (4 cycles)
              |
              +--> lw/sw: S2 ADDRESS ALUout <- A + imm
              |          S3 MEMREAD MDR <- MEM[ALUout]
              |          S4 WRITEBACK reg[rd] <- MDR              (5 cycles)
              |          S5 MEMWRITE MEM[ALUout] <- B            (4 cycles)
              |
              +--> branch: S8 compare and update PC              (3 cycles)

```

MULTI-CYCLE CPI, COMPUTED

With the cycle counts above and a typical instruction mix - 25% loads, 10% stores, 45% arithmetic, 15% branches, 5% jumps - the average is $0.25(5) + 0.10(4) + 0.45(4) + 0.15(3) + 0.05(3) = 4.05$ **cycles per instruction**. But each cycle is now only as long as the slowest **single** step (200 ps in Chapter 11's numbers, not 750), so the time per instruction is $4.05 \times 200 = 810$ ps against the single-cycle machine's 750 ps. **Multi-cycle is slightly slower here and much smaller** - one ALU instead of three adders, one memory port instead of two. That was the right trade in 1985, when transistors were scarce; pipelining, which gets both the short cycle and a CPI near 1, is the right trade now.

11.6 Control as microcode

Once control is a state machine with dozens of states, it can be stored as a **table in ROM** rather than built as logic: each entry holds the control signals for one step plus the address of the next entry. That is microcode, and it is how complex instruction sets became implementable at all - one machine instruction becomes a small program in the microcode ROM.

Approach	Advantage	Where it survives
Hardwired control	Fastest; minimal area for a simple ISA	RISC cores, every pipelined design's common path
Microcoded control	Handles very complex instructions; patchable after manufacture	x86's rare and complex instructions; microcode updates that fix errata and security issues in the field
Hybrid	Simple instructions hardwired, complex ones microcoded	Every modern CISC implementation

Exercises

- 1 Draw the single-cycle datapath and mark every multiplexer with the instruction that made it necessary.
- 2 Extend the control table to `jal` and `jalr`, adding whatever signals you need.
- 3 Compute the clock period and the effective performance if a multiply taking 900 ps is added to the ISA.
- 4 Explain why a single-cycle machine's CPI is exactly 1 and why that number, alone, tells you nothing about performance.
- 5 Estimate how many bits of control signals your extended design needs, and how you would generate them from the opcode.

12. Pipelining and Hazards

Pipelining is the single most valuable idea in processor design: instead of making one instruction fast, overlap many so that one **finishes** every cycle. It costs almost no extra hardware and it creates every complication in the rest of Part III.

12.1 The idea, and its arithmetic

cycle:	1	2	3	4	5	6	7	8	9
instr 1	IF	ID	EX	MEM	WB				
instr 2		IF	ID	EX	MEM	WB			
instr 3			IF	ID	EX	MEM	WB		
instr 4				IF	ID	EX	MEM	WB	
instr 5					IF	ID	EX	MEM	WB

^
five instructions in flight at once;
one completes per cycle from here on.

Latency per instruction is unchanged - it is still five stages. What changes is throughput, and throughput is what a program's runtime depends on.

Ideal speedup over the single-cycle machine:

$$T_{\text{single}} = t_{\text{total}}$$

$$T_{\text{pipe}} = \max(\text{stage delay}) + t_{\text{register_overhead}}$$

speedup $\rightarrow$ number of stages, if the stages are balanced

With the delays of Chapter 11 (200,100,200,200,50 ps) and 30 ps of

register overhead: $T_{\text{pipe}} = 200 + 30 = 230 \text{ ps} \rightarrow 4.35 \text{ GHz}$

against 750 ps (1.33 GHz) unpipelined

speedup = $750 / 230 = 3.26x$, not $5x$, because the stages are

unbalanced and each pays the register overhead.

Balanced stages are worth real money: the slowest stage sets the clock for every stage.

12.2 The three hazards

Hazard	Cause	Cure	Residual cost
Structural	Two instructions need the same hardware in the same cycle	Duplicate it: split instruction and data caches, a second register-file port	None, once fixed
Data	An instruction needs a result that is not written back yet	Forwarding: route the result from where it is produced straight to where it is needed	One stall for a load-use pair
Control	The next PC is not known until a branch resolves	Predict (Chapter 13), and flush if wrong	The full pipeline depth on a misprediction

12.3 Data hazards and forwarding, traced

```

add x5, x6, x7      IF ID EX MEM WB
sub x8, x5, x9      IF ID EX MEM WB
                    ^
                    x5 is written back in cycle 5 but needed in cycle 4.
                    FORWARD the ALU output from EX/MEM straight into the ALU
                    input - no stall at all.

lw  x5, 0(x6)      IF ID EX MEM WB
sub x8, x5, x9      IF ID ** EX MEM WB    ** = one stall
                    ^
                    The load's data arrives at the END of MEM, one cycle too
                    late for the consumer's EX. Forwarding cannot travel back
                    in time: this LOAD-USE hazard costs exactly one bubble.

```

Three dependence types exist - RAW (read after write, the real one), WAR and WAW (name conflicts). An in-order pipeline suffers only RAW; the other two appear once instructions can execute out of order, and register renaming in Chapter 15 removes them entirely.

WHAT THE COMPILER DOES ABOUT THE LOAD-USE STALL

It **schedules**: it moves an independent instruction into the slot between the load and its use. This is free performance, and it is why **-O2** code interleaves what looks like unrelated work. When you write a loop that loads and immediately uses each value, the compiler often unrolls it precisely so that it can overlap the next iteration's load with this iteration's arithmetic.

12.4 Control hazards and the cost of depth

$$\text{CPI} = \text{CPI}_{\text{ideal}} + \text{stalls per instruction}$$

loads = 25% of instructions, 40% of them cause a load-use stall

branches = 15% of instructions, misprediction rate 8%,

misprediction penalty 3 cycles

$$\begin{aligned} \text{CPI} &= 1 + (0.25 \times 0.40 \times 1) + (0.15 \times 0.08 \times 3) \\ &= 1 + 0.100 + 0.036 = 1.136 \end{aligned}$$

Now the same program on a 20-stage pipeline, penalty 15 cycles:

$$\text{CPI} = 1 + 0.100 + (0.15 \times 0.08 \times 15) = 1.280$$

...but the clock may be 2x faster, so it still wins - until the misprediction rate or the penalty grows a little further.

This is the calculation that killed the very deep pipelines of the early 2000s: at 31 stages, a 5% misprediction rate consumes more than the frequency gain, and power scales worse than linearly with frequency (Chapter 30).

WHERE THE BRANCH PENALTY COMES FROM

If a branch is resolved in the EX stage of a five-stage pipeline, two instructions have already been fetched behind it and must be discarded: a **2-cycle** penalty. Move the comparison into ID - possible on RISC-V because a branch compares two registers directly - and the penalty falls to **1 cycle**, at the cost of an extra comparator and a tighter critical path in ID. On a 15-stage machine that resolves branches at stage 12, the penalty is **11 cycles**, and no amount of cleverness in the front end changes that - which is why Chapter 13's prediction accuracy matters so much more on deep pipelines.

12.5 What else the pipeline must handle

- **Multi-cycle operations.** A divider taking 20 cycles cannot occupy EX for 20 cycles without blocking everything; it gets its own unpipelined unit, and the pipeline must handle results returning out of order - the first step towards Chapter 15.
- **Precise exceptions.** A fault detected in MEM belongs to an instruction whose successors are already in EX and ID. The pipeline must squash them and record the right PC - which is why exception information travels down the pipeline registers with each instruction.
- **Memory stalls.** A cache miss (Chapter 18) freezes the whole pipeline for tens or hundreds of cycles. In a simple in-order machine, that single event can dominate the CPI calculation above by an order of magnitude.
- **Interlocks versus software.** Early MIPS exposed the load-use hazard to the compiler (the 'delay slot'), saving hardware at the cost of an ISA quirk that outlived its usefulness. Every modern ISA interlocks in hardware instead - a good example of an architectural decision that should not be visible in the contract.

Exercises

- 1 Draw the pipeline diagram for `lw x5,0(x6); add x7,x5,x8; sub x9,x7,x10` with forwarding, and mark every forwarding path.
- 2 Recompute the CPI example with a 3% misprediction rate and explain how much performance prediction accuracy is worth.
- 3 Explain why WAR and WAW hazards cannot occur in a simple in-order five-stage pipeline.
- 4 Reorder a short instruction sequence to remove a load-use stall without changing its meaning.
- 5 Compute the pipeline depth at which the register overhead of Chapter 3 consumes half the clock period, using your own numbers.

13. Branch Prediction and Speculation

About one instruction in five is a branch, and a pipelined machine must decide what to fetch next before it knows the answer. Guessing well is worth more than almost any other single technique in a modern core - and guessing at all turned out to have consequences nobody anticipated until 2018.

13.1 Two questions, two predictors

Question	Structure	Difficulty
Is this instruction a branch at all?	Branch target buffer (BTB), indexed by PC in the fetch stage	Must answer in the same cycle as the fetch
Will it be taken?	Direction predictor: counters indexed by PC and history	The classic problem; 95-99% achievable
Where does it go?	BTB for direct branches	Easy - the target is fixed
Where does an indirect branch go?	Indirect target predictor	Hard: virtual calls, switch tables, interpreter dispatch
Where does a return go?	Return address stack (RAS)	Easy and very accurate - calls and returns nest, so a small stack of 16-32 entries predicts almost perfectly

13.2 From one bit to gigabytes of history

2-bit saturating counter (per branch):

```

    [00]      [01]      [10]      [11]
    strongly  weakly    weakly    strongly
    not taken not taken taken     taken
    <---- not taken ----  ---- taken ---->

```

The extra bit means ONE surprising outcome does not flip the prediction - which is exactly what a loop needs at its exit.

WHY TWO BITS BEAT ONE ON A LOOP

A loop with 10 iterations, executed repeatedly. The backward branch is taken 9 times and not taken once per execution. A **1-bit** predictor mispredicts **twice** per loop execution: once at the exit (it predicted taken) and once on the first iteration of the next execution (it now predicts not-taken) - accuracy $8/10 = 80\%$. A **2-bit** counter, sitting in 'strongly taken', mispredicts only at the exit and stays in 'weakly taken', so the next execution starts correctly - accuracy $9/10 = 90\%$. One extra bit per branch halves the mispredictions.

Predictor	Idea	Typical accuracy
Static (backward taken, forward not)	No state; loops are backward branches	60-70%
2-bit counters (bimodal)	One saturating counter per PC	85-93%
Two-level / gshare	Index the counter table with the PC XOR a global history register, so the same branch is predicted differently in different contexts	93-96%
Tournament / hybrid	A meta-predictor chooses between a local and a global predictor per branch	95-97%
TAGE	Several tagged tables indexed with geometrically increasing history lengths; the longest matching one wins	97-99% - the current state of the art
Perceptron / neural	A linear model over the history bits, learning which correlate	Comparable to TAGE; used in some commercial cores

WHAT ONE PERCENTAGE POINT OF ACCURACY IS WORTH

Take a machine with a 15-cycle misprediction penalty on which 20% of instructions are branches. At **95%** accuracy the added CPI is $0.20 \times 0.05 \times 15 = \mathbf{0.15}$. At **99%** it is $0.20 \times 0.01 \times 15 = \mathbf{0.03}$. If the base CPI is 0.5 (a wide out-of-order machine), those are 0.65 and 0.53 - a **23% difference in total performance from four percentage points of branch accuracy**. That is why predictors occupy tens of kilobytes of state on a modern core, more than the entire memory of the microcontroller in Chapter 24.

13.3 Speculation: acting on the guess

Prediction alone is useless unless the machine executes down the predicted path. Speculative execution means instructions run before it is known whether they should - so the machine must be able to undo them. What is undone: register writes (through renaming and the reorder buffer, Chapter 15), memory writes (held in a store buffer until retirement), and the PC. What is **not** undone is the effect on caches and predictors - and that omission is the basis of the Spectre family of attacks in Chapter 32.

- **Branchless code** removes the problem instead of predicting it: a conditional select (`csel` on ARM, `cmov` on x86) computes both sides and picks one. It wins when the branch is unpredictable, and loses when it is predictable, because it always pays for both sides. Measure before converting.
- **Loop unrolling** removes branches outright and gives the scheduler more independent work.
- **Profile-guided optimisation** lets the compiler lay out the common path as the fall-through, which helps the front end as much as the predictor.
- **Sorting data before processing it** can double the speed of a loop containing a data-dependent branch - the classic demonstration that branch prediction is a first-order performance effect, not a detail.

MEASURE BRANCH MISSES, DO NOT GUESS AT THEM

Every performance-monitoring unit exposes branch instructions and branch misses (Chapter 36). `perf stat` reports both, and a misprediction rate above a few percent in a hot loop is a strong signal that the algorithm - not the code generation - needs attention.

Exercises

- 1 Simulate a 1-bit and a 2-bit predictor on the taken/not-taken sequence of a nested loop and count the mispredictions.
- 2 Explain why a return address stack works so well, and construct a program that defeats it.
- 3 Measure branch misses in a loop over sorted and unsorted data and explain the difference.
- 4 Convert a data-dependent branch into a conditional select and measure both versions with random and with predictable data.
- 5 Compute the CPI impact of branch mispredictions for your own machine's penalty and a measured misprediction rate.

14. Instruction-Level Parallelism and Superscalar Issue

A pipeline finishes at most one instruction per cycle. To go faster, a processor must start several at once - which is possible only to the extent that nearby instructions are independent. That property is **instruction-level parallelism**, and how much of it exists, and who finds it, is the central question of this chapter.

14.1 Measuring the parallelism in a piece of code

```

1: lw    t0, 0(a0)      1 ---+
2: lw    t1, 4(a0)      2 ---|---+      dependence graph
3: add   t2, t0, t1      +---+--> 3 ---+
4: mul   t3, t2, t2      +--> 4 ---+
5: addi  a0, a0, 8       5 (independent) +--> 6
6: sw    t3, 0(a1)

```

Critical path: 1 -> 3 -> 4 -> 6. With a 2-cycle load, 1-cycle add, 3-cycle multiply and 1-cycle store, that is $2+1+3+1 = 7$ cycles of latency, while there are 6 instructions of work. Maximum IPC for this block = $6 / 7 = 0.86$ - and no machine, however wide, can do better on this code alone.

The critical path through the dependence graph is a hard limit. Widening the machine helps only when there is independent work to fill the width - which is why unrolling and interleaving iterations is the standard remedy.

THE TWO WAYS TO FIND PARALLELISM

Statically, in the compiler: it sees the whole function, can unroll and reorder freely, and pays nothing at run time - but it cannot know whether a load will hit in the cache or which way a branch will go. **Dynamically**, in the hardware: it sees only a window of a few hundred instructions and costs power and area - but it knows exactly what happened. Every real machine uses both, and the balance between them is the main difference between a VLIW DSP and an out-of-order server core.

14.2 In-order superscalar

The simplest way to issue two instructions per cycle is to fetch a pair and issue both **if** they are independent and need different units. This is cheap - Cortex-M7, Cortex-A53 and many embedded cores do exactly this - and it keeps the machine deterministic enough to reason about.

Constraint	Effect on the pair
A data dependence between them	The second waits: only one issues
Both need the same functional unit	Only one issues
Register file ports	A 2-wide machine needs 4 read and 2 write ports - the quadratic cost of Chapter 5
A branch in the first slot	The second may be from the wrong path
Alignment of the pair in the fetch buffer	Some designs can only pair instructions that fall in the same aligned block

The result is that a 2-wide in-order machine typically achieves an IPC of **1.2 to 1.5** on general code rather than 2.0 - and it stalls completely on a cache miss, because nothing behind the stalled instruction may proceed. That single weakness is what motivates Chapter 15.

14.3 What the compiler can do about it

- **Instruction scheduling** reorders independent operations to hide latency. It needs accurate latencies, which is why compilers have per-microarchitecture scheduling models.
- **Loop unrolling** replicates the body so that several iterations' worth of independent work is visible at once - the single most effective way to raise ILP.

- **Software pipelining** overlaps iterations explicitly: iteration i 's load, iteration $i-1$'s compute and iteration $i-2$'s store all issue in the same cycle. It is what a DSP compiler does automatically for tight loops.
- **If-conversion** turns a short branch into predicated or select-based code, removing a control dependence at the cost of executing both sides.
- **Register pressure** is the limit on all of this: unroll too far and the values spill to memory, which costs more than the parallelism gained.

14.4 VLIW: give the whole job to the compiler

A **very long instruction word** machine issues a fixed bundle of operations every cycle, one per functional unit, and the compiler is responsible for filling the slots and respecting every latency. The hardware has no dependency checking, no scoreboard and no reordering - so it is small, fast and extremely power-efficient.

Advantage	Problem
Very simple hardware: no dynamic scheduling logic	Code size: unfilled slots are wasted bytes
Deterministic timing - excellent for signal processing	Binary compatibility: a new implementation with different latencies needs recompilation
The compiler sees the whole loop, not a 200-instruction window	Unpredictable latencies (cache misses) cannot be scheduled around
Excellent performance per watt on regular code	Poor on branchy, pointer-chasing general-purpose code

THE ITANIUM LESSON

Itanium (EPIC) was a serious attempt to move scheduling wholly into the compiler for general-purpose computing, with predication, explicit parallelism and speculative loads. It failed commercially for one architectural reason above the business ones: **memory latency is not statically knowable**. A compiler cannot schedule around a cache miss it cannot predict, while an out-of-order machine simply executes past it. VLIW remains the right answer where latencies **are** predictable - DSPs, GPUs' scheduling within a warp, and machine-learning accelerators (Chapter 33) - which is a large and growing share of computing, just not the general-purpose part.

14.5 How much ILP is actually there?

THE LIMIT STUDIES, AND WHAT THEY CONCLUDED

Classic experiments removed constraints one at a time from real programs. With a **perfect** branch predictor, unlimited registers, perfect memory disambiguation and an infinite window, average ILP across general-purpose code is around **10-50** instructions per cycle. Restrict the window to a realistic few hundred instructions and the predictor to a realistic 97%, and it falls to **4-8**. Build the machine, and sustained IPC on real workloads is **1-3**. The gap between 8 and 3 is branches, memory and the cost of building anything wider - and the flatness of that curve after about 4-wide is exactly why the industry turned to multiple cores in 2005 instead of continuing to widen one.

Exercises

- 1 Draw the dependence graph of a loop body you have written and compute its critical path and its maximum IPC.
- 2 Unroll that loop by four and recompute both numbers.
- 3 Explain why a 2-wide in-order machine rarely reaches IPC 2, with a concrete instruction sequence.
- 4 Software-pipeline a three-stage loop by hand and count the registers you needed.
- 5 Find a DSP or GPU ISA that issues bundles and describe how its compiler fills the slots.

15. Out-of-Order Execution

An in-order machine stops the moment an instruction cannot proceed, even if the next fifty are ready. Out-of-order execution removes that restriction: instructions execute as soon as their operands are available, in any order, and are then put back in order before they become visible. It is the most complex and most valuable mechanism in a modern processor.

15.1 Why it exists: hiding memory

```
In-order, on a 200-cycle L3 miss:
lw  t0, 0(a0)      |===== 200 cycles =====|
add t1, t2, t3      X stalled
mul t4, t5, t6      X stalled
... 60 independent instructions ...      X stalled
```

```
Out-of-order, same code:
lw  t0, 0(a0)      |===== 200 cycles =====|
add t1, t2, t3      X (executes immediately)
mul t4, t5, t6      X
... 60 independent instructions execute during the miss ...
then everything RETIRES in program order.
```

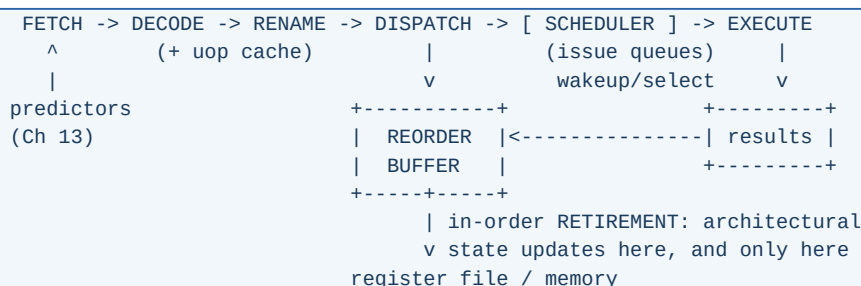
The win is not that arithmetic is faster - it is that the machine keeps finding work during the hundreds of cycles a memory access takes. On memory-heavy code this is worth a factor of two to four.

15.2 Register renaming: removing false dependences

RENAMING A FOUR-INSTRUCTION SEQUENCE

Consider: (1) `add x1, x2, x3` (2) `sub x2, x4, x5` (3) `mul x1, x6, x7` (4) `add x8, x1, x2`. Instruction 2 writes `x2`, which instruction 1 reads - a **WAR** hazard. Instruction 3 writes `x1`, which instruction 1 also writes - a **WAW** hazard. Neither is a real data dependence; both are name conflicts. Renaming maps each architectural write to a fresh physical register: (1) `p20 <- p2 + p3` (2) `p21 <- p4 + p5` (3) `p22 <- p6 x p7` (4) `p23 <- p22 + p21`. Now only instruction 4's genuine RAW dependences remain, and instructions 1, 2 and 3 can execute **in any order, simultaneously**. This is why a physical register file has 150-600 entries behind 32 architectural names.

15.3 The structures of a modern core



The reorder buffer is the machine's conscience: instructions may execute in any order, but they become architecturally visible strictly in program order, which is what makes exceptions precise (Chapter 10) and speculation recoverable (Chapter 13).

Structure	Job	Typical size (large core)
Reorder buffer (ROB)	Tracks every in-flight instruction and retires them in order	300-600 entries
Physical register file	Holds renamed values	200-600 integer, similar vector
Scheduler / issue queues	Holds instructions until their operands are ready, then selects	60-200 entries
Load queue / store queue	Tracks memory ordering and forwards stores to loads	50-200 entries

Structure	Job	Typical size (large core)
Miss-status holding registers (MSHRs)	Track outstanding cache misses - the limit on memory parallelism	10-30 per cache level
Micro-op cache	Skips decode for recently executed code	Thousands of micro-ops

MEMORY-LEVEL PARALLELISM IS THE REAL PRIZE

A single cache miss costs 200 cycles. Ten **independent** misses issued together also cost about 200 cycles, because they overlap. The number of misses a core can have outstanding - set by the MSHRs and by how far the window can run ahead - is therefore as important as any latency number. It is also why pointer chasing (a linked list, a tree walk) is so slow: each miss depends on the previous one, so **none of them can overlap**, and a 200-cycle latency is paid in full, every node.

15.4 Loads, stores and speculation

- **Memory disambiguation.** A load may not pass a store to the same address, but addresses are not known until execution. Modern cores **speculate** that no conflict exists, execute the load early, and detect violations in the store queue - squashing and replaying when wrong. A dedicated predictor learns which loads to be cautious about.
- **Store-to-load forwarding** supplies a value directly from the store queue when a load matches an in-flight store; a partial overlap (storing 8 bytes then loading 4 from the middle) often cannot be forwarded and costs a stall - a real and measurable effect in unaligned or type-punning code.
- **Speculative execution** is undone by squashing the ROB entries after the mispredicted branch and restoring the rename map from a checkpoint. What is **not** undone - cache and predictor state - is Chapter 32's subject.

15.5 What it costs, and the alternative

COMPLEXITY IS SUPERLINEAR

Scheduler wakeup logic compares every result tag against every waiting operand, so its cost grows roughly with the **square** of the window size; register-file ports grow with issue width squared (Chapter 5). Doubling the width of a core therefore costs far more than twice the power for far less than twice the performance - measured gains from 4-wide to 8-wide are typically **20-30%** on general code. That curve is why modern chips pair a few large out-of-order cores with several small in-order or narrow cores: on many workloads a cluster of small cores delivers several times the throughput per watt, which is the arrangement of Chapter 27.

Exercises

- 1 Rename a six-instruction sequence containing one WAR and one WAW hazard, and mark which instructions may then execute in parallel.
- 2 Explain, with a diagram, how a precise exception is delivered for an instruction whose successors have already executed.
- 3 Write two loops that make the same number of memory accesses, one with independent addresses and one pointer-chasing, and measure the difference. Explain it with MLP.
- 4 Find your processor's ROB size and issue width in its optimisation manual, and estimate the largest cache-miss latency it can fully hide.
- 5 Construct a case where store-to-load forwarding fails and measure the penalty.

16. Data and Thread Parallelism: SIMD, Vectors, Threads and GPUs

Instruction-level parallelism ran out at about four instructions per cycle. The two ways past it are to make each instruction do more work - **data parallelism** - and to run several instruction streams at once - **thread parallelism**. Every processor built in the last twenty years uses both.

16.1 SIMD: one instruction, many elements

scalar: a0+b0 a1+b1 a2+b2 a3+b3 4 instructions

SIMD 128-bit:
[a3 | a2 | a1 | a0]
+ [b3 | b2 | b1 | b0] ONE instruction, 4 lanes
= [s3 | s2 | s1 | s0]

Widths in practice: NEON 128-bit, SSE 128, AVX2 256, AVX-512 512, giving 4, 4, 8 or 16 float32 lanes per instruction.

Concern	What you must handle
Alignment	Older instructions require 16- or 32-byte alignment; modern ones tolerate misalignment with a small penalty
Tails	An array of 1,003 floats leaves 3 elements after 125 vectors of 8 - handled by a scalar tail or by masked operations
Reductions	Summing a vector register requires a horizontal add chain; note that this changes the floating-point result (Chapter 4)
Control flow	Divergent branches become masks: both sides are computed and blended
Gather/scatter	Indexed access is supported but far slower than contiguous access - data layout matters more than the instruction choice
Portability	Intrinsics are per-ISA; auto-vectorisation and libraries are portable but less predictable

THE SPEEDUP YOU ACTUALLY GET

A dot product over 1 million float32 values with AVX2 (8 lanes) looks like an 8x opportunity. Count the memory traffic instead: 2 arrays x 4 bytes x 1e6 = **8 MB moved** for 2e6 floating-point operations - an arithmetic intensity of 0.25 FLOP per byte. On a machine sustaining 25 GB/s that caps the kernel at 6.25 GFLOP/s regardless of vector width, while the scalar version already achieves perhaps 4 GFLOP/s. **Measured speedup: about 1.5x, not 8x** - because the kernel is memory-bound, which Chapter 17's roofline makes explicit. Vectorise compute-bound kernels; for memory-bound ones, fix the data movement first.

16.2 Vector architectures: length-agnostic SIMD

Fixed-width SIMD bakes the width into the binary, so AVX-512 code will not run on a 256-bit machine. **Scalable vector** designs - ARM SVE and RISC-V's V extension - instead let the program ask the hardware how long its vectors are and loop accordingly.

```
# RISC-V vector strip-mining: the same binary runs on any width.
loop:  vsetvli t0, a2, e32, m1    # t0 = min(a2, hardware VL)
      vle32.v v0, (a0)           # load t0 elements
      vle32.v v1, (a1)
      vmacc.vv v2, v0, v1        # multiply-accumulate
      slli    t1, t0, 2          # bytes consumed
      add     a0, a0, t1
      add     a1, a1, t1
      sub     a2, a2, t0         # remaining elements
      bnez    a2, loop           # no tail code needed at all
```

`vsetvli` returns however many elements this hardware can process, so the loop handles the tail automatically and the same instructions run on a 128-bit microcontroller and a 2048-bit supercomputer core.

16.3 Thread-level parallelism

Mechanism	What is shared	When it helps
Simultaneous multithreading (SMT/hyper-threading)	One core's execution units, caches and predictors, between 2+ threads	When threads stall often (memory-bound): typically +10-30% throughput. It can hurt cache-sensitive or latency-critical code, and it shares microarchitectural state across security boundaries (Chapter 32)
Multicore	Last-level cache, memory controller, interconnect	Genuine parallelism; limited by Amdahl's law and by coherence traffic (Chapter 23)
Heterogeneous cores (big.LITTLE)	The ISA, so threads migrate freely	Energy: a small core runs background work at a fraction of the power of a large one
Accelerators (GPU, NPU, DSP)	Only memory, usually	Regular, data-parallel work at 10-100x the efficiency (Chapter 33)

16.4 GPUs, as an architecture rather than a product

A GPU is what you build when you assume the work is enormously data-parallel and latency does not matter, only throughput. Its architecture inverts almost every choice a CPU makes.

Choice	CPU	GPU
Latency hiding	Out-of-order execution, caches, prediction	Many threads: when one warp stalls, another issues; thousands of threads per core
Control	One instruction stream per core	SIMT: one stream drives 32 lanes; divergent branches execute both sides with masks
Caches	Large, deep hierarchy for reuse	Small caches plus a software-managed scratchpad ; bandwidth over latency
Registers	Few hundred physical	Hundreds of thousands - the register file is larger than the cache
Memory access	Any pattern; prefetchers help	Coalescing: adjacent lanes must touch adjacent addresses or bandwidth collapses
Best case	Anything, including branchy pointer code	Regular arithmetic over large arrays

OCCUPANCY, COALESCING, DIVERGENCE

Three GPU-specific performance rules follow directly from the table. **Occupancy:** enough resident threads must exist to hide memory latency, and using too many registers per thread reduces how many fit. **Coalescing:** a warp's 32 lanes accessing consecutive addresses become one or two memory transactions; a strided or random pattern becomes 32, cutting effective bandwidth by up to 32x. **Divergence:** an `if` that splits a warp costs the sum of both paths. Every GPU optimisation guide is these three ideas applied repeatedly.

Exercises

- 1 Vectorise a simple loop with intrinsics and measure the speedup. Compute its arithmetic intensity and explain the result.
- 2 Find a loop your compiler refused to auto-vectorise and determine why (aliasing, a dependence, a reduction, control flow).
- 3 Measure a multithreaded benchmark with SMT enabled and disabled.
- 4 Write the same kernel for CPU and GPU and compare both the performance and the effort.
- 5 Take a GPU kernel with a strided access pattern, change the data layout to make it coalesced, and measure the difference.

17. Measuring Performance Without Fooling Yourself

Architecture is a quantitative discipline, and almost every published performance claim is wrong in a predictable way. This chapter is the small set of laws, metrics and habits that keep measurement honest.

17.1 The iron law

$$\text{CPU time} = (\text{instructions} / \text{program}) \times (\text{cycles} / \text{instruction}) \times (\text{seconds} / \text{cycle})$$

$$= \text{instruction count} \times \text{CPI} \times \text{clock period}$$

Only three levers exist, and each belongs to someone different:

instruction count $\leftarrow$ algorithm, compiler, ISA

CPI $\leftarrow$ microarchitecture, memory system, code

clock period $\leftarrow$ circuit design, process, power budget

Any optimisation that improves one term while worsening another by more has made things worse - which is exactly what deep pipelining and aggressive vectorisation sometimes do.

THE IRON LAW, APPLIED

A program executes 2.0e9 instructions at CPI 1.6 on a 3.0 GHz core: time = 2.0e9 x 1.6 / 3.0e9 = **1.07 s**. A compiler change removes 20% of the instructions but, by vectorising, raises CPI to 1.9: 1.6e9 x 1.9 / 3.0e9 = **1.01 s**, a 5% gain - far less than the 20% the instruction count suggested. Meanwhile a memory optimisation that leaves the instruction count alone but cuts CPI to 1.1 gives 2.0e9 x 1.1 / 3.0e9 = **0.73 s**, a 32% gain. **Instruction count is the term people optimise and CPI is usually the term that matters.**

17.2 Amdahl, and what parallelism can and cannot do

$$\text{Speedup} = 1 / ((1 - f) + f / s)$$

f = fraction of the work improved, s = its speedup

$$f = 0.90, s = 8 \rightarrow 1 / (0.10 + 0.1125) = 4.71x$$

$$f = 0.90, s = \text{inf} \rightarrow 1 / 0.10 = 10x \quad (\text{the ceiling})$$

$$f = 0.99, s = 64 \rightarrow 1 / (0.01 + 0.0155) = 39.3x$$

Gustafson's counterpoint: in practice the problem grows with the machine, so f rises with scale - which is why supercomputers are useful despite Amdahl.

The practical reading: measure f before buying parallelism, and spend the first effort on making the serial fraction smaller.

17.3 Benchmarks, and their failure modes

Benchmark	Measures	Caveat
SPEC CPU	Compute-bound single-thread and rate throughput	The industry standard; results are compiler-flag sensitive and must be published with the configuration
CoreMark / CoreMark-PRO	Small-core integer performance	Fits in cache, so it says nothing about the memory system

Benchmark	Measures	Caveat
Dhrystone (DMIPS)	Historic integer performance	Largely meaningless today: optimisers delete much of it; quoted only because vendors still do
STREAM	Sustained memory bandwidth	The right tool for the memory system, not for the core
LINPACK / HPL	Dense floating-point peak	Very high arithmetic intensity - flattering to any machine
MLPerf	Machine-learning training and inference	Closest to real ML workloads; results depend heavily on software stack
Your own workload	What you actually care about	Always the tiebreaker. Every other row is a proxy

USE THE GEOMETRIC MEAN FOR RATIOS

Averaging speedups with the arithmetic mean gives a different answer depending on which machine you call the baseline, which makes it meaningless. The **geometric mean** of ratios is baseline-independent and is what SPEC reports. Similarly, average **rates** (like instructions per second) with the harmonic mean, not the arithmetic one - averaging speeds directly overweights the fast cases.

17.4 Top-down analysis with performance counters

Every issue slot in every cycle is exactly one of four things:

```

+-----+
|  RETIRING          useful work (want this up) |
+-----+
|  BAD SPECULATION  work thrown away           | -> Ch 13
+-----+
|  FRONT-END BOUND  no instructions delivered | -> I-cache,
|                                     |      decode
+-----+
|  BACK-END BOUND   cannot accept more work    | -> memory
|  (core bound / memory bound)                |      or execution
+-----+
```

Measure the split first; it tells you WHICH chapter of this book to open, before you change a line of code.

The top-down method turns a hundred confusing counters into one decision tree, and it is available on every modern x86 and ARM core.

17.5 The roofline model

Attainable performance = min(peak FLOP/s ,
arithmetic intensity x peak bandwidth)

arithmetic intensity = FLOPs performed / bytes moved from memory

ridge point = peak FLOP/s / peak bandwidth

READING A ROOFLINE

A core with 100 GFLOP/s of peak and 25 GB/s of memory bandwidth has its ridge point at $100/25 = 4$ **FLOP per byte**. A dense matrix-multiply with blocking achieves an intensity of about 10 - well past the ridge - so it is **compute-bound** and vectorisation helps. A vector addition performs 1 FLOP per 12 bytes (two loads and a store), an intensity of 0.083, so its ceiling is $0.083 \times 25 = 2.1$ **GFLOP/s: 2% of peak, and no amount of SIMD will change it**. Raising intensity - by blocking, fusing loops, or keeping data in cache - is the only lever, which is the entire content of Chapter 37's optimisation work.

17.6 Measuring honestly

Before you believe a performance number

- ☐ The machine was quiet, the frequency governor fixed, turbo and thermal throttling accounted for.
- ☐ The measurement was repeated enough times to report a median and a spread, not one run.
- ☐ The workload is warm or cold deliberately, and you know which you measured.
- ☐ The compiler flags, the machine and the input set are recorded with the result.
- ☐ The comparison is against a real baseline, not against an unoptimised version of your own code.
- ☐ The metric matches the goal: throughput, p99 latency and energy per operation are three different questions.
- ☐ You checked that the optimiser did not delete the benchmark - a surprisingly common outcome.

Exercises

- 1 Apply the iron law to a change you made recently: measure all three terms before and after.
- 2 Compute the Amdahl ceiling for a program you have profiled, using its measured serial fraction.
- 3 Run a top-down analysis on a hot loop and state which category dominates.
- 4 Compute the arithmetic intensity of three kernels you use and place them on your machine's roofline.
- 5 Take a published benchmark result and list every piece of configuration you would need to reproduce it.

PART IV

The Memory System

The gap between a processor that completes several instructions per nanosecond and a memory that answers in eighty is where most real performance is won and lost: caches and their arithmetic, virtual memory, DRAM timing, storage, and the rules that keep many cores agreeing about what memory contains.

18. Caches from First Principles

A modern core can issue four instructions per cycle at 3 GHz; DRAM answers in about 80 nanoseconds, which is **240 cycles**, during which the core could have executed a thousand instructions. Caches exist to make that gap invisible most of the time, and understanding them is the highest-value thing a programmer can learn from this book.

18.1 Why caches work at all

Principle	Statement	Exploited by
Temporal locality	A location used now is likely to be used again soon	Keeping recently-used data in a small fast store
Spatial locality	A location near one just used is likely to be used soon	Fetching a whole line (64 bytes) on every miss
Sequential locality	Programs mostly walk forward	Hardware prefetching

THE HIERARCHY IS A BET, AND IT PAYS

Each level is a bet that a small fast memory can serve most requests. Typical modern figures: L1 32-64 KB at **4 cycles**, L2 0.5-2 MB at **12-20 cycles**, L3 8-64 MB shared at **40-60 cycles**, DRAM at **200-350 cycles**. Because L1 hit rates on real code are typically 90-97%, the **average** access lands near the top of that list even though the bottom is fifty times slower - which is the entire argument for the hierarchy.

18.2 Anatomy: how an address becomes a hit

A 32 KB, 4-way set-associative cache with 64-byte lines, on a 32-bit address:

$$\text{sets} = 32768 / (64 \times 4) = 128 \text{ sets}$$

```

31                               13 12           6 5           0
+-----+-----+-----+-----+
|          TAG (19 bits)          | INDEX (7) | OFFSET (6) |
+-----+-----+-----+-----+
|          compared against       | selects  | selects the |
|          the 4 tags in the set   | one set  | byte in the |
|                                  |         | line      |
set 0: [tag|V|D| 64 bytes ] [tag|V|D| ... ] [ ... ] [ ... ]
set 1: ...                               4 ways

```

Index bits choose the set, the tag distinguishes which of the many addresses that map to that set is present, and the offset picks the byte. Every cache in every machine is this picture with different numbers.

$$\text{offset bits} = \log_2(\text{line size})$$

$$\text{index bits} = \log_2(\text{number of sets})$$

$$= \log_2(\text{cache size} / (\text{line size} \times \text{associativity}))$$

$$\text{tag bits} = \text{address bits} - \text{index} - \text{offset}$$

$$\text{Direct-mapped} = 1 \text{ way} \quad (\text{fast, cheap, conflict-prone})$$

$$\text{Fully associative} = 1 \text{ set} \quad (\text{no conflicts, expensive: a CAM, Ch 5})$$

$$\text{n-way set assoc.} = \text{the compromise everyone ships (4-16 ways)}$$

WHICH CACHE LINE DOES THIS ADDRESS USE?

Address 0x1234ABCD in the cache above. Offset = bits [5:0] = 0xD & 0x3F = **0x0D**, byte 13 of the line. Index = bits [12:6]: 0x1234ABCD >> 6 = 0x48D2AF, and & 0x7F = **0x2F** = set 47. Tag = bits [31:13] = 0x1234ABCD >> 13 = **0x91A5**. Now note something important: address 0x1236ABCD differs only in bit 17, so it has the **same index** and lands in the same set. Four such addresses fit; a fifth evicts one. **This is why arrays whose stride is a large power of two perform badly** - every row maps to the same few sets.

18.3 Writes, and the policies that matter

Policy	Options	Consequence
On a write hit	Write-through (update both levels) vs write-back (mark dirty, write later)	Write-back is universal in L1/L2: it absorbs repeated writes to the same line and cuts bandwidth several-fold
On a write miss	Write-allocate (fetch the line first) vs no-write-allocate	Write-allocate suits code that reads what it writes; streaming stores bypass the cache entirely to avoid polluting it
Replacement	LRU, pseudo-LRU, random, RRIP	True LRU is expensive above 4 ways; pseudo-LRU costs a few bits per set and performs nearly as well
Inclusion	Inclusive / exclusive / non-inclusive	Inclusive L3 simplifies coherence (Chapter 23) but wastes capacity duplicating L1/L2 contents

18.4 What a miss actually costs

Core	L1	L2	L3	DRAM
4cy	14cy	45cy	250cy	
+-----+	+-----+	+-----+	+-----+	+-----+
hit	hit	hit	hit	miss -> row activate, CAS, transfer (Ch 21)

A line fill brings 64 bytes - so one miss on a 4-byte int costs the same as a miss on the whole 64-byte line. Using all 16 ints in that line is FREE; using one and moving on wastes 94% of the bandwidth you just paid for.

This is the single most actionable fact in the book: **arrange data so that everything in a fetched line gets used**. Chapter 37 turns it into concrete techniques.

18.5 How the parameters trade against each other

Increase this	Miss rate	Hit time	Miss penalty	Power
Cache size	Falls (roughly as the square root of size)	Rises	-	Rises
Associativity	Falls (conflict misses)	Rises slightly	-	Rises: more tags compared per access
Line size	Falls, then rises again when lines bring in unused data	-	Rises: more bytes per fill	-
Number of levels	-	-	Falls	Rises

TWO RULES OF THUMB WORTH REMEMBERING

The square-root rule: doubling a cache typically cuts the miss rate by about 30% (a factor of $1/\sqrt{2}$), so going from 32 KB to 64 KB might take a 4% miss rate to 2.8% - useful, but each doubling costs area and hit time. **The 2:1 rule:** a direct-mapped cache of size N has about the same miss rate as a two-way set-associative cache of size $N/2$. Together they say why nearly every L1 is 4- or 8-way and 32-64 KB: below that, misses dominate; above it, hit time and the VIPT constraint of Chapter 20 bite.

18.6 Caches you can see: microcontrollers and scratchpads

Small cores often have no cache at all (Chapter 24) - SRAM is already single-cycle. In between sit designs with a **line buffer or prefetch buffer in front of flash**, which is a cache in everything but name: it makes sequential code fast and a taken branch expensive, which is exactly why a tight loop that fits in the buffer can run twice as fast as one that does not.

Device class	Typical arrangement
8/16-bit MCU	None: SRAM at one cycle, flash at one or two
Cortex-M3/M4 class	A flash accelerator or prefetch buffer, plus wait states
Cortex-M7 class	Real L1 instruction and data caches, plus tightly-coupled memory for deterministic code
Application processor	L1 per core, L2 per core or cluster, shared L3, sometimes a system-level cache in front of DRAM
GPU	Small L1 per multiprocessor plus a software-managed scratchpad, and an L2 in front of high-bandwidth memory

MEASURING YOUR CACHE HIERARCHY IN TEN LINES

Time a loop that walks an array of size S with stride 64 bytes, repeatedly, for S from 4 KB to 64 MB, and plot nanoseconds per access against S . The result is a staircase: flat at the L1 latency while S fits in L1, a step up at the L2 boundary, another at L3, and a plateau at DRAM latency. Then repeat with varying **stride** at fixed size to find the line size (the time jumps once the stride exceeds it) and the associativity. This one experiment reproduces every number in this chapter for a machine you actually own.

Exercises

- 1 Compute the tag, index and offset split for an 8-way 512 KB L2 with 64-byte lines on a 48-bit address space.
- 2 Find the address that conflicts with 0x2000_0000 in a 32 KB 4-way cache with 64-byte lines - that is, has the same index but a different tag.
- 3 Write a loop that strides by 4096 bytes through a large array and measure its speed against a stride of 64. Explain the difference with the index bits.
- 4 Determine your machine's L1, L2 and L3 sizes, line size and associativity from the operating system, and verify the line size with a stride experiment.
- 5 Explain why write-back caching reduces memory traffic for a loop that increments every element of an array ten times.

19. Cache Performance and Optimisation

Chapter 18 built the mechanism. This chapter is the arithmetic that turns cache behaviour into cycles, the taxonomy that says which misses you can do something about, and the hardware techniques that hide the rest.

19.1 Average memory access time

$$\text{AMAT} = \text{hit time} + \text{miss rate} \times \text{miss penalty}$$

Multi-level, recursively:

$$\text{AMAT} = t_{L1} + m_{L1} \times (t_{L2} + m_{L2} \times (t_{L3} + m_{L3} \times t_{\text{DRAM}}))$$

where m_{Lx} is the LOCAL miss rate (misses at that level divided by accesses to that level) - not the global one. Confusing the two is the most common error in cache arithmetic.

AMAT COMPUTED, AND WHERE THE TIME GOES

L1: 4 cycles, 3% miss. L2: 14 cycles, 40% of L1 misses miss again. L3: 45 cycles, 50% of L2 misses miss. DRAM: 250 cycles. Working inwards: L3 level = $45 + 0.50 \times 250 = 170$. L2 level = $14 + 0.40 \times 170 = 82$. **AMAT = $4 + 0.03 \times 82 = 6.46$ cycles.** Now the diagnosis: of those 6.46 cycles, 4 are the unavoidable L1 hit and **2.46 are misses** - meaning the memory system costs 62% on top of a perfect cache. Halving the L1 miss rate to 1.5% gives AMAT = 5.23, a **19% improvement in every memory access**, which is why a data-layout change can outperform any amount of instruction-level tuning.

19.2 The three Cs, and what to do about each

Miss type	Cause	Fixed by
Compulsory	The first reference to a line - it was never there	Larger lines, prefetching. Cannot be eliminated, only overlapped
Capacity	The working set exceeds the cache	A bigger cache, or blocking/tiling the algorithm so the working set fits (Chapter 37)
Conflict	Too many hot lines map to one set	Higher associativity, or padding arrays so the stride is not a large power of two
(Coherence)	A line invalidated by another core's write	Fix the sharing pattern - see false sharing in Chapter 23

DIAGNOSING WHICH C YOU HAVE, EXPERIMENTALLY

Run the workload with a simulator or on machines with different cache sizes. If **increasing capacity** helps, they are capacity misses. If **increasing associativity** at the same size helps, they are conflict misses. If neither helps, they are compulsory - and the only remaining lever is prefetching or reducing the data touched. This three-line experiment aims your effort correctly and takes an hour.

19.3 The hardware techniques, and what each buys

Technique	Attacks	Typical gain
Non-blocking (lockup-free) cache	Miss penalty - the core continues during a miss	Essential; enables the memory-level parallelism of Chapter 15
Hardware prefetching (stride, stream)	Compulsory and capacity misses on regular access patterns	Large on sequential code; useless or harmful on pointer chasing

Technique	Attacks	Typical gain
Critical word first / early restart	Miss penalty for the requested word	A few cycles per miss
Victim cache	Conflict misses	A small fully-associative buffer of recent evictions
Way prediction	Hit time in an associative cache	Saves energy and a little latency
Banking / multi-ported	Bandwidth	Allows two loads per cycle
Write buffer with merging	Write traffic	Absorbs bursts; merges multiple writes to one line
Software prefetch instructions	Predictable but irregular patterns	Useful when you can compute the address far enough ahead - typically 100-200 cycles

19.4 What software can do

- **Improve spatial locality.** Structure of arrays instead of array of structures when you touch one field of many objects; that single change often halves the bytes moved.
- **Block (tile) loops** so that the working set of the inner loops fits in L1 or L2. For matrix multiplication this changes the arithmetic intensity of Chapter 17 from $O(1)$ to $O(\text{block size})$, and with it the achievable fraction of peak.
- **Fuse loops** that walk the same array, so it is traversed once.
- **Align and pad** hot structures to line boundaries, and avoid power-of-two strides that alias in the index bits.
- **Use streaming (non-temporal) stores** for data you write once and will not read - it avoids the write-allocate fetch and does not evict useful data.
- **Reduce the data**, by using narrower types or compressing - a 16-bit value moves half the bytes of a 32-bit one, and bandwidth is usually the binding constraint.

WHY BLOCKING MATTERS, IN ONE CALCULATION

Multiply two 1024 x 1024 float32 matrices naively: the inner loop walks B by columns, so each access is a new cache line - about 4 MB of traffic per pass over B, repeated 1024 times, roughly **4 GB** moved for 2.1 GFLOP of work: an intensity of 0.5 FLOP/byte, deep in the memory-bound region. Block it into 64 x 64 tiles: each tile pair is $2 \times 64 \times 64 \times 4 = 32$ KB, which fits in L1/L2, and each loaded element is reused 64 times. Traffic falls to roughly **64 MB** and the intensity rises to about 32 FLOP/byte, past the ridge point - so the kernel becomes compute-bound and can approach peak. **Same arithmetic, same hardware, 60x less memory traffic, and typically 5-20x faster.**

Exercises

- 1 Compute AMAT for your own machine's published latencies and a miss rate you measure with performance counters.
- 2 Write matrix multiplication naively and then blocked, and measure both time and cache misses.
- 3 Convert an array-of-structures loop into structure-of-arrays and measure the bytes moved.
- 4 Demonstrate conflict misses: allocate an array with a power-of-two row stride, measure, then pad each row by one line and measure again.
- 5 Add software prefetches to a linked-list traversal and find the prefetch distance that helps most - then explain why too small and too large both fail.

20. Virtual Memory and the TLB

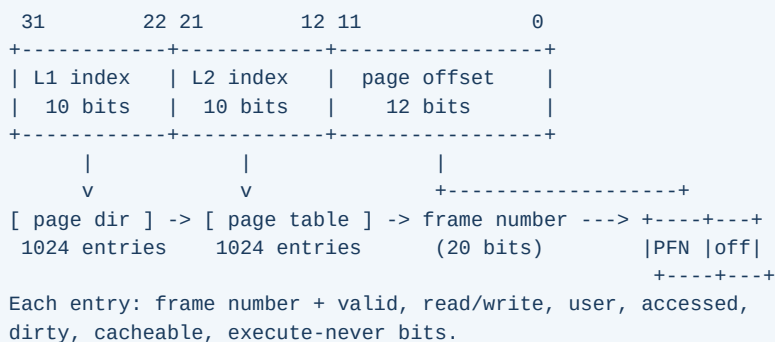
Virtual memory gives every process its own address space, lets a program use more memory than exists, and enforces the isolation that operating-system security depends on. It does all of this by inserting a translation between every address a program computes and the address that reaches memory - and that translation must not cost anything, which is what makes it interesting.

20.1 What translation buys

Property	Mechanism
Isolation	A process can only name addresses its page table maps
Relocation	The same virtual address maps to different physical pages in different processes
Protection	Per-page read/write/execute and user/supervisor bits
Over-commit and demand paging	A page can be absent, faulting in on first use
Sharing	Two page tables may point at the same physical frame - shared libraries, copy-on-write after <code>fork</code>
Contiguity	A program sees a flat space while physical memory is fragmented

20.2 Page tables and the walk

32-bit virtual address, 4 KB pages, two-level table:



64-bit systems use four or five levels (48- or 57-bit addresses), so a walk can cost four or five dependent memory accesses - each of which may itself miss in the cache.

ONE PAGE-TABLE WALK, BY HAND

Virtual address **0x00403004** with the split above. L1 index = bits [31:22] = 0x00403004 >> 22 = **1**. L2 index = bits [21:12] = (0x00403004 >> 12) & 0x3FF = **3**. Offset = **0x004**. So: read entry 1 of the page directory to find the page table's frame; read entry 3 of that table to find the data frame - say frame 0x0025A; the physical address is (0x0025A << 12) | 0x004 = **0x0025A004**. Two memory accesses before the real one, and on a 4-level 64-bit system, four. **That is why the TLB exists.**

20.3 The TLB: a cache for translations

TLB reach = entries x page size

64 entries x 4 KB = 256 KB (an L1 TLB)

1536 entries x 4 KB = 6 MB (a large L2 TLB)

64 entries x 2 MB = 128 MB (huge pages)

32 entries x 1 GB = 32 GB (giant pages, for databases)

Cost of a miss: a page-table walk, 20-200 cycles depending on how much of the walk hits in the data cache. Some cores have a dedicated page-walk cache for the upper levels.

If your working set exceeds the TLB reach, you pay a walk on a large fraction of accesses even when every one of them hits in the cache - a failure mode that looks mysterious until you measure `dtlb_load_misses`.

HUGE PAGES: WHEN THEY ARE WORTH IT

A database or an in-memory analytics job touching 40 GB with 4 KB pages needs 10 million translations - hopeless for any TLB. With 2 MB pages it needs 20,000, and with 1 GB pages, 40. Measured gains of **10-30%** on such workloads are common, and essentially all of it is TLB misses removed. The costs are internal fragmentation, longer allocation pauses, and (with transparent huge pages) occasional latency spikes from background compaction - which is why latency-sensitive services often disable transparent huge pages and allocate explicit ones instead.

20.4 Where the TLB sits relative to the cache

Arrangement	Behaviour	Used by
Physically indexed, physically tagged (PIPT)	Translate first, then look up - simple and correct, but serial	L2 and L3 caches
Virtually indexed, physically tagged (VIPT)	Index with the virtual address while the TLB translates in parallel , then compare physical tags - fast and correct provided index+offset bits fit within the page offset	Almost every L1 cache
Virtually indexed, virtually tagged (VIVT)	No translation on a hit, but aliasing and context switches become painful	Rare; some embedded designs

VIPT explains an otherwise odd fact: **L1 caches stopped growing at 32-64 KB**. With 4 KB pages and 64-byte lines, the offset is 6 bits and the page offset is 12, leaving 6 index bits - so a VIPT L1 can have at most 64 sets, and capacity can only grow by adding ways. A 32 KB 8-way cache is exactly at that limit, which is why the same number appears on processor after processor.

20.5 Context switches, ASIDs and virtualisation

- Switching processes changes the translation, which would require flushing the TLB entirely. **Address-space identifiers** (ASID on ARM, PCID on x86) tag each entry with its owner so entries survive the switch - worth several percent of system-level performance.
- Virtualisation adds a second translation**: guest-virtual to guest-physical to host-physical. A TLB miss then costs a **nested** walk of up to $4 \times 4 = 16$ memory accesses in the worst case, which is why huge pages matter even more inside virtual machines.
- Kernel page-table isolation**, introduced against Meltdown (Chapter 32), unmaps most kernel pages while in user mode, adding TLB pressure and system-call cost - a security fix with a measurable architectural price.

Exercises

- 1 Perform a two-level page-table walk by hand for three addresses, including one that faults.

- 2 Compute the TLB reach of your machine (the entry counts are in `cpuid` or the ARM ID registers) and compare it with the working set of a program you run.
- 3 Measure `dtlb_load_misses` for a random-access benchmark over 1 GB with and without huge pages.
- 4 Explain why a VIPT L1 cache of 128 KB with 4 KB pages would need 16 ways, and why that is unattractive.
- 5 Write a program that demonstrates copy-on-write by forking and measuring when the physical memory usage actually rises.

21. DRAM and the Memory Controller

Below the caches sits a device with a completely different character: capacitors that must be refreshed, an internal structure of banks and rows, and timing rules with a dozen named constraints. Understanding it explains why memory latency has barely improved in twenty years while bandwidth has grown a hundredfold.

21.1 The structure, from chip to system

channel -> DIMM -> rank -> chip -> bank group -> bank -> row/column

bank (one of 8-32 per chip):

rows (16K-128K)	ACTIVATE copies one row into the ROW BUFFER, destroying it in the array
ROW BUFFER (1-2 KB)	READ/WRITE then access columns of that buffer; PRECHARGE writes it back.

Access to an OPEN row = CAS only (fastest)
 Access to a CLOSED row = ACTIVATE + CAS (medium)
 Access to a DIFFERENT row in the same bank = PRECHARGE + ACTIVATE + CAS (slowest)

Three latencies for the same instruction, differing by a factor of three, depending only on what the previous access touched. This is why memory latency is a distribution, not a number.

DRAM LATENCY AND BANDWIDTH, COMPUTED

DDR4-3200 CL22: the clock is 1600 MHz, so $t_{CK} = 0.625 \text{ ns}$, and $CL22 = 22 \times 0.625 = 13.75 \text{ ns}$. With t_{RCD} and t_{RP} also 22 cycles, the three cases are: row hit = $CL = 13.75 \text{ ns}$; row empty = $t_{RCD} + CL = 27.5 \text{ ns}$; row conflict = $t_{RP} + t_{RCD} + CL = 41.25 \text{ ns}$ - before any queueing or controller overhead, which typically doubles the number a program observes. **Bandwidth:** $3200 \text{ MT/s} \times 8 \text{ bytes per transfer} = 25.6 \text{ GB/s per channel}$, so a dual-channel desktop has 51.2 GB/s and an eight-channel server 205 GB/s. Note what this means per core: on a 16-core machine with 51.2 GB/s, each core's share is **3.2 GB/s** - about one byte per cycle at 3 GHz, which is why arithmetic intensity (Chapter 17) decides so much.

21.2 What the memory controller does

- **Schedules out of order.** It reorders requests to maximise row-buffer hits and to group reads together (bus turnaround between read and write costs cycles), typically improving throughput by 20-40% over first-come-first-served.
- **Manages refresh.** Every row must be refreshed roughly every 32-64 ms; refresh commands steal 2-5% of bandwidth and can delay a request by hundreds of nanoseconds - a real source of tail latency.
- **Chooses an address mapping.** Which address bits select the channel, bank and row decides how well a given access pattern spreads across banks. A pathological stride can map an entire loop onto one bank and lose most of the available bandwidth.
- **Enforces the timing constraints** - t_{RC} , t_{FAW} , t_{RRD} and a dozen others - which is why a controller is a substantial piece of hardware and why DRAM cannot simply be driven faster.

Technology	Character	Where
DDR4 / DDR5	Highest capacity per dollar, 25-70 GB/s per channel	Servers, desktops
LPDDR4X / LPDDR5	Lower voltage, narrower channels, deep low-power states	Phones, laptops, embedded
GDDR6 / GDDR6X	Very high bandwidth per pin, higher latency	GPUs
HBM2e / HBM3	Stacked dies with a very wide interface: hundreds of GB/s to several TB/s	Accelerators, HPC

Technology	Character	Where
On-chip SRAM	Nanosecond latency, megabyte capacity	Caches, microcontrollers (Chapter 24)

THE MEMORY WALL, STATED WITH NUMBERS

Between 1990 and today, processor throughput rose by roughly four orders of magnitude while DRAM **latency** improved by less than one - from about 100 ns to about 50 ns. Bandwidth grew much faster than latency fell, because bandwidth can be bought with parallelism (more channels, wider buses, more banks) and latency cannot. Every structure in Part III and IV - caches, prefetchers, out-of-order windows, multithreading - exists to convert that abundant bandwidth into tolerable **effective** latency.

21.3 Address mapping and bank parallelism

The memory controller decides which address bits select the channel, the bank and the row. That mapping is not arbitrary: it determines whether a given access pattern spreads across banks (good) or concentrates on one (disastrous).

A typical mapping for a dual-channel, 16-bank system:

```
[ row 16 bits ][ bank 4 ][ column 10 ][ channel 1 ][ byte 3 ]
                ^
```

Low bits choose the channel and column, so consecutive cache lines alternate channels and stay in one open row: sequential access gets both channels AND row-buffer hits.

A stride of exactly (row size x banks) hits the SAME bank and a DIFFERENT row every time: every access is a row conflict, the slowest case, and effective bandwidth can fall by 5-10x.

This is the DRAM-level analogue of the cache conflict misses in Chapter 19, and it has the same cure: avoid large power-of-two strides, or pad the array.

BANK-LEVEL PARALLELISM, QUANTIFIED

One bank can serve a new row every tRC, typically about **45 ns**. With 16 banks and requests spread evenly, the device can start a row activation every $45/16 = 2.8$ ns on average - which is what lets a channel sustain its full 25.6 GB/s despite each individual access taking 41 ns of latency. This is Little's law again: the bandwidth exists only if enough independent requests are in flight, which is why an out-of-order core with many MSHRs (Chapter 15) achieves a large fraction of peak while a strictly in-order core with one outstanding miss achieves almost none.

21.4 Commands, in the order they happen

```
time ->
PRECHARGE  |--trp--|
              ACTIVATE  |--trcd--|
                        READ  |--cl--|
                                data burst
                                |8 beats|

trc = minimum time between two ACTIVATES to the same bank
tfaW = a window limiting how many ACTIVATES may occur at once
      (a POWER constraint - activating a row is expensive)
refresh must be interleaved throughout, stealing 2-5% of time
```

The controller's scheduler exists to keep as many banks as possible in the useful part of this timeline at once.

Exercises

- 1 Compute row-hit, row-empty and row-conflict latencies for your machine's memory from its published timings.
- 2 Compute the per-core memory bandwidth share of a machine you use, and compare it with the bandwidth a memory-bound loop achieves.

- 3 Measure latency and bandwidth with a pointer-chase benchmark and a STREAM-style benchmark, and explain why they measure different things.
- 4 Estimate the bandwidth lost to refresh for a device with 32,768 rows refreshed every 64 ms with a 350 ns refresh cycle.
- 5 Explain why a stride that maps every access to the same bank can cost an order of magnitude of bandwidth.

22. Storage and the Memory-Storage Boundary

Below DRAM the hierarchy continues, and the gaps get larger: a nanosecond of cache, a hundred nanoseconds of DRAM, tens of microseconds of flash, milliseconds of a spinning disk. Storage now sits close enough to memory that its architecture affects processor design - and its internals leak into performance in ways that surprise software.

22.1 The full hierarchy, with numbers

Level	Latency	Bandwidth	Cost per GB (order)
Register	0 cycles	-	-
L1 cache	~1 ns	1-3 TB/s	-
L3 cache	~15 ns	300-800 GB/s	-
DRAM	60-100 ns	25-400 GB/s	\$2-5
CXL-attached memory	150-300 ns	30-60 GB/s	\$2-5
NVMe SSD (TLC)	20-100 us	3-14 GB/s	\$0.05-0.15
SATA SSD	100-200 us	0.5 GB/s	\$0.05
Hard disk	5-10 ms	0.1-0.3 GB/s	\$0.01-0.02
Network / object store	0.5-100 ms	varies	\$0.005-0.03

Each step down is roughly 100x slower and 10x cheaper. The important consequence for architecture is that the **top four rows are now close enough to each other** that the boundary between memory and storage has become a design choice rather than a fact.

22.2 Inside a flash SSD

```

host LBA ----> [ FTL: logical-to-physical map, in DRAM ]
                |
                v
channel 0 --- die --- plane --- BLOCK (erase unit, 4-16 MB)
channel 1 --- ...                |
...                               +-- PAGE (program unit, 16 KB)
channel 7 ---                   +-- PAGE

Rules: read a page; program a page **once**; erase a whole block
before reprogramming any page in it. Nothing can be overwritten
in place - which is why an FTL exists at all.

```

Every property of an SSD that surprises software - write amplification, latency spikes, the benefit of TRIM, the value of free space - follows from that erase-before-write rule.

WRITE AMPLIFICATION AND DRIVE LIFETIME

Suppose the host writes 4 KB and the drive must, to make room, copy three valid pages elsewhere: the flash sees 16 KB written for 4 KB requested, a **write amplification factor of 4**. Now compute endurance for a 512 GB TLC drive rated at 1,000 program/erase cycles: total flash writes available = 512 GB x 1,000 = 512 TB, so host writes = 512 TB / WAF. At WAF 4 that is **128 TB** - about 7 years at 50 GB/day. Reduce WAF to 1.5 by leaving 20% of the drive free and issuing TRIM, and it becomes **341 TB**, or 18 years. **Free space is endurance**, and a full drive is both slower and shorter-lived - which is why enterprise drives ship with 7-28% hidden over-provisioning.

Cell type	Bits per cell	Endurance (P/E cycles)	Use
SLC	1	50,000-100,000	Industrial, caches inside drives
MLC	2	3,000-10,000	Legacy enterprise

Cell type	Bits per cell	Endurance (P/E cycles)	Use
TLC	3	1,000-3,000	Mainstream consumer and datacentre
QLC	4	300-1,000	Read-mostly bulk storage

22.3 Interfaces, and why NVMe changed things

A SATA drive speaks a protocol designed for rotating disks: one command queue, 32 entries, through a host controller that assumed seeks dominate. NVMe instead exposes up to 64K queues of 64K entries each, mapped directly into memory over PCIe, with an interrupt per completion queue. The result is not only more bandwidth but far lower **software** overhead - a few microseconds per operation instead of tens - which is why the operating system's I/O path became the bottleneck and why polling-mode drivers and `io_uring` exist.

- **Memory-mapped files** let the page-fault mechanism of Chapter 20 handle I/O, so file access becomes a load - convenient, but the latency of a fault is now inside your instruction stream.
- **Zero-copy** paths (DMA straight into user buffers, `sendfile`, RDMA) exist because copying data through the CPU costs more than the I/O itself at these rates.
- **Persistent memory and CXL** blur the boundary further: byte-addressable non-volatile media, and memory attached over a coherent link rather than a DDR bus, both of which change what a memory hierarchy can look like (Chapter 35).
- **Alignment and block size still matter**: a 4 KB write that straddles two flash pages costs two program operations, and a misaligned partition can quietly halve a drive's write performance.

Exercises

- 1 Compute the endurance of a drive you own at WAF 2 and WAF 5 with your actual daily write volume.
- 2 Measure the latency distribution of 4 KB random reads on an SSD and note the tail - then explain the tail with garbage collection.
- 3 Compare sequential and random write throughput on a nearly-full and a nearly-empty drive.
- 4 Explain why a database's write-ahead log benefits from a small amount of very fast, high-endurance storage.
- 5 Trace one `read()` system call through the layers from application to flash die, listing every queue and copy on the way.

23. Cache Coherence and Memory Consistency

Once there are several cores with private caches, two of them can hold the same line and disagree about its contents. **Coherence** is the hardware mechanism that stops that; **consistency** is the architectural rule about what orderings software may observe. They are different questions, they are routinely confused, and getting either wrong produces bugs that appear once a week on one machine.

23.1 Coherence: keeping caches honest

MESI: every cached line is in one of four states

MODIFIED this cache has the only copy, and it is dirty
 EXCLUSIVE this cache has the only copy, and it is clean
 SHARED several caches may have this line, all clean
 INVALID this cache does not have it

Core A reads X -> A: EXCLUSIVE
 Core B reads X -> A: SHARED, B: SHARED
 Core B writes X -> A: INVALID, B: MODIFIED (invalidate sent)
 Core A reads X again -> B writes back or forwards; both SHARED

The invariant is simple: at most one writer, or any number of readers. Everything else - the extra O and F states of MOESI and MESIF, directory protocols, snoop filters - is engineering to make that invariant scale.

Approach	How it finds the copies	Scales to
Snooping	Every cache watches a shared bus or broadcast network	A handful of cores; broadcast traffic grows with the square of the count
Directory	A directory records which caches hold each line, and messages are sent point to point	Dozens to hundreds of cores; costs directory storage
Snoop filter	A directory-like structure that filters broadcasts	The common middle ground in modern server chips

FALSE SHARING: THE COST OF A SHARED CACHE LINE

Two threads increment two different counters that happen to sit in the same 64-byte line. Logically independent; physically a **ping-pong**: each write invalidates the other core's copy, so every increment costs a coherence transaction of **100-300 cycles** instead of about 1. A loop of 10 million increments per thread thus takes roughly $10^6 \times 200 = 2e9$ cycles, about 0.7 s at 3 GHz, against about 3 ms when the counters are padded onto separate lines - a **200x** slowdown from a data-layout accident. The fix is one **alignas(64)** per counter, and finding it is one of the highest-value uses of a profiler in multithreaded code.

23.2 Atomics, and what they cost

Primitive	Mechanism	Typical cost
Relaxed load / store	Ordinary cached access	1-4 cycles
fetch_add (uncontended)	Gets the line in Modified state, operates locally	20-40 cycles
fetch_add (contended)	The line moves between cores on every operation	100-500 cycles, and worsens with core count
Compare-and-swap loop	Retries on failure	Unbounded under contention - livelock is possible
Load-reserved / store-conditional	RISC-V and ARM's building block; the store fails if the line was touched	Similar, with a guaranteed forward-progress rule
Mutex (uncontended)	One atomic plus a fence	20-50 cycles
Mutex (contended)	A system call and a context switch	1,000-10,000 cycles

The practical lesson is that **the cost of an atomic is a property of the sharing pattern, not of the instruction**. Per-core counters aggregated occasionally, sharded locks, and read-mostly structures (RCU, seqlocks) all exist to keep lines out of the ping-pong regime.

23.3 Consistency: what orderings software may observe

The store-buffer litmus test. Initially $x = y = 0$.

```
Core A      Core B
x = 1;      y = 1;
r1 = y;     r2 = x;
```

Can $r1 == 0$ AND $r2 == 0$?

Sequential consistency (SC): NO - some interleaving must order one store before the other's load.

x86 (TSO): YES - each core's store sits in its own store buffer while its load proceeds.

ARM / RISC-V / POWER (weak): YES, and more besides - loads and stores to DIFFERENT addresses may be reordered freely unless a barrier says otherwise.

This is not a bug in the hardware; it is the memory model, and it is part of the ISA. Software that needs an ordering must ask for it.

Model	Reordering allowed	Barriers needed
Sequential consistency	None	None - and it is too slow to implement literally
x86-64 (TSO)	Store-to-later-load only	Rarely; mfence or a locked instruction for the store-load case
ARM, RISC-V, POWER (weak/relaxed)	Almost anything between independent addresses	Acquire/release on every synchronisation point, plus full fences where needed

THE RULE FOR PROGRAMMERS, IN ONE PARAGRAPH

Do not reason about barriers directly unless you are writing a runtime. Use the **acquire/release** discipline of the language memory model: a release store publishes everything written before it, and an acquire load that reads that value sees all of it. The compiler then emits the right barriers for each ISA - **dmb ish** on ARM, **fence r, rw** on RISC-V, often nothing at all on x86. And note the trap this closes: **code developed on x86 and tested only there will contain missing barriers that appear as impossible bugs on ARM**, because TSO hid them.

23.4 Scaling: NUMA and the limits of sharing

Beyond one socket, memory is **non-uniform**: local DRAM might be 90 ns away and a remote socket's 140 ns, with a fraction of the bandwidth. Coherence traffic crosses the same link. The practical rules are the same ones that make single-core code fast, applied at a larger scale: allocate memory on the node that will use it (first-touch policy), pin threads so they stay near their data, shard state instead of sharing it, and measure remote-access counters before assuming any of it is happening.

Exercises

- 1 Trace the MESI states of one line as two cores read and write it in a sequence you choose.
- 2 Reproduce false sharing: two threads incrementing adjacent counters, then padded ones, and measure both.
- 3 Measure the cost of an uncontended and a heavily contended atomic increment as a function of thread count.
- 4 Write the store-buffer litmus test and run it on x86 and on ARM. Record how many iterations it takes to observe the weak outcome.
- 5 Take a lock-free structure from a textbook and identify every acquire, release and full fence it needs, and why.

PART V

Real Devices

What the abstractions of Parts I to IV look like when they are packaged and sold: inside a microcontroller, the core families you can actually buy, systems on chip and their boot process, the interconnect that ties them together, interrupt controllers and DMA as architectural features, and how to choose between them.

24. Inside a Microcontroller

A microcontroller is a complete computer on one die: processor, memory, and the peripherals that connect it to the physical world. Its architecture is shaped by three constraints that do not apply to a server processor - **cost measured in cents, energy measured in microamps, and timing that must be predictable** - and every design decision follows from those.

24.1 The block diagram, and what each block costs

```

+-----+
| CORE (Cortex-M / RISC-V) | DEBUG (SWD/JTAG, trace) |
+-----+
|          BUS MATRIX (AHB-Lite / AXI-Lite)          |
+-----+
| FLASH | SRAM | DMA | PERIPHERAL BUS BRIDGE (APB) | | | | | | |
| 32KB- | 4KB- | 2-16 ch | | | | | | | |
| 2MB   | 512KB |      | UART SPI I2C TIMER ADC |
+-----+
| CLOCK/PLL | POWER/LDO | RESET/BROWN-OUT | WATCHDOG |
+-----+

```

Die area is dominated by FLASH and SRAM, not by the core:
a Cortex-M0+ core is well under 0.01 mm² in a modern process,
while 256 KB of flash is a large multiple of that.

The processor is the cheapest part of a microcontroller. That single fact explains why vendors offer forty variants of the same core with different memory and peripheral mixes.

Design choice	Reason	Consequence for software
Flash on-chip, executed in place	No external memory, instant boot	Wait states at high clock (Chapter 2 in the firmware sense); code and constants share bandwidth
SRAM, not DRAM	No refresh, no controller, deterministic	Kilobytes, not gigabytes - static allocation is the norm
No cache, or a small one	Determinism and area	Timing is countable; a cache, if present, often has a lockdown/scratchpad mode
MPU instead of MMU	Area and determinism	Protection without translation; no demand paging
Harvard-ish buses	Fetch and data access in the same cycle	Instruction and data can be in different memories with different speeds
Rich peripheral set	The application is I/O, not computation	Most of the reference manual is peripherals
Deep sleep modes	Battery life dominates many products	The architecture exposes retention, wake sources and clock gating to firmware

24.2 Deterministic execution, and what it is worth

COUNTING CYCLES ON A MICROCONTROLLER

On a Cortex-M4 with zero flash wait states, **add** takes 1 cycle, **ldr** 2, a taken branch 3, and interrupt entry 12. A control loop of 40 instructions with 8 loads and 2 taken branches therefore costs $30 \times 1 + 8 \times 2 + 2 \times 3 = 52$ cycles, or 0.65 us at 80 MHz - and that number is **exact and repeatable**, run after run. The equivalent code on an out-of-order application processor might average 0.05 us and occasionally take 5 us because of a cache miss, a TLB miss, or the scheduler. **For a motor control loop, the predictable 0.65 us is worth more than the fast-but-variable alternative**, and that is the entire argument for microcontrollers in real-time systems.

Determinism is an **architectural property**, not an accident: it follows from having no cache to miss, no translation to walk, no speculation to squash, and no operating system to preempt. Each performance feature added to a microcontroller - a cache, a prefetcher, a branch predictor - buys throughput and spends predictability, which is why real-time profiles (Cortex-R, and the TCM-equipped M7) offer ways to switch the

trade back.

24.3 The memory map is the architecture

A microcontroller's programming model is largely its memory map: everything - flash, SRAM, every peripheral register, the interrupt controller, even the core's own control registers - has a fixed address, and there is exactly one address space. There is no enumeration, no driver probing, no device tree; the addresses are constants in a header file generated from the datasheet.

WHY THIS MATTERS CONCEPTUALLY

In Part IV, memory was a hierarchy that software could ignore and hardware managed automatically. Here it is an **explicit map that software manages**: which memory a buffer lives in, whether a peripheral can reach it by DMA, and how many cycles an access costs are all programmer-visible decisions. The two worlds are the same architecture with the automation turned off - which is why learning the microcontroller version makes the server version easier to reason about, and vice versa.

Exercises

- 1 Draw the block diagram of a microcontroller you have used, from its datasheet, and mark which buses connect what.
- 2 Find its flash wait states at maximum clock and compute the cost of an instruction fetch that misses the prefetch buffer.
- 3 Count the cycles of a small function by hand from the instruction timing table, then measure it with a cycle counter and reconcile any difference.
- 4 Compare the die area of the core and of the memory for a part whose die photo you can find.
- 5 List every distinct region in the part's memory map and what lives there.

25. Core Families: What You Can Actually Buy

Architecture is taught with idealised machines; products are built with specific cores that embody particular trade-offs. This chapter is a map of the ones you will meet, and of the properties that distinguish them.

25.1 The ARM Cortex families

Family	Target	Key architecture
Cortex-M0/M0+/M23	Lowest cost and power	2-3 stage pipeline, ARMv6-M/v8-M baseline, ~1 KB of gates' worth of core; M23 adds TrustZone
Cortex-M3/M4/M33	Mainstream microcontrollers	3-stage, hardware divide, optional FPU and DSP instructions; M33 adds TrustZone-M
Cortex-M7/M55/M85	High-performance embedded	6-stage superscalar, caches, tightly-coupled memory; M55/M85 add Helium vector processing for edge ML
Cortex-R	Hard real-time with high performance	Deeper pipeline plus TCM and often lockstep dual cores for safety (Chapter 31)
Cortex-A (32/64-bit)	Application processors	MMU, multi-level caches, out-of-order in the larger members, multi-core clusters
Neoverse	Servers	Cortex-A design points scaled for throughput, many cores, large caches, high memory bandwidth

WHAT 'CORTEX-M4 VERSUS CORTEX-M7' REALLY MEANS

Both run the same instruction set, so the same binary works. The M7 is **superscalar with caches and branch prediction**, so it executes perhaps 2-3x more work per cycle and clocks higher - but it also introduces cache-miss variability, cache-coherency concerns with DMA, and a longer interrupt latency. Choosing between them is choosing between throughput and predictability, exactly the trade of Chapter 24, and it should be made from the application's timing requirements rather than from the megahertz number.

25.2 RISC-V profiles

Profile	Extensions	Comparable to
RV32E / RV32I	Base only, 16 or 32 registers	8/16-bit microcontrollers
RV32IMC	Multiply, compressed	Cortex-M0+/M3
RV32IMAFC	Plus atomics and float	Cortex-M4F
RV64GC	IMAFD + compressed	Cortex-A: enough to run Linux
RV64GCV	Plus vectors	Cortex-A with SVE; edge ML and HPC
Custom extensions	Whatever your workload needs	The reason many silicon vendors adopted RISC-V

The architectural interest of RISC-V is not that its base ISA is unusual - it is a clean, conventional RISC - but that the **modularity is part of the specification**. A vendor can implement exactly the extensions a product needs and add custom instructions for its own workload without asking permission, which changes the economics of building a specialised processor.

25.3 The rest of the field

Family	Distinguishing property	Still used for
8051 / AVR / PIC	8-bit, tiny, decades of tooling	Cost-driven consumer goods, legacy designs
MSP430	16-bit, exceptional low-power design	Battery instruments, metering

Family	Distinguishing property	Still used for
DSPs (C6000, SHARC, Blackfin)	VLIW or dual-MAC, specialised addressing	Audio, radar, communications, motor control
Automotive cores (TriCore, Power Architecture)	Lockstep, ECC everywhere, safety certification	Engine, braking and steering controllers
Softcores (Nios V, MicroBlaze, PicoRV32)	Implemented in FPGA fabric	Custom systems, prototypes, small control tasks inside a larger FPGA design
Application-specific cores	Custom ISA for one domain	Modems, ML accelerators, storage controllers (Chapter 33)

READING A CORE'S SPECIFICATION CRITICALLY

Vendors quote **DMIPS/MHz** and **CoreMark/MHz**, which describe the core alone with everything in cache. Real performance on a microcontroller is usually set by flash wait states, and on an application processor by the memory system - neither of which appears in those numbers. Ask instead: what is the pipeline depth, is it superscalar, does it have caches and are they configurable as scratchpad, what is the interrupt latency, and what is the memory bandwidth to the code and data it will actually use.

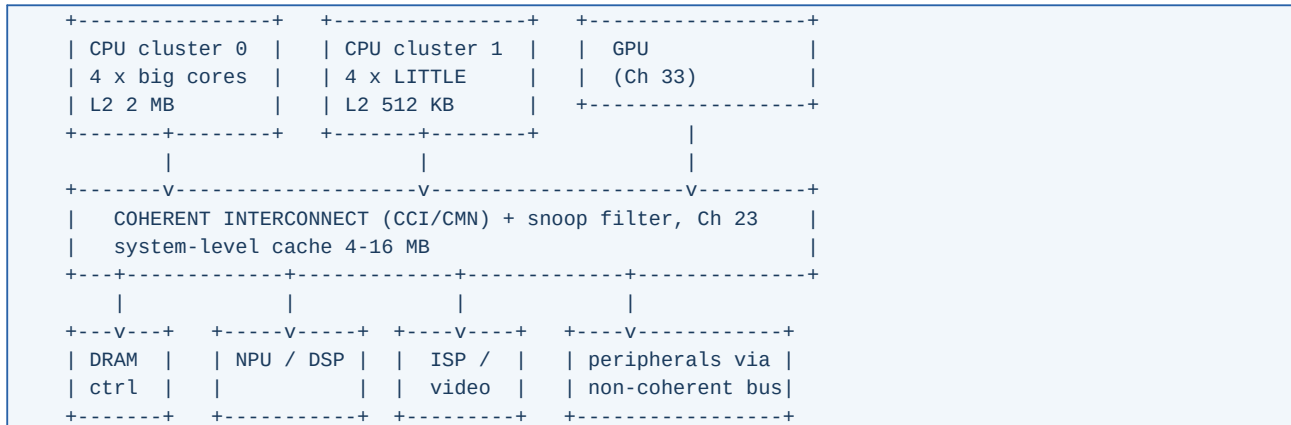
Exercises

- 1 For a product you know, justify the core family chosen and name one alternative that would also have worked.
- 2 Compare the interrupt latency of a Cortex-M0+, a Cortex-M7 and a Cortex-A core, and explain the differences architecturally.
- 3 Find a RISC-V core's specification and list its extensions; work out which ARM core it competes with.
- 4 Take a CoreMark/MHz figure and estimate what the same core would achieve running from flash with three wait states.
- 5 Identify one workload for which a DSP or a VLIW core beats a general-purpose core, and explain why in terms of Chapter 14.

26. Systems on Chip: Application Processors

An application processor is not a CPU with extras; it is a small distributed system on one die - several CPU clusters, a GPU, an image pipeline, a neural accelerator, memory controllers, and dozens of peripherals, all sharing an interconnect and a memory system. Its architecture is mostly about **integration**: who talks to whom, who gets bandwidth, and who is allowed to see what.

26.1 A representative floorplan



Every arrow is a bandwidth negotiation and a potential source of interference: a camera pipeline streaming 4K frames can starve the CPU of memory bandwidth, which is why quality-of-service settings exist in the interconnect.

26.2 The boot chain

1. Boot ROM (immutable, in silicon) - reads straps, loads the first-stage loader from eMMC/SPI/USB, and VERIFIES it (Ch 32)
2. First-stage loader (SPL/TF-A) - brings up DRAM, clocks, power
3. Second-stage loader (U-Boot/UEFI) - device tree or ACPI tables, selects a kernel, may verify it too
4. Kernel - MMU on, drivers probed, secondary cores released
5. Init / user space

A microcontroller (Chapter 24) collapses all of this into:
reset -> vector table -> your code, in microseconds.

Each stage exists because the previous one lacked something - the boot ROM has no DRAM, the first-stage loader has no filesystem - and each hands over a slightly more capable machine.

Integration concern	Architectural mechanism
Which masters can reach which memory	A system MMU (SMMU/IOMMU) translating and restricting DMA from every device - the same protection idea as Chapter 20, applied to peripherals
Bandwidth sharing	Quality-of-service classes and bandwidth regulators in the interconnect
Cache coherency with accelerators	Coherent ports (ACE, CHI) so a GPU or NPU sees CPU writes without explicit flushes
Power and clock domains	Independent voltage/frequency islands, each with its own controller (Chapter 30)
Secure and non-secure worlds	TrustZone: a security state attribute carried on every bus transaction (Chapter 32)
Heterogeneous cores	Cache-coherent big and LITTLE clusters, and often a separate always-on microcontroller for sensors

THE ALWAYS-ON ISLAND

Nearly every modern phone SoC contains a small microcontroller - a Cortex-M or a RISC-V core - that stays awake while the application processor sleeps, handling sensors, wake words and power sequencing at microamps. It is a perfect illustration of this book's central comparison: **the same silicon holds a machine optimised for throughput and one optimised for efficiency and determinism, because no single design point serves both.**

Exercises

- 1 Find a published SoC block diagram and identify the coherent and non-coherent paths.
- 2 Trace the boot chain of a board you have and name what each stage verifies.
- 3 Explain what an IOMMU protects against, with a concrete attack.
- 4 Measure the effect of a memory-bandwidth-heavy background task on a latency-sensitive foreground one.
- 5 Identify the always-on domain of a device you own and what it is responsible for.

27. Interconnect: Buses, Fabrics and Networks on Chip

Once a chip contains more than a few blocks, the wiring between them becomes an architecture of its own, with protocols, arbitration, ordering rules and performance characteristics that decide how the whole system behaves.

27.1 The AMBA family, in order of complexity

Protocol	Character	Used for
APB	Simple, low-bandwidth, non-pipelined; two cycles per transfer	Peripheral registers: UART, timers, GPIO
AHB / AHB-Lite	Pipelined, single outstanding transfer, burst support	Microcontroller main buses, small SoCs
AXI	Five independent channels, multiple outstanding transactions , out-of-order completion by ID	The workhorse of modern SoCs: memory, DMA, accelerators
ACE / CHI	AXI plus coherency messages, snoops and a scalable mesh protocol	Multi-core clusters, coherent accelerators, server fabrics

WHY AXI'S SEPARATE CHANNELS MATTER

AXI splits a transaction into **address-read**, **read-data**, **address-write**, **write-data** and **write-response** channels that operate independently. A master can therefore issue several addresses before any data returns, and the slave may answer out of order using transaction IDs. This is exactly the memory-level parallelism of Chapter 15, expressed at the bus level: latency is hidden by having many requests in flight, not by making any one of them faster.

Interconnect performance, the two numbers that matter:

bandwidth = bus width x clock x utilisation

128-bit AXI at 800 MHz = 12.8 GB/s (at 100% utilisation)

latency = arbitration + protocol overhead + slave response

outstanding transactions needed to sustain full bandwidth:

$N = \text{bandwidth} \times \text{latency} / \text{transfer size}$ (Little's law)

$12.8 \text{ GB/s} \times 100 \text{ ns} / 64 \text{ B} = 20 \text{ outstanding transactions}$

Little's law again: if the master supports only 4 outstanding transactions, it can use at most a fifth of the available bandwidth, no matter how wide the bus is.

27.2 From buses to networks on chip

Shared bus (small SoC)	Crossbar	Mesh NoC (many cores)
<pre> M M M ===== one at a time S S S </pre>	<pre> M M [X-bar] S S </pre>	<pre> R--R--R--R R--R--R--R R--R--R--R </pre>
simple, cheap, contention-bound	full bandwidth, area $\sim N^2$	scalable to hundreds of nodes

A network on chip carries packets through routers with flow control and routing algorithms - the same discipline as computer networking, with nanosecond budgets and no room for retransmission.

- **Ordering rules** are part of the protocol: transactions with the same ID must complete in order, different IDs may not. Getting this wrong in an accelerator's interface produces data corruption that looks like a software bug.
- **Deadlock** is a real hazard once there are dependent request and response paths; interconnects use virtual channels and strict ordering rules to prevent it.
- **Off-chip links** - PCIe, CXL, USB, Ethernet - are the same idea at longer distances: serialised, packetised, credit-flow-controlled, with latencies measured in hundreds of nanoseconds rather than tens.
- **CXL** is architecturally notable because it carries **coherence** off-chip, allowing memory attached to a device to join the processor's coherent address space (Chapter 35).

27.3 Budgeting bandwidth across a system

DOES THE SOC HAVE ENOUGH MEMORY BANDWIDTH?

A device with a 4K60 display, a 4K30 camera and a CPU doing real work. **Display:** $3840 \times 2160 \times 4 \text{ bytes} \times 60 \text{ Hz} = 1.99 \text{ GB/s}$ read, continuously and with a hard deadline - a missed line is a visible artefact. **Camera:** $3840 \times 2160 \times 1.5 \text{ bytes (NV12)} \times 30 \text{ Hz} = 0.37 \text{ GB/s}$ written, plus the same read again by the encoder and written back compressed. **CPU and GPU:** whatever is left. Against a dual-channel LPDDR4X interface at about **17 GB/s** achievable, the fixed-function capture and display traffic is 2.4 GB/s, or 14% - comfortable on paper. But the display's traffic cannot wait, so it is given the highest quality-of-service class, and the CPU sees its effective bandwidth drop whenever the display is refreshing. **This is why SoC interconnects have QoS classes and bandwidth regulators at all:** not to make the average work, but to protect the deadline.

Traffic class	Requirement	Interconnect mechanism
Display, audio out	Hard deadline, constant rate	Highest priority, guaranteed bandwidth, deep FIFOs
Camera, sensor capture	Cannot back-pressure the sensor	High priority plus buffering
CPU	Latency-sensitive but elastic	Low latency class, moderate priority
GPU, NPU	Throughput-hungry, latency-tolerant	Lowest priority, largest queues - they absorb whatever is left
DMA, storage	Bulk	Best effort

LATENCY AND BANDWIDTH ARE DIFFERENT CURRENCIES

A GPU wants **bandwidth** and does not care about latency, because it has thousands of threads to hide it (Chapter 16). A CPU wants **latency**, because a single miss can stall its window. A display wants **guaranteed** bandwidth with a bounded worst case, which is neither. One interconnect must serve all three, and the settings that optimise one degrade the others - which is why SoC performance engineering is largely arbitration tuning rather than raw capacity.

Exercises

- 1 Compute the sustainable bandwidth of a 64-bit AXI bus at 500 MHz and the number of outstanding transactions needed at 150 ns latency.
- 2 Explain why a peripheral register bus can be simple while a memory bus cannot.
- 3 Find the interconnect topology of an SoC you use, and identify the likely bottleneck between the CPU and DRAM.
- 4 Describe a scenario in which two masters with different IDs return data out of order and the software must not care.
- 5 Compare the latency of an on-chip AXI transaction, a PCIe read and a network round trip, in nanoseconds.

28. Interrupts, DMA and Platform Peripherals as Architecture

Peripherals are usually treated as a firmware topic, but three of them - the interrupt controller, the DMA engine and the timer subsystem - are architectural: they determine latency, they move data without the core, and they change what the processor must be able to do.

28.1 Interrupt controllers compared

	NVIC (Cortex-M)	GIC (Cortex-A)	PLIC/CLIC (RISC-V)
Coupling	Inside the core	Separate distributor + per-core interfaces	Platform-level, or core-local for CLIC
Vectoring	Hardware vector table lookup	The handler reads an acknowledge register	PLIC: read a claim register; CLIC: vectored
State saving	8 registers stacked in hardware	Software saves everything	Software
Latency	12 cycles, deterministic	Tens to hundreds of cycles	Implementation-defined
Priorities	Up to 256 levels with preemption	Priorities plus affinity routing across cores	Priority arbitration
Best for	Real-time control	Many sources across many cores	Either, depending on the profile chosen

INTERRUPT LATENCY AS AN ARCHITECTURAL BUDGET

The response time to an event is: hardware detection + controller arbitration + pipeline drain or exception entry + state saving + handler dispatch. On a Cortex-M4 at 100 MHz these are roughly $1 + 1 + 12 + 0$ (hardware-stacked) + 3 = **17 cycles = 170 ns**, and it is the **worst case**, not an average. On a 3 GHz out-of-order core the entry itself may be 300 cycles = 100 ns, but the variance from cache and TLB misses in the handler and from higher-priority activity can push the observed worst case into microseconds. **Deterministic microseconds beat fast-on-average nanoseconds** whenever a deadline must be guaranteed, which is why control loops stay on microcontrollers.

28.2 DMA as a second processor

A DMA engine is a programmable data mover with its own bus master port. Architecturally it matters for three reasons: it competes for bus and memory bandwidth with the CPU; it is a coherency participant (or, worse, is not); and it is a security principal, because an unrestricted DMA master can read all of memory - which is why IOMMUs exist.

Capability	Consequence
Scatter-gather descriptors	The engine walks a list of transfers without CPU help - the basis of every network and storage driver
Circular buffers with half/full interrupts	Continuous streaming with two interrupts per buffer instead of one per sample
Peripheral request lines	Transfers paced by the peripheral, so no polling is needed anywhere
Bus priority and burst size	Tuning knobs that decide whether the CPU or the DMA wins under contention
Coherency	On a cached system, either the DMA is coherent (snoops the caches) or software must clean and invalidate (Chapter 23)
Address translation	With an IOMMU, DMA uses virtual addresses and is confined to what the OS mapped

WHAT DMA IS WORTH

Streaming 16-bit samples at 1 MHz into memory: with a per-sample interrupt costing ~25 cycles of entry and exit plus a few of work, that is about 30 million cycles per second - **30% of a 100 MHz core**, doing nothing but copying. With DMA into a double buffer of 1,024 samples, the core is interrupted 1,953 times per second and spends perhaps 0.2% of its cycles. The data still consumes memory bandwidth - 2 MB/s - but the **processor** is free, which is the point.

28.3 Timers and the sense of time

- **A monotonic counter** (SysTick, the RISC-V `mtime` register, x86's TSC) is the architectural time base; everything else is derived from it. Reading it must be cheap, which is why it is memory-mapped or a register rather than a system call.
- **Comparators generate interrupts** at exact counts, which is what makes jitter-free periodic execution possible - the timer, not the software loop, defines when things happen.
- **Capture units timestamp inputs in hardware**, removing interrupt latency from the measurement entirely.
- **Watchdogs** are the architectural admission that software fails: an independent timer that resets the system unless it is serviced.
- **Synchronising time across a system** - between cores, between chips (PTP/IEEE 1588), or with a network - is its own discipline, and the hardware support (timestamping in the MAC, a shared counter across clusters) is what makes microsecond agreement possible.

Exercises

- 1 Compute the interrupt latency budget of a system you use, listing every contributing term.
- 2 Convert a polling loop to DMA and measure the change in CPU utilisation and in memory bandwidth.
- 3 Explain how an IOMMU would prevent a malicious peripheral from reading kernel memory.
- 4 Measure the jitter of a software timer against a hardware timer interrupt on the same device.
- 5 Compare the interrupt priority mechanisms of NVIC and GIC and describe a case each handles better.

29. Choosing a Device: MCU, MPU, DSP, FPGA or ASIC

Every architectural idea in this book eventually becomes a purchasing decision. This chapter is the framework for making it, and the arithmetic that turns a vague requirement into a defensible choice.

29.1 The five options

Option	Best at	Costs	Time to working system
Microcontroller	Deterministic control, low power, low cost	Limited compute and memory	Days
Application processor + OS	Connectivity, UI, filesystems, general software	Power, boot time, non-determinism, BOM	Weeks
DSP	Regular signal processing at low power	Specialised toolchain and skills	Weeks
FPGA	Custom parallel hardware, hard real-time I/O, evolving standards	Unit cost, power, RTL expertise (Chapter 34)	Months
ASIC / custom silicon	Volume, efficiency, integration	Millions of dollars and 1-2 years of NRE	Years

WHEN CUSTOM SILICON PAYS

Suppose an ASIC costs \$8M in non-recurring engineering and \$2 per unit, while an off-the-shelf part costs \$9 per unit. The crossover is $8,000,000 / (9 - 2) = \mathbf{1.14 \text{ million units}}$. Below that, buy the standard part; above it, and if the design is stable enough to freeze for two years, the custom part wins - and it also wins on power and board area, which may matter more than the money. The same calculation with an FPGA in the middle (higher unit cost, no NRE, reprogrammable) is how most real decisions are actually made.

29.2 A decision procedure that works

Answer these before choosing

- ☐ What is the hardest real-time deadline, and is it hard or soft? (Chapter 24 versus Chapter 26.)
- ☐ What is the compute requirement in operations per second, and the memory footprint in bytes? Measure a prototype rather than estimating.
- ☐ What is the energy budget per operation and per day?
- ☐ What connectivity and software stack is required - and does that force an operating system?
- ☐ What is the production volume, and the expected product lifetime?
- ☐ What are the safety, security and certification obligations?
- ☐ What does the team already know, and what tools exist? This is a legitimate engineering input, not a cop-out.
- ☐ What is the second source if this part becomes unavailable?

BENCHMARK THE SHORTLIST ON YOUR OWN WORKLOAD

Vendor benchmarks measure the core; your product measures the system. Port the hottest 200 lines of your application to each candidate, run it from the memory it will really execute from, with the peripherals it will really use, and measure cycles and current. A week of this routinely overturns a decision that looked obvious from the datasheets - most often because the memory system, not the core, turned out to be the limit.

29.3 A selection worked through

CHOOSING A DEVICE FOR A VIBRATION-MONITORING PRODUCT

Requirements: sample three accelerometer axes at 4 kHz, compute a 1,024-point FFT every 250 ms, classify the spectrum with a small neural network, report over BLE, and run two years on a 2,000 mAh battery. **Compute:** a 1,024-point real FFT is about $5 \times 1024 \times \log_2(1024) = 51,200$ operations; four per second is 205 kOP/s - trivial. The classifier at 250 kMAC per inference, four times per second, is 1 MMAC/s - also small. **A Cortex-M4F at 80 MHz with CMSIS-DSP does the FFT in about 100 us**, so the duty cycle of computation is $4 \times 100 \text{ us} = 0.04\%$. **Energy:** two years from 2,000 mAh is $2,000 / (2 \times 8,760) = 114 \text{ uA average}$, which is comfortable for an M4F sleeping at 2 uA between bursts, and impossible for an application processor whose idle power alone is tens of milliamps. **Conclusion: a BLE microcontroller**, decided by the energy budget rather than the compute - which is the usual outcome, and the reason to compute both before choosing.

If the requirement had been...	The answer would change to
Camera input and image classification at 10 fps	An application processor or an MCU with an NPU: the compute rises by four orders of magnitude
A touchscreen user interface and Wi-Fi	An application processor - the software stack, not the compute, forces it
Sub-microsecond deterministic actuation	An MCU or an FPGA, never a Linux system (Chapter 24)
Ten million units and a fixed algorithm	Custom silicon becomes worth evaluating (the crossover above)
A protocol that will change after launch	An FPGA, or a device with generous headroom and field update

Exercises

- 1 Compute the ASIC crossover volume for a product you know, with real numbers.
- 2 Take one requirement of a current project and decide MCU versus MPU from the deadline and the software stack alone.
- 3 Port a hot kernel to two candidate devices and compare cycles, energy and code size.
- 4 Identify a product that made the wrong choice - a device that boots for thirty seconds, or one that cannot be updated - and say what it should have used.
- 5 Write the one-page justification for the device your current project uses, as if for a design review.

PART VI

Physical and Modern

The constraints that now decide architecture - power, heat, reliability and security - together with the accelerators that answer them, the RTL design flow that turns an idea into silicon, and where the field is heading.

30. Power, Clocking and Thermal Limits

Performance stopped being limited by transistors and started being limited by energy around 2005, and everything since - multicore, heterogeneous cores, accelerators, aggressive sleep states - is a response to that. This chapter is the physics and the arithmetic behind it.

30.1 The two power terms

$$\text{Dynamic } P_{\text{dyn}} = \alpha \times C \times V^2 \times f$$

α = activity factor (fraction of nodes switching per cycle)

C = switched capacitance, V = supply, f = frequency

Static $P_{\text{static}} = V \times I_{\text{leak}}$ (grows with temperature and with smaller, faster transistors)

Energy per operation $E = P \times t = \alpha C V^2$ (independent of f !)

$$\text{Total } P = \alpha C V^2 f + V I_{\text{leak}}$$

The V-squared term is why voltage scaling was the most valuable lever in the industry's history - and why its end changed everything.

WHY DVFS WORKS, AND WHAT IT IS WORTH

Reduce frequency by 20% and, because a lower frequency permits a lower voltage, reduce V by 20% as well. Dynamic power scales as $V^2 f$, so it falls to $0.8^2 \times 0.8 = \mathbf{0.512}$ - roughly half the power for 80% of the performance. **Energy per operation** falls as V^2 to **0.64**, so the work also costs a third less energy. That is the whole case for dynamic voltage and frequency scaling. The counter-argument is **race to idle**: if static power is significant and the platform can enter a deep sleep state, finishing quickly at full speed and switching everything off can use less total energy than running slowly. Which wins depends on the leakage share and on how deep the idle state is - so it is measured, not assumed.

DENNARD SCALING, AND ITS END

Until about 2005, shrinking transistors let voltage fall in proportion, so power density stayed constant while both transistor count and frequency rose: **free performance every two years**. Voltage then stopped scaling, because threshold voltages could not fall further without leakage exploding. Transistor counts kept growing (Moore's law continued for a while longer) but they could no longer all be switched at full speed within the power budget - the **dark silicon** problem. The architectural consequences define the modern era: many cores instead of faster ones, specialised accelerators that do more work per joule, aggressive power gating, and heterogeneous designs where the right core is chosen per task.

30.2 The techniques, and what each saves

Technique	Attacks	Typical saving
Clock gating	Dynamic power in idle blocks	10-30% of dynamic; nearly free to implement
Power gating	Leakage in idle blocks	Most of the leakage, at the cost of state loss and wake-up latency
DVFS	Both, when performance can be traded	Up to 50% for a 20% frequency reduction
Multiple voltage domains	Letting each block run at its own point	Significant; costs level shifters and complexity

Technique	Attacks	Typical saving
Near-threshold operation	Energy per operation	Up to 10x better energy efficiency at a large frequency cost
Heterogeneous cores	Matching the work to the core	3-10x for background tasks moved to a small core
Accelerators	Doing the work with far fewer transistor switches	10-1000x for the workload they target (Chapter 33)
Reducing data movement	The dominant energy cost (Chapter 5)	Often the largest single win available

30.3 Clocking a large chip

- **Distribution:** a clock must reach millions of flip-flops with bounded skew, through an H-tree or a mesh. The clock network alone can consume 20-40% of a chip's dynamic power, which is why gating it is so valuable.
- **Skew and jitter** eat directly into the timing budget of Chapter 3, and both grow with distance - one reason large chips are partitioned into many clock domains.
- **Multiple domains** mean clock-domain crossings everywhere, with the synchroniser discipline of Chapter 3 applied at every boundary.
- **Adaptive clocking** slows the clock for a few cycles when the supply droops, instead of designing the whole chip for the worst-case droop - a technique that buys several percent of frequency.

30.4 Heat, and why it is the real limit

$$T_{\text{junction}} = T_{\text{ambient}} + P \times \theta_{\text{JA}}$$

A 5 W SoC in a package with $\theta_{\text{JA}} = 20 \text{ degC/W}$ in a 40 degC enclosure reaches $40 + 100 = 140 \text{ degC}$ - far above the $105\text{-}125 \text{ degC}$ limit. Either the power falls, the thermal path improves (heatsink, copper area, airflow), or the chip throttles.

Thermal design point (TDP) is a SUSTAINED budget; short bursts are allowed above it because silicon and packages have thermal mass - which is what 'turbo' is.

This is why phone benchmarks look excellent for thirty seconds and then fall by 40%, and why sustained performance, not peak, is the number that matters for anything that runs for minutes.

Exercises

- 1 Compute the dynamic power of a block at 1.0 V/1 GHz and at 0.8 V/800 MHz, and the energy per operation in each case.
- 2 Measure your machine's performance on a fixed workload for five minutes and plot the frequency; identify the moment it throttles.
- 3 Estimate the junction temperature of a device you use from its power and package thermal resistance.
- 4 Compare energy per instruction between a big and a little core on the same task, if your platform exposes energy counters.
- 5 Explain, with the equations above, why doubling the core count at half the frequency can be a net win.

31. Reliability, Error Correction and Test

Silicon fails: bits flip from cosmic rays, transistors age, manufacturing leaves defects, and a chip with a billion transistors cannot be exhaustively tested. Architecture answers with detection, correction, redundancy and testability - all of which cost area and must be justified by the application.

31.1 How things go wrong

Failure	Cause	Character
Soft error (single-event upset)	A neutron or alpha particle deposits charge in a cell	Transient: the value is wrong, the hardware is fine
Manufacturing defect	A particle or a process variation	Permanent; caught by test, or the die is discarded
Ageing (NBTI, electromigration, hot carriers)	Wear over years of operation	Gradual slowing, eventually a failure - mitigated by voltage margin and monitors
Marginal timing	A path that only fails hot, cold or slow	Intermittent, temperature-dependent, extremely hard to diagnose
Design bug (erratum)	A logic error that testing missed	Documented and worked around in software - every chip has them

HOW OFTEN DOES A BIT ACTUALLY FLIP?

Soft-error rates are quoted in **FIT**: failures per billion device-hours. Take a representative figure of 5 FIT per megabit for modern DRAM. A server with 64 GB holds 524,288 Mbit, so its rate is $524,288 \times 5 = 2.6\text{e}6$ FIT, i.e. $2.6\text{e}6/1\text{e}9 = 0.0026$ failures per hour - **one bit flip roughly every 380 hours, or about every 16 days, per machine**. A ten-thousand-machine fleet therefore sees around **26 per hour**, which is why hyperscale operators consider ECC mandatory and why field studies of large fleets report continuous correctable-error activity. Two multipliers make it worse: **altitude** (the neutron flux is several times higher at aircraft cruise, and hundreds of times higher in space) and **cell size** (smaller cells hold less charge, so a given particle is more likely to flip one). For a microcontroller with 64 KB of SRAM the same arithmetic gives a flip every few tens of thousands of years - which is why a thermostat has no ECC and a satellite has three votes.

31.2 Detection and correction

Scheme	Detects / corrects	Overhead	Where
Parity	Detects 1 bit	1 bit per byte or word	L1 caches (clean lines can simply be refetched)
SECCDED (Hamming)	Corrects 1, detects 2	8 bits per 64	DRAM, L2/L3 caches, on-chip memories in safety parts
Chipkill / SDDC	Survives the failure of a whole DRAM device	More redundancy per rank	Servers
CRC	Detects burst errors	Small	Interconnect links, storage, communications
Lockstep cores	Detects any divergence between two cores running the same code	2x the core area	Automotive and industrial safety (Cortex-R, TriCore)
Triple modular redundancy	Corrects by majority vote	3x plus a voter	Space, avionics
Scrubbing	Prevents accumulation of correctable errors	A background read pass	DRAM and large SRAMs

31.3 Testing a chip you cannot probe

- **Scan chains** stitch every flip-flop into a giant shift register so that any state can be loaded and read back. This is how manufacturing test achieves high fault coverage, and it costs a few percent of area and a little timing.
- **Built-in self-test (BIST)** puts pattern generators and analysers on the chip: memory BIST tests every SRAM at speed, and logic BIST does the same for random logic.
- **Boundary scan (JTAG)** tests the connections **between** chips on a board without a bed of nails - the same port used for debugging.
- **Binning and speed grades** follow from test: dice that fail at 3.5 GHz but pass at 3.0 are sold as slower parts, and dice with one faulty core are sold with that core disabled. Yield engineering is why product lines exist.
- **On-chip monitors** - ring oscillators, temperature and voltage sensors - report ageing and margin in the field, feeding the adaptive clocking of Chapter 30.

ERRATA ARE ARCHITECTURE TOO

Every processor ships with a list of documented defects and their workarounds, and some of those workarounds are in the compiler, the kernel or your firmware. Read the errata sheet for any part you depend on: 'the pipeline may corrupt a register if a specific instruction sequence follows an interrupt' is the kind of entry that explains an otherwise impossible bug, and the fix is usually one compiler flag away.

Exercises

- 1 Compute the SECDDED overhead for a 64-bit word and explain why 8 bits are enough.
- 2 Estimate the soft-error rate of a system you own, and decide whether ECC is justified for it.
- 3 Find the errata document for a processor you use and identify one workaround that affects software.
- 4 Explain how scan chains would let a tester find a stuck-at fault in the middle of an ALU.
- 5 Describe how a lockstep pair detects a fault, and what it does when it does.

32. Security Architecture

Hardware enforces the boundaries that software relies on: between processes, between an operating system and its applications, between a hypervisor and its guests, and between a device's secrets and its owner. This chapter covers the mechanisms that draw those boundaries - and the class of attacks that showed the boundaries were leakier than anyone had specified.

32.1 The mechanisms, from oldest to newest

Mechanism	Protects	Limitation
Privilege levels (Chapter 10)	The OS from applications	Only as strong as the syscall interface's own checks
MMU / page permissions	Processes from each other; execute-never and write-xor-execute stop code injection	Nothing above the OS; and translation is a shared, observable resource
IOMMU / SMMU	Memory from malicious or buggy DMA devices	Must actually be enabled and correctly configured
TrustZone / secure world	Keys and trusted services from a compromised OS	A large trusted-services codebase is itself attack surface
Secure boot with a hardware root of trust	The whole software stack's integrity	Only integrity, not confidentiality; and key management is the hard part
Memory encryption (SEV, TDX, CCA)	Data in DRAM from a physical attacker or an untrusted host	Timing and access patterns still leak
Pointer authentication, MTE, CFI hardware	Memory-safety exploitation	Mitigations, not guarantees
Physical countermeasures	Glitching, probing, side channels	Cost and area; a determined lab still wins eventually

32.2 Side channels: when performance features leak

WHY SPECULATIVE EXECUTION LEAKED SECRETS

Chapter 13's speculation executes instructions that are later discarded. Their **architectural** effects are undone - registers, memory. Their **microarchitectural** effects are not: a line loaded into the cache during a mis-speculated path stays there. An attacker who can steer a victim's branch prediction can therefore make the victim speculatively read a secret and use it to index an array, leaving a cache footprint that the attacker then measures by timing. **Nothing in the ISA was violated** - the specification never said anything about cache state, which is precisely the point. This is Spectre; Meltdown was the related case where a fault's permission check was resolved too late to stop the speculative load.

Channel	What it measures	Mitigation
Cache timing (Flush+Reload, Prime+Probe)	Which lines the victim touched	Partitioning, constant-time code, flushing on domain switches
Branch predictor state	Which way a victim branched	Tagging or flushing predictors on context and privilege switches
TLB and page-table walks	Which pages were touched	Kernel page-table isolation, ASIDs
Port and unit contention (SMT)	What another thread on the same core executed	Disable SMT across security boundaries, or schedule same-trust-domain threads together
Power and electromagnetic emissions	Key-dependent activity	Constant-power designs, masking, noise

Channel	What it measures	Mitigation
Fault injection (glitching)	Skipping a check by disturbing the supply or clock	Redundant checks, sensors, randomised timing

The architectural response has three parts: **isolate** shared predictive state across security boundaries (flush or tag it), give software **explicit barriers** to stop speculation where it matters (**lfence**, **csdb**, speculation barriers), and design **constant-time** code for cryptography so that no secret influences timing, branches or memory addresses at all.

THE COST OF THE MITIGATIONS

Kernel page-table isolation, indirect-branch barriers, retpolines and SMT restrictions together cost between a few percent and 30% depending on the workload - system-call-heavy and I/O-heavy code suffers most. That is a large architectural price paid permanently for a class of bug that was, in a real sense, a specification gap: the ISA never promised anything about microarchitectural state, so nobody had to prove anything about it. Modern ISA work takes that obligation seriously.

32.3 Building a trustworthy device

Architectural security checklist

- ☐ A hardware root of trust that cannot be reprogrammed, verifying the first mutable stage.
- ☐ Every subsequent stage verified before execution, with anti-rollback on versions.
- ☐ Keys stored where software cannot read them - fuses, a secure element, or a key store that only performs operations.
- ☐ A hardware random number generator, used to seed everything.
- ☐ Debug interfaces closed in production, with an authenticated unlock if diagnostics are needed.
- ☐ DMA-capable devices constrained by an IOMMU.
- ☐ Memory permissions applied (write-xor-execute), and hardware mitigations (pointer authentication, memory tagging) enabled where available.
- ☐ Cryptography implemented in constant time, or in a hardware engine.
- ☐ A documented threat model that says what is **not** defended - usually a determined attacker with physical possession and a laboratory.

Exercises

- 1 Explain, in five sentences, how a Spectre v1 gadget leaks a byte.
- 2 Measure the cost of a system call with and without the kernel's mitigations enabled, if your system allows toggling them.
- 3 Write a timing-attack demonstration on a table-based AES implementation, then repeat with a constant-time one.
- 4 List which hardware security features your own device has and which of them are actually enabled.
- 5 Describe how you would protect a device whose attacker is its owner - and be honest about what is achievable.

33. Accelerators: GPUs, NPUs, DSPs and Custom Silicon

When a general-purpose core costs a hundred times more energy per useful operation than a specialised one, specialisation wins. This chapter is about why the gap is so large, what the main accelerator families look like architecturally, and how to decide whether to build or use one.

33.1 Where the hundred-fold gap comes from

THE ENERGY BUDGET OF ONE ADD

A 32-bit integer add costs on the order of **0.1 pJ** in a modern process. Executing that add as an **instruction** on a general-purpose out-of-order core also costs instruction fetch, decode, rename, scheduling, register-file access, reorder-buffer bookkeeping and retirement - **10 to 100 pJ**, so **99% or more of the energy is overhead, not arithmetic**. An accelerator removes that overhead by amortising control across many operations: one instruction driving 1,024 multiply-accumulate units, with operands flowing between them directly instead of through a register file. That is the entire principle behind every accelerator in this chapter - and it also explains their weakness, since the amortisation only works if the work really is regular.

33.2 The families

Accelerator	Architecture	Wins on	Loses on
GPU	Thousands of SIMT lanes, huge register file, bandwidth-optimised memory	Dense regular parallelism, graphics, training	Branchy, latency-sensitive or small work
NPU / TPU-class	Systolic arrays of MACs with local reuse, INT8 or bf16 arithmetic	Neural-network inference and training	Anything that is not a tensor operation
DSP	VLIW issue, MAC units, specialised addressing	Filters, transforms, control loops at low power	General-purpose code, large memories
FPGA	A fabric of lookup tables, flip-flops, DSP slices and RAM blocks, configured after manufacture	Custom dataflow, hard real-time I/O, protocols that change	Clock rate, power, and engineering effort
Fixed-function block	A hardwired pipeline (video codec, crypto, image signal processor)	Efficiency: often 100-1000x a CPU	Zero flexibility - it does one thing

33.3 The systolic array, in one page

Matrix multiply on a 4x4 array of MAC cells:

```

weights held in the cells (stationary)
activations flow in from the left ->
partial sums flow downward

a3 a2 a1 a0 -> [w][w][w][w] -> each cell: acc += a x w
                  | | | |      then passes a right, acc down
                  [w][w][w][w]
                  | | | |      N x N cells perform N^2 MACs
                  [w][w][w][w] per cycle while reading only
                  | | | |      2N values from memory:
                  [w][w][w][w] arithmetic intensity ~ N/2.
                  v v v v
                  results
  
```

The point is not the multipliers - a CPU has those - but the **data reuse**: each value fetched from memory is used N times inside the array, which is what converts a memory-bound problem (Chapter 17) into a compute-bound one.

- **Quantisation** to INT8 or lower is what makes these arrays affordable: an INT8 multiply-accumulate costs roughly a tenth of the energy and a quarter of the area of an FP32 one, and a fifth of the memory bandwidth per value.
- **Sparsity support** skips zero weights, and structured sparsity (fixed patterns such as 2-of-4) is what hardware can actually exploit without losing the regularity that made it efficient.
- **The memory system is usually the limit**, not the arithmetic: an accelerator's specification quotes TOPS, but its useful throughput is set by how much data reuse the mapping achieves - which is why compilers for accelerators are mostly loop-transformation tools.
- **Integration matters as much as the core**: whether the accelerator is cache-coherent with the CPU (Chapter 27), whether it has its own IOMMU context, and how much it costs to hand work to it decide whether small operations are worth offloading at all.

THE OFFLOAD BREAK-EVEN CALCULATION

Handing work to an accelerator costs setup, data movement and synchronisation - typically **5-100 microseconds** for a discrete GPU and 1-10 for an on-chip block. If the kernel itself takes less than that, offloading **loses**, however fast the accelerator is. Always compute the break-even size before designing an offload path, and prefer moving a batch of work rather than a single item.

Exercises

- 1 Compute the arithmetic intensity of an $N \times N$ systolic array and explain why it rises with N .
- 2 Measure the fixed offload cost of a GPU or NPU on your platform, and compute the smallest problem worth offloading.
- 3 Compare energy per inference between a CPU and an accelerator for the same model, if your platform exposes energy counters.
- 4 Explain why quantisation helps an accelerator more than it helps a general-purpose CPU.
- 5 Pick a fixed-function block in a device you own (video decoder, crypto engine) and estimate what its work would cost on the CPU.

34. RTL Design and the Path to Silicon

Everything described so far is eventually written as **register transfer level** code - a description of what each register holds on each clock edge and what combinational logic computes in between - and then transformed by tools into transistors. Knowing that flow makes the architecture concrete, and it is how you would build any of it yourself.

34.1 RTL is registers plus logic, and nothing else

```
// The synthesisable pattern: sequential and combinational separated.
module counter #(parameter W = 8) (
    input          clk, rst_n, enable,
    output logic [W-1:0] count
);
    logic [W-1:0] next;

    always_comb next = enable ? count + 1'b1 : count;    // logic

    always_ff @(posedge clk or negedge rst_n)           // registers
        if (!rst_n) count <= '0;
        else      count <= next;
endmodule
```

Two rules prevent most beginner disasters: use non-blocking assignment (`<=`) in clocked blocks and blocking (`=`) in combinational ones, and never assign the same signal from two different blocks.

```
// A fragment of a single-cycle RV32I core: decode and execute.
// This is the datapath of Chapter 11, written as it would be built.
always_comb begin
    opcode = instr[6:0]; rd = instr[11:7]; funct3 = instr[14:12];
    rs1    = instr[19:15]; rs2 = instr[24:20]; funct7 = instr[31:25];

    imm_i = {{20{instr[31]}}, instr[31:20]};    // sign-extend
    imm_s = {{20{instr[31]}}, instr[31:25], instr[11:7]};

    alu_a = rf[rs1];
    alu_b = (opcode == OP_IMM || opcode == LOAD) ? imm_i : rf[rs2];

    unique case (funct3)
        3'b000: alu_y = (funct7[5] && opcode == OP)
                     ? alu_a - alu_b : alu_a + alu_b;
        3'b111: alu_y = alu_a & alu_b;
        3'b110: alu_y = alu_a | alu_b;
        3'b100: alu_y = alu_a ^ alu_b;
        default: alu_y = 'x;
    endcase
end
```

Compare this with the format diagram of Chapter 7: the bit ranges are taken directly from it, and the regularity of the encoding is what makes the decode logic this short.

34.2 The flow, from text to silicon

```

RTL (SystemVerilog)
|  simulate + assertions + constrained-random tests
v
SYNTHESIS ---- library of standard cells + timing constraints (SDC)
|  -> gate-level netlist
v
PLACE AND ROUTE ---- floorplan, clock tree, wires
|  -> layout with real wire delays
v
STATIC TIMING ANALYSIS (Chapter 3's setup/hold, over corners:
|                        slow/fast process, voltage, temperature)
v
DRC / LVS / signoff -> MASKS -> fabrication -> test (Ch 31)

FPGA flow: the same first two steps, then 'place and route' into
lookup tables and block RAMs - and a bitstream in minutes to hours,
instead of masks in months.

```

The difference in iteration time between FPGA and ASIC - hours against months, and thousands of dollars against millions - is why almost every design is proven in an FPGA or an emulator first.

Stage	What can go wrong	How it is caught
RTL	Functional bugs	Simulation, constrained-random testing, assertions, formal property checking
Synthesis	Code that simulates but does not synthesise; latches inferred by accident	Lint tools and synthesis warnings - never ignore an inferred latch
Timing	Paths that miss setup or hold at some corner	Static timing analysis across process, voltage and temperature corners
Clock domain crossings	Metastability (Chapter 3)	CDC analysis tools, plus a strict synchroniser discipline
Physical	Congestion, IR drop, electromigration	Place-and-route reports and signoff checks
Post-silicon	Everything that escaped	Bring-up, scan/BIST, errata and metal-layer respins

VERIFICATION IS MOST OF THE WORK

On a serious chip project, **60-80% of the engineering effort is verification**, not design. The reason is economic: a bug found in simulation costs an hour, one found after tapeout costs a mask set and three months. This is why the industry invented constrained-random verification, coverage-driven closure, formal property checking and hardware emulation - and it is the single biggest cultural difference between hardware and software engineering.

34.3 Higher-level ways in

Approach	Idea	Where it fits
SystemVerilog RTL	The industry standard	Everything production
Chisel / SpinalHDL / Amaranth	Hardware generators written in a general-purpose language, producing Verilog	Parameterised families of designs; used for several RISC-V cores
High-level synthesis (C/C++ to RTL)	Describe the algorithm, let the tool build the pipeline	Dataflow accelerators; good results need HLS-aware code
FPGA overlays and softcores	A processor or coarse-grained fabric on the FPGA	Quick programmability without a full RTL flow

HOW TO LEARN THIS FOR REAL

Write a single-cycle RV32I core in about 300 lines of SystemVerilog, simulate it against a handful of assembly tests, then run it on a cheap FPGA board. Then pipeline it (Chapter 12) and watch the hazard logic appear exactly where the theory says it will. That project - a few weekends - teaches more about processor architecture than any amount of reading, this book included.

Exercises

- 1 Write and simulate the counter above, then deliberately use blocking assignment in the clocked block and explain the difference in the waveform.
- 2 Extend the decode fragment to handle loads, stores and branches.
- 3 Write a testbench that drives 100 random instructions and checks the results against a reference model.
- 4 Synthesise a small design for an FPGA and read the timing report: find the critical path and shorten it.
- 5 Estimate the cost and schedule difference between fixing a bug in RTL, in an FPGA prototype, and after tapeout.

35. Where Computer Architecture Is Going

Two of the three sources of historical performance growth have stopped: Dennard scaling ended around 2005, and the economics of Moore's law have flattened since. What remains is architecture itself - which is why this is a more interesting time to study it than the thirty years when performance arrived on schedule regardless.

35.1 The forces

Force	Consequence
Dennard scaling ended	Power, not transistors, is the budget; dark silicon; heterogeneous designs
Cost per transistor has flattened	Bigger monolithic dies are no longer automatically cheaper - hence chiplets
Memory latency is stuck	Everything is designed to tolerate it rather than reduce it
Machine learning workloads exploded	Enormous, regular, low-precision arithmetic - the ideal target for specialisation
Security became architectural	Isolation and side channels are now first-class design requirements, not afterthoughts
Open ISAs matured	Custom processors are now feasible for teams that could never have licensed one

35.2 What is actually being built

- **Chiplets and advanced packaging.** Rather than one enormous die, several smaller ones - each on the process node that suits it - joined by an interposer or a die-to-die link such as UCIe. Yields improve, mixed processes become possible, and the interconnect of Chapter 27 extends across packages. 3D stacking goes further, putting cache or memory directly above the logic that uses it.
- **Domain-specific architectures.** The 'new golden age' argument: when general-purpose performance grows slowly, the returns from specialisation are enormous, and we now have the tools (HLS, generators, open ISAs) to build specialised machines affordably.
- **CXL and disaggregated memory.** Coherent, byte-addressable memory attached over a serial link, so capacity can be pooled across machines and tiered between fast and slow media - the hierarchy of Part IV extended past the package.
- **Processing in and near memory.** Putting simple compute inside the DRAM or on the memory stack, because moving data costs more than computing on it (Chapter 5). Commercial products exist; the hard part is the programming model.
- **New memory technologies.** MRAM is already replacing embedded flash in some microcontrollers; ReRAM and phase-change memory promise density between DRAM and flash. Each changes the hierarchy's shape rather than merely its numbers.
- **Vector and matrix extensions everywhere,** from RVV and SVE on application processors down to Helium on microcontrollers - the edge of the machine-learning wave reaching devices that run on batteries.

QUANTUM AND NEUROMORPHIC, HONESTLY

Quantum computing is a genuinely different model of computation with real prospects for specific problems - factoring, simulation of quantum systems, some optimisation - and it is not a faster classical computer. Current devices are limited by error rates, and useful fault-tolerant machines require error correction with large qubit overheads. Neuromorphic designs - spiking, event-driven, extremely low power - are promising for always-on sensing. Both are worth watching; neither changes anything in Parts I to VI of this book for the foreseeable future.

35.3 What will still be true in twenty years

THE DURABLE PRINCIPLES

Locality - because moving data will always cost more than computing on it. **Parallelism** - because it is the only way to use more transistors within a fixed power budget. **Specialisation** - because the general-purpose overhead of Chapter 33 is enormous wherever the work is regular. **Amdahl's law** - because the serial part always dominates in the end. And **measurement** - because every one of these principles is a quantitative claim about a specific workload, and the only way to apply it is to measure. Whatever the technology, those five will still be how you reason about a machine.

Exercises

- 1 Find a recent processor built from chiplets and identify what each chiplet contains and why.
- 2 Estimate the benefit of near-memory computing for a workload you know, using the energy figures from Chapter 5.
- 3 Read the specification of one accelerator announced in the last year and classify it using Chapter 33's table.
- 4 Argue both sides of: 'general-purpose cores will keep most of the market'. Use numbers.
- 5 Pick one of the five durable principles and trace where it appeared in five different chapters of this book.

PART VII

Practice

Turning the theory into results: the tools that show what a machine is really doing, the code changes that follow from each chapter, and one line of C traced from the source file to the transistors.

36. Tools: Counters, Profilers and Simulators

A processor is opaque unless you instrument it. Every modern core contains a performance-monitoring unit that counts what actually happened, and using it converts architecture from a subject you have read about into one you can observe.

36.1 Performance counters

Counter	Tells you	Chapter
Instructions, cycles	IPC - the first number to look at	17
Branch instructions and misses	Whether prediction is the problem	13
L1/L2/LLC loads and misses	Where the memory hierarchy is failing	18-19
dTLB and iTLB misses	Whether the working set exceeds TLB reach	20
Stall cycles by resource	What the back end is waiting for	15
Memory bandwidth (uncore)	How close to the DRAM roofline you are	17, 21
Energy (RAPL and equivalents)	Joules per unit of work	30

```
# The three commands that answer most questions
perf stat -d ./program          # IPC, cache and branch summary
perf record -g ./program        # sampling profile with call graphs
perf report                     # ...and where the time went

# Targeted questions
perf stat -e cycles,instructions,cache-misses,branch-misses ./prog
perf stat -e dTLB-load-misses,LLC-load-misses ./prog
perf c2c record / report        # find false sharing (Chapter 23)

# On a microcontroller: the cycle counter is the equivalent
# t0 = DWT->CYCCNT; work(); cycles = DWT->CYCCNT - t0;
```

COUNTERS LIE IN SPECIFIC, KNOWABLE WAYS

Sampled counts are statistical, so a small difference is noise. Events are often attributed to a nearby instruction rather than the exact one (use precise-event sampling where available). Frequency scaling makes 'cycles' and 'seconds' disagree. Hardware prefetchers make cache-miss counts smaller than the misses a naive model predicts. And on a shared machine, another tenant's activity shows up in your last-level cache counters. **Always compare a measurement against a baseline taken the same way, on the same machine, in the same session.**

36.2 Profiling strategies

Method	Shows	Cost
Sampling profiler	Where the time goes, statistically	1-5% - safe to run in production
Instrumentation	Exact counts and call graphs	10-100% - changes the thing being measured
Top-down analysis (Chapter 17)	Which architectural resource is the limit	Low
Cache simulator (Valgrind/cachegrind)	Miss counts for a hypothetical cache	20-100x slowdown, but deterministic and repeatable
Full-system simulator (gem5)	Cycle-level behaviour of a machine you do not have	1,000-100,000x - used for architecture research

Method	Shows	Cost
Hardware tracing (Intel PT, ARM ETM)	The exact instruction path	A few percent, with a lot of data

THE ORDER TO USE THEM IN

Start with `perf stat` on the whole program to learn whether you are front-end bound, back-end bound, badly speculating or simply executing too many instructions. Then sample to find **where**. Only then reach for a simulator or a microbenchmark, and only for the one loop that matters. Reversing this order - starting with a microbenchmark - is how weeks get spent optimising code that was never on the critical path.

36.3 A top-down analysis, walked through

READING FOUR NUMBERS AND KNOWING WHAT TO DO

A run reports: IPC **0.62**, retiring **18%**, bad speculation **6%**, front-end bound **9%**, back-end bound **67%** - and within the back end, memory bound **58%**, of which DRAM bound is **41%**. The reading is immediate and requires no guessing: **two thirds of the machine's issue slots are lost waiting for memory, and most of that is DRAM**, not cache. So do not vectorise (the arithmetic is not the limit), do not chase branches (6% is unremarkable), and do not reduce the instruction count. **Fix the data access**: check the arithmetic intensity, look for a layout change, block the loops, or reduce the footprint - Chapters 19 and 37. If instead retiring had been 70% and IPC 2.8, the machine would already be working well and the only remaining lever would be a better algorithm.

Dominant category	Read it as	Go to
Retiring high, IPC high	The machine is busy doing your work	Reduce the work: algorithm, or Chapter 16's SIMD
Bad speculation	Branches are unpredictable	Chapter 13; consider branchless or sorted data
Front-end bound	Instructions are not being delivered	Code size and layout, inlining policy, PGO
Back-end, core bound	Execution ports or dependences	Chapter 14: break dependence chains, unroll
Back-end, memory bound (L1/L2)	Working set too large for the close caches	Chapter 19: blocking, layout
Back-end, memory bound (DRAM)	Streaming or random over a large footprint	Chapters 17, 21: intensity, prefetch, compression

36.4 Microbenchmarks that tell you about your machine

- **Pointer chase** over a randomly permuted array of varying size: gives the latency of each cache level and of DRAM directly, because each access depends on the previous one and nothing can overlap.
- **Stream (copy, scale, add, triad)**: gives sustained memory bandwidth - the denominator of every roofline.
- **A dependent chain of adds**: gives the latency of the ALU; a **parallel** set of adds gives the throughput. The ratio is the machine's ILP for that operation.
- **A loop of taken branches with a random pattern**: gives the misprediction penalty when compared with a predictable version.
- **Increasingly large working sets with a fixed stride**: gives the TLB reach as a second staircase beyond the cache one.
- Run all five once on any new machine, keep the numbers, and every later measurement has a context to be interpreted in.

Exercises

- 1 Run `perf stat` on a program you wrote and interpret every line of the output.
- 2 Find a loop whose IPC is below 1.0 and determine which resource is the limit.

- 3 Use a cache simulator to predict the miss rate of a kernel, then compare with the hardware counters and explain any difference.
- 4 Measure the same benchmark ten times and report the median and the spread; decide what difference would be significant.
- 5 On a microcontroller, measure a function with a cycle counter and with a GPIO on a scope, and reconcile the two.

37. Writing Code That Fits the Machine

Every chapter of this book implies something about how to write fast code. This chapter collects those implications and applies them to one kernel, measuring after each step, so that the size of each effect is visible rather than asserted.

37.1 A worked optimisation, step by step

The kernel: for a particle simulation, update one million particles - read position and velocity, apply a force, write the new position. Baseline is a naive array-of-structures loop with a branch inside.

Step	Change	Why it helps	Cumulative speed
0	Naive: array of structs with 12 fields; a branch per particle	-	1.0x
1	Split into structure of arrays; touch only the 4 fields the loop needs	Bytes moved fall by 3x; every cache line is fully used (Chapter 18)	2.4x
2	Remove the data-dependent branch with a branchless clamp	Misprediction rate falls from 12% to 0 (Chapter 13)	3.1x
3	Block the loop so the working set fits in L2	Capacity misses removed (Chapter 19)	3.6x
4	Vectorise with 8-wide SIMD	8 lanes per instruction, now that the data is contiguous (Chapter 16)	6.9x
5	Parallelise across 8 cores with per-core output arrays	Thread-level parallelism, no false sharing (Chapter 23)	34x
6	Use 32-bit floats instead of 64-bit	Half the bytes moved, twice the SIMD lanes (Chapters 4, 17)	48x

READ THE ORDER, NOT JUST THE NUMBERS

Steps 1 to 3 - **data layout and control flow** - deliver a 3.6x before a single vector instruction is written, and they are what make step 4 possible at all: SIMD on an array-of-structures layout would have gained almost nothing, because the loads would still have been scattered. **Fix the memory access pattern first, then the branches, then vectorise, then parallelise.** Doing it in the other order is the most common way to spend a week for 10%.

37.2 The checklist, by chapter

Symptom (from counters)	Likely cause	What to try
Low IPC, high cache misses	Poor locality	SoA layout, blocking, prefetching, smaller types (Ch 18-19)
High dTLB misses	Working set exceeds TLB reach	Huge pages, or reduce the footprint (Ch 20)
High branch misses	Unpredictable control flow	Branchless code, sorting the data, restructuring the loop (Ch 13)
Front-end bound	Instruction footprint too large	Reduce inlining, improve code layout, PGO (Ch 15)
High memory bandwidth, low IPC	Memory-bound kernel	Raise arithmetic intensity: fuse loops, block, compress (Ch 17)
Poor scaling with threads	Contention or false sharing	Shard the data, pad shared lines, use per-core state (Ch 23)
Good IPC but slow anyway	Too many instructions	A better algorithm - the only fix that can win by orders of magnitude

THE TWO RULES THAT KEEP OPTIMISATION HONEST

Measure before and after, on the real input. A change that helps a microbenchmark and hurts the application is a common outcome, because the microbenchmark had the cache to itself. **And check the result is still correct** - especially after floating-point reassociation (Chapter 4), which can change the answer in the last bits and occasionally in ways that matter.

37.3 The transformations, in code

```
// 1. Array of structures: 48 bytes per particle, 12 fields.
//    The loop uses 4 of them, so 75% of every cache line is waste.
struct P { float x,y,z, vx,vy,vz, m, r, pad[4]; };
P p[N];
for (int i = 0; i < N; i++) p[i].x += p[i].vx * dt;

// 2. Structure of arrays: the loop touches two dense streams,
//    every byte of every line is used, and it can be vectorised.
struct Particles { float *x, *y, *z, *vx, *vy, *vz; };
for (int i = 0; i < N; i++) px[i] += pvx[i] * dt;

// 3. Restrict tells the compiler the arrays do not alias, which is
//    usually what BLOCKS auto-vectorisation in real code.
void step(float * restrict px, const float * restrict pvx,
         float dt, int n)
{ for (int i = 0; i < n; i++) px[i] += pvx[i] * dt; }
```

Step 3 is the one people miss: the compiler must assume that **px** and **pvx** may overlap unless told otherwise, and that assumption alone forces it to keep the loop scalar.

```
// 4. Blocking, so the working set of the inner loops fits in cache.
for (int ii = 0; ii < N; ii += B)
    for (int jj = 0; jj < N; jj += B)
        for (int i = ii; i < ii + B; i++)
            for (int j = jj; j < jj + B; j++)
                c[i] += a[i][j] * b[j]; // a[][] tile stays hot

// 5. Per-thread accumulators avoid false sharing (Chapter 23).
struct alignas(64) Acc { double sum; }; // one cache line each
Acc acc[NTHREADS];
```

CHOOSING THE BLOCK SIZE

Pick B so that the tiles used by the inner loops fit in the level of cache you are targeting, with room to spare for everything else. For three 32-bit tiles in a 32 KB L1: $3 \times B^2 \times 4 \leq 32,768$ gives $B \leq 52$, so $B = 32$ or 48 (a power of two or a multiple of the vector width is convenient). Then **measure** - the best value is often half what the arithmetic suggests, because the cache is shared with everything else the program is doing.

Exercises

- 1 Take a loop of your own and run the six steps above, measuring after each. Record which ones did nothing.
- 2 Convert one array-of-structures to structure-of-arrays and measure both the time and the bytes moved.
- 3 Find the arithmetic intensity of your hottest kernel and place it on your machine's roofline before deciding what to optimise.
- 4 Deliberately introduce false sharing into a parallel loop and measure the damage, then fix it.
- 5 Compare a hand-vectorised loop with the compiler's auto-vectorised version, and explain any difference in the generated code.

38. One Line of C, All the Way Down

This last chapter traces a single statement through every layer this book has built, from the source file to the transistors, accounting for the time it takes at each level. It is the whole book in one example.

```
// The statement, inside a loop over a large array:
sum += a[i] * b[i];
```

38.1 Layer 1: the compiler

The compiler sees the whole loop. It keeps `sum` and `i` in registers, unrolls the loop by four to expose independent work (Chapter 14), vectorises it if the arrays do not alias (Chapter 16), and emits a fused multiply-add so that one instruction does both operations with a single rounding (Chapter 4). What was one line becomes, per four elements, two vector loads, one FMA and a pointer increment.

```
loop:  vle32.v   v0, (a0)           # load 8 floats from a[]
      vle32.v   v1, (a1)           # load 8 floats from b[]
      vfmacc.vv v2, v0, v1         # v2 += v0 * v1, elementwise
      addi      a0, a0, 32
      addi      a1, a1, 32
      addi      a2, a2, -8
      bnez      a2, loop
```

38.2 Layer 2: the instruction stream

Each of those instructions is a 32-bit word in memory (Chapter 7), fetched from the instruction cache. The branch at the bottom is predicted taken by the branch predictor (Chapter 13) with essentially perfect accuracy after the first iteration, so the front end never stalls waiting to know where to go next.

38.3 Layer 3: the core

The instructions are decoded, renamed onto physical registers (Chapter 15) so that successive iterations do not conflict on `v0` and `v1`, and dispatched into the scheduler. The two loads issue to the load unit; the FMA waits for both. Because the machine is out of order, iteration `i+1`'s loads issue while iteration `i`'s FMA is still executing - the memory-level parallelism that hides latency.

38.4 Layer 4: the memory system

```

vle32.v -> address = a0
          |
          v
+-----+ hit (4 cycles) -> data to the register
| L1 D-cache | ----->
+-----+
| miss      | (Chapter 18)
|          |
|          v
+-----+ hit (14 cycles)
| L2      | ----->
+-----+
| miss    |
|          v
+-----+ hit (45 cycles)
| L3 / SLC | ----->
+-----+
| miss    | -> DRAM: row activate + CAS (Chapter 21)
|          | 250+ cycles, 64 bytes returned
|          v
+-----+
| DRAM    |
+-----+

```

In parallel: the TLB translates the virtual address (Chapter 20).
 A stride-detecting prefetcher has probably already requested the
 next lines, so most iterations hit in L1 after all.

Sequential access is the best case in every part of this diagram, which is why this loop runs near the memory bandwidth limit rather than the latency limit.

38.5 Layer 5: the arithmetic, the gates, the transistors

The FMA unit multiplies two 24-bit significands in a Wallace tree, adds the accumulator through a carry-save path, normalises and rounds once (Chapter 4). Every one of those steps is gates (Chapter 2), and every gate is a handful of CMOS transistors switching a capacitance and burning $C V^2$ of energy (Chapter 30). The result is written to a physical register - a multiported SRAM-like cell array (Chapter 5) - and, when the instruction retires in order, becomes architecturally visible (Chapter 15).

38.6 The accounting

WHERE THE TIME ACTUALLY GOES

Per 8 elements: 2 vector loads (64 bytes), 1 FMA (16 FLOPs), and 4 cheap integer instructions. **Arithmetic intensity** = 16 FLOP / 64 bytes = **0.25 FLOP/byte** - the same dot product as Chapter 16. On a machine with 25 GB/s of bandwidth, the ceiling is 6.25 GFLOP/s regardless of vector width, and the loop will run at that rate for arrays larger than the last-level cache. For arrays that **fit in L2**, bandwidth is roughly 20x higher and the same code runs several times faster - **the identical instructions, differing only in where the data lives**. That single observation is, in the end, what this entire book has been about: the machine is fast, the memory is far away, and architecture is everything people have invented to bridge the gap.

38.7 Where to go next

- **Build a processor** (Chapter 34's project). Nothing else consolidates Parts I to III so quickly.
- **Write a cache simulator** and validate it against hardware counters - Part IV becomes concrete in a weekend.
- **Optimise one real kernel** with the discipline of Chapter 37, and write down the number after each step.
- **Read the optimisation manual** for a processor you use. They are long, specific and unexpectedly readable once this book's vocabulary is in place.
- **Follow the primary literature**: ISCA, MICRO, ASPLOS and HPCA publish the ideas that reach products five to ten years later.

Exercises

- 1 Compile the dot-product loop and read the assembly; identify the unrolling, the vectorisation and the FMA.
- 2 Measure its performance for array sizes that fit in L1, L2, L3 and DRAM, and plot the result. You will see the hierarchy directly.
- 3 Compute the arithmetic intensity of the kernel and check the measured GFLOP/s against your machine's roofline.
- 4 Repeat the measurement with the two arrays interleaved into one array of pairs, and explain the change.
- 5 Trace one instruction of your own choosing through all five layers as this chapter did, and write it up.

APPENDICES

Appendices

A formula and reference sheet, a glossary of every term used in this book, and a study roadmap with projects and tools.

Appendix A. Formula and Reference Sheet

Performance

Iron law $\text{time} = \text{instructions} \times \text{CPI} \times \text{clock period}$
 CPI $= 1 + \text{stalls per instruction}$
 Speedup $= T_{\text{old}} / T_{\text{new}}$
 Amdahl $S = 1 / ((1 - f) + f/s)$
 Little's law $N = \text{throughput} \times \text{latency}$
 MFU-style utilisation = achieved / peak
 Roofline $\text{attainable} = \min(\text{peak FLOP/s}, \text{AI} \times \text{peak bandwidth})$
 arithmetic intensity $\text{AI} = \text{FLOPs} / \text{bytes moved}$
 ridge point = peak FLOP/s / peak bandwidth
 Geometric mean for ratios; harmonic mean for rates

Memory hierarchy

AMAT $= \text{hit time} + \text{miss rate} \times \text{miss penalty}$
 Multilevel = $t_1 + m_1 (t_2 + m_2 (t_3 + m_3 \times t_{\text{DRAM}}))$ (local miss rates)
 Cache address split:
 offset bits = $\log_2(\text{line size})$
 index bits = $\log_2(\text{size} / (\text{line size} \times \text{associativity}))$
 tag bits = address bits - index - offset
 TLB reach = entries $\times$ page size
 Three Cs compulsory / capacity / conflict (+ coherence)
 DRAM row hit = CL; row empty = $t_{\text{RCD}} + \text{CL}$;
 row conflict = $t_{\text{RP}} + t_{\text{RCD}} + \text{CL}$
 Bandwidth = transfers/s $\times$ bus width $\times$ channels

Logic, timing and power

```

T_clk    >= t_cq + t_logic(max) + t_setup + t_skew
hold:    t_cq + t_logic(min) >= t_hold + t_skew
f_max    = 1 / T_clk
MTBF     = exp(t_r / tau) / (T0 x f_clk x f_data)

P_dynamic = alpha C V^2 f      E_per_op = alpha C V^2
P_static  = V x I_leak

DVFS      scaling V and f by k scales dynamic power by k^3
T_junction = T_ambient + P x theta_JA

```

Number representation

```

Two's complement n bits: -2^(n-1) .. 2^(n-1) - 1
Negate: invert and add 1.  Sign-extend: replicate the top bit.

IEEE-754 binary32: 1 sign | 8 exponent (bias 127) | 23 fraction
value = (-1)^s x 1.f x 2^(e - 127)
binary64: 1 | 11 (bias 1023) | 52
e = 0, f = 0 -> zero;  e = 0, f != 0 -> subnormal
e = all 1s, f = 0 -> infinity;  otherwise -> NaN
Qm.n fixed point: value = integer / 2^n

```

RV32I encoding

Format	Layout (bit 31 down to bit 0)	Used by
R	funct7[7] rs2[5] rs1[5] funct3[3] rd[5] opcode[7]	add sub and or xor sll srl sra slt
I	imm[12] rs1[5] funct3[3] rd[5] opcode[7]	addi lw jalr slli and CSR access
S	imm[11:5] rs2[5] rs1[5] funct3[3] imm[4:0] opcode[7]	sb sh sw
B	imm[12 10:5] rs2 rs1 funct3 imm[4:1 11] opcode	beq bne blt bge bltu bgeu
U	imm[31:12] rd[5] opcode[7]	lui auipc
J	imm[20 10:1 11 19:12] rd[5] opcode[7]	jal

Opcode	Value	Meaning
0110011	0x33	R-type arithmetic
0010011	0x13	I-type arithmetic (addi etc.)
0000011	0x03	Loads
0100011	0x23	Stores
1100011	0x63	Branches
1101111	0x6F	jal
1100111	0x67	jalr

Opcode	Value	Meaning
0110111 / 0010111	0x37 / 0x17	lui / auipc

Latency and energy, orders of magnitude

Operation	Latency	Energy (order)
Integer add	1 cycle	~0.1 pJ
Register file read	1 cycle	~0.1 pJ
L1 cache hit	4 cycles	~5 pJ
L2 hit	12-20 cycles	~20 pJ
L3 hit	40-60 cycles	~50 pJ
DRAM access	200-350 cycles (60-100 ns)	~50-200 pJ
Branch misprediction	12-20 cycles	-
Atomic, contended	100-500 cycles	-
System call	150-450 cycles	-
Context switch	1,000-10,000 cycles	-
NVMe SSD read	20-100 us	-
Network round trip (datacentre)	50-500 us	-

Coherence and consistency

Item	Summary
MESI	Modified / Exclusive / Shared / Invalid; at most one writer or many readers
False sharing	Two variables in one line, written by two cores: 100-300 cycles per access instead of ~1
x86-64 (TSO)	Only store-then-load may be reordered
ARM / RISC-V	Almost anything may be reordered; use acquire/release
Acquire/release	A release store publishes everything before it to any acquire load that reads it

Appendix B. Glossary

Term	Definition
ABI	The binary conventions - argument registers, stack layout, symbol naming - that let separately compiled code interoperate.
AMAT	Average memory access time: hit time plus miss rate times miss penalty.
Amdahl's law	Speedup is limited by the fraction of work left unimproved.
Arithmetic intensity	FLOPs performed per byte moved from memory; decides where a kernel sits on the roofline.
Associativity	How many places in a cache a given address may occupy.
AXI	The AMBA interconnect protocol with five independent channels and multiple outstanding transactions.
Branch target buffer	A cache of branch targets, consulted during fetch so the next address is available immediately.
Cache line	The unit of transfer between cache levels, typically 64 bytes.
Chiplet	A separately manufactured die combined with others in one package.
Clock gating	Stopping the clock to an idle block to save dynamic power.
CMOS	Complementary MOS logic: an NMOS pull-down network and a PMOS pull-up network, so no static current flows.
Coherence	The hardware guarantee that all caches agree about the value of each memory location.
Consistency model	The architectural rules about which memory orderings software may observe.
CPI	Cycles per instruction; its reciprocal is IPC.
Critical path	The longest logic path between two registers; it sets the maximum clock frequency.
Dark silicon	Transistors that cannot all be switched at full speed within the power budget.
Dennard scaling	The historical property that shrinking transistors kept power density constant; it ended around 2005.
Directory protocol	Coherence implemented with a record of which caches hold each line, instead of broadcast snooping.
DMA	A controller that moves data between memory and peripherals without the processor.
DVFS	Dynamic voltage and frequency scaling.
ECC	Error-correcting code; SECDED corrects one bit and detects two.
False sharing	Two independent variables sharing a cache line, causing coherence traffic on every write.
FIT	Failures in time: failures per billion device-hours.
Flip-flop	An edge-triggered one-bit storage element; the basic unit of sequential logic.
FMA	Fused multiply-add: $a \times b + c$ computed with a single rounding.
Forwarding	Routing a result directly from where it is produced to where it is needed, avoiding a pipeline stall.
Harvard architecture	Separate instruction and data memories or ports.
Hazard	A situation in which the next instruction cannot execute in the following cycle: structural, data or control.
Iron law	$\text{Time} = \text{instructions} \times \text{CPI} \times \text{clock period}$.
ISA	Instruction set architecture: the hardware/software contract.
Little's law	$\text{Outstanding requests} = \text{throughput} \times \text{latency}$.
Locality	The tendency of programs to reuse recent data (temporal) and nearby data (spatial).
MESI	The four cache-line states used by most coherence protocols.

Term	Definition
Metastability	A flip-flop's unbounded settling time when its input changes during the setup/hold window.
Microarchitecture	The implementation of an ISA - pipeline, caches, predictors - none of which is visible in the contract.
MLP	Memory-level parallelism: how many independent misses are outstanding at once.
MMU / MPU	Memory management unit (translation plus protection) versus memory protection unit (protection only).
NUMA	Non-uniform memory access: memory attached to another socket is slower and narrower.
Out-of-order execution	Executing instructions as their operands become ready, then retiring them in program order.
Page table	The tree that maps virtual pages to physical frames; walked on a TLB miss.
Pipelining	Overlapping the stages of successive instructions to raise throughput without lowering latency.
Precise exception	An exception delivered such that all earlier instructions completed and no later one had any effect.
Prefetcher	Hardware that predicts and fetches future addresses before they are requested.
Register renaming	Mapping architectural registers onto a larger physical set to remove WAR and WAW dependences.
Reorder buffer	The structure that tracks in-flight instructions and retires them in order.
RISC-V	An open, modular instruction set architecture; RV32I is its 32-bit integer base.
Roofline	A model bounding attainable performance by peak compute and by bandwidth times arithmetic intensity.
Scratchpad	Software-managed on-chip memory at a fixed address, used instead of a cache where determinism matters.
Setup and hold time	How long data must be stable before and after a clock edge.
SIMT	The GPU execution model: one instruction stream driving many lanes, with masks for divergence.
SMT	Simultaneous multithreading: several threads sharing one core's execution resources.
Snooping	Coherence implemented by every cache observing a shared broadcast.
Speculation	Executing instructions before it is known whether they should run, and undoing them if not.
Store buffer	A queue holding stores until retirement; the reason x86's TSO allows store-to-load reordering.
Systolic array	A grid of MAC cells through which data flows, maximising reuse per byte fetched.
TLB	Translation lookaside buffer: a cache of virtual-to-physical translations.
TSO	Total store order: x86's memory model, in which only store-then-load may be reordered.
Two's complement	The standard signed integer representation, in which addition is identical to unsigned addition.
VLIW	Very long instruction word: the compiler packs independent operations into a bundle issued every cycle.
von Neumann	An organisation in which instructions and data share one memory.
Wait state	An extra cycle inserted while a slow memory (usually flash) responds.
Write-back / write-through	Whether a write updates only the cache (marking it dirty) or both levels immediately.

Appendix C. Study Roadmap, Projects and Tools

A 16-week study plan

Weeks	Focus	Deliverable
1	Chapters 1-3: layers, logic, timing	Simulate a small FSM; compute <code>f_max</code> for a stage
2	Chapters 4-5: arithmetic and memory cells	An ALU in Verilog; IEEE-754 encoding by hand
3-4	Chapters 6-8: ISA, RV32I, the ABI	Assemble and decode instructions by hand; write a leaf function in assembly
5	Chapters 9-10: other ISAs, exceptions and privilege	A trap handler; compare compiler output across ISAs
6-7	Chapters 11-12: datapath and pipelining	A single-cycle RV32I core in RTL, simulated against tests
8	Chapter 13: branch prediction	Simulate 1-bit, 2-bit and gshare predictors on a real trace
9-10	Chapters 14-16: ILP, out-of-order, SIMD and threads	Vectorise a kernel; measure IPC and scaling
11	Chapter 17: performance measurement	A roofline plot for your machine and three kernels
12-13	Chapters 18-21: caches, virtual memory, DRAM	A cache simulator validated against hardware counters
14	Chapters 22-23: storage, coherence and consistency	Reproduce false sharing and a memory-model litmus test
15	Chapters 24-29: real devices	Compare an MCU and an application processor on one task
16	Chapters 30-38: power, security, accelerators, RTL, practice	Pipeline your CPU; optimise one kernel through the six steps of Chapter 37

Projects, in increasing order of ambition

- **Encode and decode instructions by hand**, then write an assembler for a dozen RV32I instructions.
- **Write a functional simulator** for RV32I - about 400 lines - and run a compiled C program on it.
- **Build a single-cycle CPU in RTL** and run it on an FPGA.
- **Pipeline that CPU**, adding forwarding, hazard detection and a branch predictor. Measure the CPI change on the same programs.
- **Write a cache simulator** with configurable size, associativity and line size; reproduce the three Cs experimentally.
- **Build a roofline for your own machine** by measuring peak FLOP/s and bandwidth, then place ten kernels on it.
- **Optimise a real application** with counters and the checklist of Chapter 37, recording the gain from each step.
- **Add a custom instruction** to a RISC-V softcore and measure the speedup on a kernel that uses it - the whole book, end to end.

Tools worth knowing

Tool	For
<code>perf</code> , VTune, ARM Streamline	Performance counters and profiling
Compiler Explorer	Seeing what a compiler emits, instantly, for any ISA
<code>objdump</code> , <code>readelf</code> , <code>nm</code>	Inspecting binaries
Verilator, Icarus, GTKWave	Free RTL simulation and waveform viewing

Tool	For
Yosys plus nextpnr, or a vendor FPGA toolchain	Synthesis and implementation
Spike, QEMU, gem5	Functional and cycle-level simulation
cachegrind, DynamoRIO	Cache and instruction analysis
A logic analyser and an oscilloscope	The only way to see what a real embedded system does

How to keep learning

- **Read the optimisation manual** for a processor you use - they document the microarchitecture in far more detail than any textbook.
- **Read the ISA specification** rather than a tutorial, once you can. The RISC-V manuals are short and unusually readable.
- **Reproduce a paper's experiment** from ISCA, MICRO or ASPLOS; the gap between the claim and your measurement is where the learning is.
- **Measure something every week** - IPC, a cache miss rate, a joule - until the numbers in this book's tables feel like familiar quantities rather than trivia.